FIXED INCOME – Commented Study Notes

Conceptual edition from the 2026 course material

20 October 2026

This study edition integrates the course notes by concept, retaining substantive definitions, assumptions, figures, calculations and supplied questions. Distinctive recorded explanations appear beside the relevant concepts. Theory first developed during the coding lessons is integrated in Part I; Part II contains the practical work, the original notebook, eight linear-product notebooks, modules, data and recorded outputs, with internal references to their theoretical foundations. The syllabus and references remain in the appendix.

The automatic transcripts contain unclear passages. Only intelligible conceptual explanations checked against the written material are integrated; uncertain speech is retained in the separate transcripts. Original manuscript visuals remain available in the separate source-faithful Notes; their pricing development is integrated here in LaTeX.

Contents

I Theory **4**

1 Contracts, discounting and contractual dates **4**
- 1.1 Why Fixed-Income Primitives Matter . . . 4
- 1.2 Zero-Coupon Contracts and Spot Discount Factors . . . 4
- 1.3 Dates, Calendars, and Contractual Conventions . . . 7

2 Spot quotations and term structures **8**
- 2.1 From Spot Discount Factors to Spot Rates . . . 8
- 2.2 The Spot Term Structure . . . 10

3 No-arbitrage replication and forward rates **12**
- 3.1 Same Cash Flows, Same Price . . . 12
- 3.2 Price Ratios and Relative Discounting . . . 13
- 3.3 From Forward Discount Factors to Forward Rates . . . 14
- 3.4 The Forward Term Structure . . . 16

4 Benchmark rates: from a representation to a fixing **17**
- 4.1 Two rates enter a derivative price . . . 17
- 4.2 Fixing, accrual and payment are three different dates . . . 18
- 4.3 Compounding overnight fixings in arrears . . . 18
- 4.4 Prof notes: benchmark questions . . . 18

5 Risk-neutral pricing revisited: bank account and numeraire **18**
- 5.1 Information and the bank account . . . 18
- 5.2 Discounted prices are martingales under the pricing measure . . . 19
- 5.3 Why interest-rate claims are difficult . . . 19
- 5.4 Changing the numeraire . . . 19
- 5.5 Prof notes: a zero coupon is the simple rate-dependent case . . . 19

6 Change of numeraire **20**
- 6.1 The price is invariant; the expectation changes . . . 20

6.2 The density and the pricing theorem . . . 20
6.3 Proof using conditional Bayes' formula . . . 20
6.4 Moving between two numeraires . . . 21
6.5 General payoffs, intermediate payments and uniqueness . . . 21
6.6 Money-market, forward and swap measures . . . 21
6.7 Prof notes: a changed expectation needs changed dynamics . . . 22
6.8 Multiple-choice activities and supplied solutions . . . 22

7 Plain vanilla interest-rate products: a cash-flow map 23
7.1 Linear products . . . 23
7.2 Products with optionality . . . 24
7.3 Beyond plain vanilla . . . 24
7.4 Prof notes: the pricing programme . . . 24

8 Discounting, floating coupons and forward rate agreements 24
8.1 One curve or several curves? . . . 24
8.2 A simple forward rate is a forward-measure martingale . . . 25
8.3 A floating coupon: fixing date, payment date and value . . . 26
8.4 Money-market instruments and curve construction . . . 27
8.5 Forward rate agreements and their settlement convention . . . 28
8.6 Hedging a future borrower . . . 28
8.7 A swap is a portfolio of period exchanges . . . 29

9 Laboratory foundations: dates, curve validation and settlement pricing 30
9.1 Dates and schedules in the laboratory . . . 30
9.2 The market-curve convention audit . . . 30
9.3 A synthetic curve with an explicit date interface . . . 31
9.4 Settlement values, dirty prices and accrued interest . . . 31
9.5 Observed fixings, spreads and floating-rate identities . . . 32
9.6 From market quotes to risk and model-based valuation . . . 32

10 Activities with worked reasoning 33
10.1 Supplied questions and generated study answers . . . 33
10.2 Activity 1 — Spot or forward? . . . 33
10.3 Activity 2 — One price, several rate quotations . . . 33
10.4 Activity 3 — Complete the replicating strategy . . . 34
10.5 Activity 4 — From spot prices to a forward rate . . . 34
10.6 Activity 5 — Read the term structures . . . 34
10.7 Optional AI-supported discussion . . . 35

II Code 35

11 Notebook 01: practical work on dates, conventions and market curves 35
11.1 Practical objectives . . . 35
11.2 Cycle 1: dates and calendar conventions . . . 36
11.3 Cycle 2: day counts and compounding . . . 38
11.4 Cycle 3: coupon schedules . . . 40
11.5 Cycle 4: reading and validating a market curve . . . 41
11.6 Suggested experiments . . . 45

12 Linear-product laboratory: original student notebooks **46**
12.1 00 library architecture . . . 46
12.2 Our fixed-income library: what do we want? . . . 46
12.3 01 python setup and syntax . . . 47
12.4 Python and financial conventions: lab reference notes . . . 47
12.5 main irc . . . 50
12.6 build get schedule dataframe . . . 51
12.7 build get price cb . . . 52
12.8 build get price frn . . . 53
12.9 build get price swap . . . 54
12.10 test library . . . 56
12.11 Original reference modules and synthetic input . . . 57

13 Function-design discussion and API reference **63**
13.1 From a checked notebook calculation to a reusable function . . . 63
13.2 Interfaces already supplied . . . 64
13.3 Proposed later responsibilities . . . 64

14 Practical support and source-data workflow **65**
14.1 Classroom workflow . . . 65
14.2 LSEG export workflow supplied for classroom verification . . . 65
14.3 NumPy array syntax . . . 65
14.4 Pandas syntax . . . 66
14.5 Jupyter: two modes and recovery . . . 66

A Course roadmap **68**
A.1 Computational applications and homework . . . 68
A.2 Assessment . . . 68
A.3 References . . . 68

Part I

Theory

1 Contracts, discounting and contractual dates

Key idea

Contract and cash-flow timeline → discount-factor price → rate quotation. A calendar controls which dates are valid; a day count controls the accrued amount. These are different inputs.

1.1 Why Fixed-Income Primitives Matter

Fixed-income securities are built from dated exchanges of money. The first task is therefore to identify the contract and its timeline; only afterwards do we choose a rate convention with which to represent its price.

This chapter is organised around a fundamental distinction between two families of contracts.

1.1.1 Spot contracts

A spot zero-coupon transaction agreed at time t exchanges an amount $P(t,T)$ at t for one unit of currency at the future date T. Its relevant interval is

$$[t,T].$$

The price $P(t,T)$ is the spot discount factor. After the dates, day-count rule, and compounding convention have been specified, the same price can be represented by a spot interest rate.

1.1.2 Forward contracts

A forward contract is agreed at time t, but the exchange to which it refers takes place at a future date. In the fixed-income application developed here, a zero-coupon bond maturing at T_2 is delivered at T_1 , with

$$t \leq T_1 < T_2.$$

The valuation date is t, whereas the relevant future interval is

$$[T_1,T_2].$$

No-arbitrage links this contract to spot zero-coupon bonds. The resulting price ratio defines a forward discount factor; only after the quotation conventions have been fixed is that price represented by a forward interest rate.

1.1.3 Prices before rates

The distinction between spot and forward is first a distinction between contracts and timelines, not between two formulas for interest rates. The logical order followed in this chapter is therefore

$$\text{contract} \longrightarrow \text{discount factor} \longrightarrow \text{rate representation}.$$

This ordering also clarifies a recurring source of error: discount factors are prices, whereas interest rates are convention-dependent representations of those prices.

1.2 Zero-Coupon Contracts and Spot Discount Factors

The basic building block of fixed-income valuation is a payment of one unit of currency at a specified future date. Its present value is represented by a zero-coupon bond price, or equivalently by a discount factor.

1.2.1 Definition

For $0 \leq t \leq T$, let

$$P(t, T) \tag{1}$$

denote the time-t value of a contract that pays one unit of currency at time T, with no intermediate payments. The date T is the maturity and T-t is the time to maturity when time is measured in years. By definition,

$$P(T, T) = 1. \tag{2}$$

For a notional amount N, the corresponding zero-coupon payment has value

$$V(t) = NP(t, T). \tag{3}$$

The notation $P(t, T)$ contains two distinct time arguments. The first, t, is the date at which the price is observed; the second, T, is the date at which the unit payment is made. When the valuation date is fixed, the map

$$T \longmapsto P(t, T), \quad T \geq t, \tag{4}$$

is called the discount function or discount curve at time t.

Worked example

Prof notes: normalising the payoff. A bond costing 99 today and paying 100 in one year is 100 units of a bond priced $P(0, 1) = 0.99$ and paying one. A payment of 200 is therefore worth $200(0.99) = 198$ today; conversely, 198 accumulates to $198/0.99 = 200$. The source's OIS example compares discount curves observed on different valuation dates.

1.2.2 Zero-coupon bond price and discount factor

The expressions zero-coupon bond price and discount factor are often used interchangeably, but they emphasise different interpretations:

- $P(t, T)$ is a price when it refers to a traded or idealised zero-coupon security;
- $P(t, T)$ is a discount factor when it is used as the multiplier that converts a cash flow at T into a value at t.

In a modern collateralised framework, the relevant discount factors depend on the currency and collateral agreement. The notation $P(t, T)$ should therefore be understood relative to a specified discounting framework. This dependence will become explicit when curves are constructed; it is suppressed here to keep the notation simple.

1.2.3 Basic properties

In the idealised setting used in this chapter, zero-coupon payments are free of default and denominated in a fixed currency. Absence of arbitrage implies

$$P(t, T) > 0, \quad t \leq T. \tag{5}$$

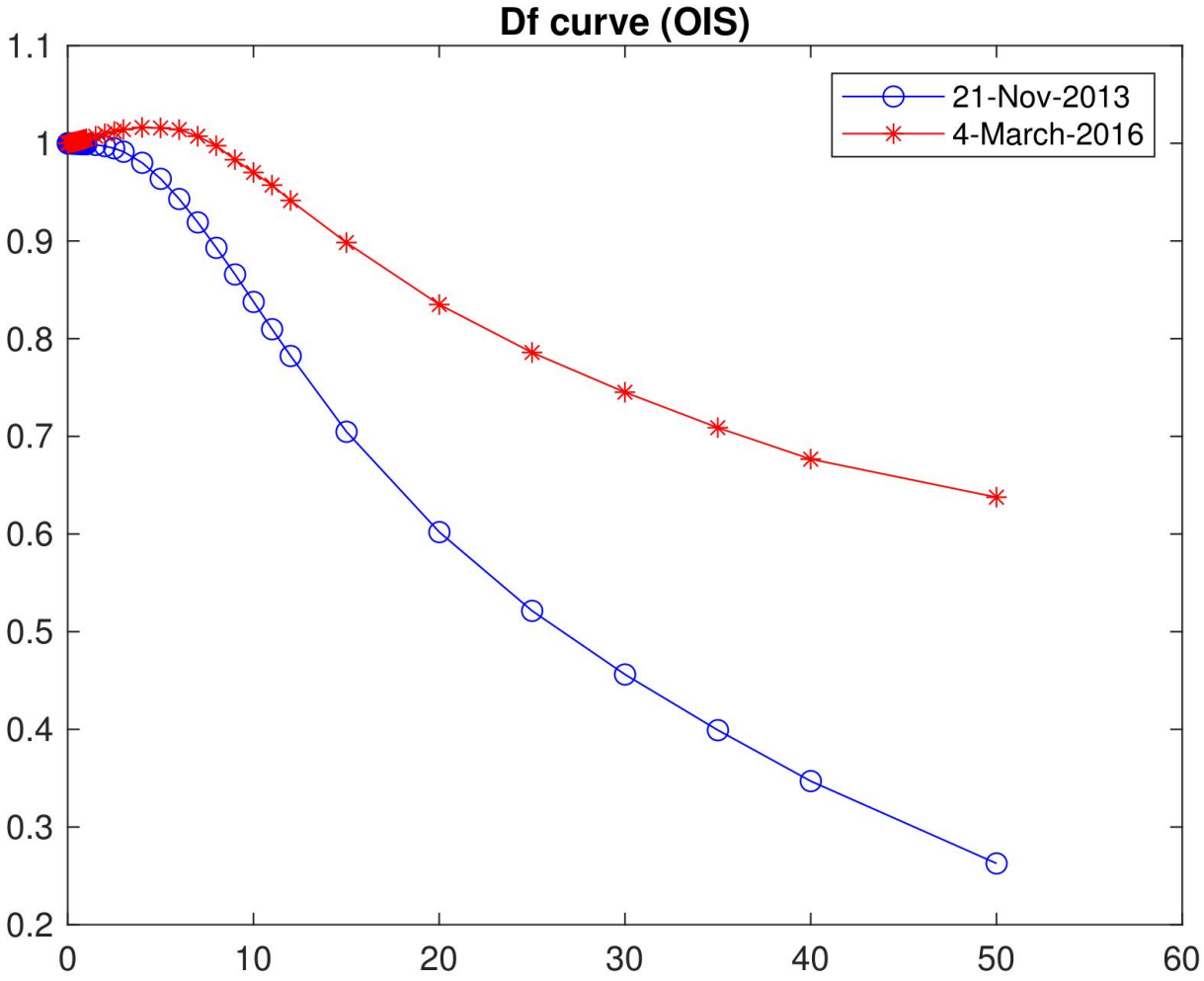


Figure 1: The discount curve at valuation date t.

A non-positive price would allow a strictly positive future payoff to be acquired without a positive initial cost.

It is common to impose

$$P(t,T) \leq 1, \quad t < T, \tag{6}$$

and to assume that $T \mapsto P(t,T)$ is non-increasing. These properties are consistent with non-negative interest rates, but they are not consequences of no arbitrage alone. If interest rates are negative over some maturities, a discount factor may exceed one and the discount curve may increase locally. The universally relevant no-arbitrage restriction in this elementary setting is positivity, not monotonicity.

1.2.4 Present and future values

If C_T is a deterministic payment at time T, its value at time t is

$$\mathrm{PV}_t(C_T) = C_T P(t,T). \tag{7}$$

Conversely, provided $P(t,T) > 0$, the time-T amount equivalent to a value $V(t)$ at time t is

$$\mathrm{FV}_T(V(t)) = \frac{V(t)}{P(t,T)}. \tag{8}$$

Thus $P(t,T)$ is the discount factor from T back to t, whereas $1/P(t,T)$ is the accumulation factor from t to T.

For deterministic cash flows C_i paid at dates T_i , value additivity gives

$$V(t) = \sum_{i=1}^{n} C_i P(t,T_i). \tag{9}$$

This formula can be interpreted as a static replication: each cash flow is replicated by C_i units of the zero-coupon bond maturing at T_i . It is the central valuation formula for deterministic fixed-income cash flows.

1.3 Dates, Calendars, and Contractual Conventions

A fixed-income contract is defined by dated cash flows. Before a rate can be computed or a cash flow discounted, calendar dates must therefore be converted into economically meaningful time intervals. This conversion is not unique: it depends on contractual and market conventions.

It is useful to keep three operations separate:

1. generate the contractual dates of the instrument;
2. adjust those dates when they are not business days;
3. compute year fractions between the relevant dates.

Changing any of these operations may change an accrual amount or a present value, even if the quoted interest rate is unchanged.

1.3.1 The relevant dates

Let t denote the valuation date. Depending on the contract, the following dates may also be required:

- the trade date, on which the transaction is agreed;
- the settlement date, on which the transaction becomes effective and the initial exchange, if any, takes place;
- the accrual start date and accrual end date, which define the interval over which interest accrues;
- the payment date, on which the corresponding cash flow is paid;
- the maturity date, which marks the contractual end of the instrument.

Dates are calendar objects. A year fraction is not a direct subtraction of calendar labels; direct subtraction is appropriate only for endpoints already expressed in compatible numerical time units.

These labels need not identify the same calendar date. In particular, an accrual end date and its payment date may differ. The accrual factor must be computed from the dates specified by the contract, whereas discounting uses the date on which the cash flow is actually paid.

The time between the valuation date and a future payment date is also distinct from the accrual fraction of the associated interest period. Following the notation used throughout these notes, for the accrual period $[T_{i-1}, T_i]$ define its length in years as

$$\tau_i = \tau(T_{i-1}, T_i). \tag{10}$$

This accrual factor determines the interest amount. If the corresponding cash flow is paid at T_i^p , its present value uses the discount factor $P(t, T_i^p)$. Thus accrual dates determine the cash-flow amount, whereas the payment date determines which discount factor is used.

1.3.2 Business-day calendars

A business-day calendar specifies the dates on which payments and settlements can occur in the relevant market. Weekends alone are not sufficient: market holidays depend on the currency, financial centre, and contract.

Let D be an unadjusted contractual date. A business-day convention maps D into an adjusted date D^{adj} . Common conventions include:

Following	move to the first following business day;
Modified Following	move to the first following business day, unless this moves the date into the next calendar month; in that case, move to the first preceding business day;
Preceding	move to the first preceding business day;
Modified Preceding	move to the first preceding business day, unless this moves the date into the previous calendar month; in that case, move to the first following business day;
Unadjusted	leave the contractual date unchanged.

The convention is part of the contract. It should not be selected after the cash flows have been generated merely to obtain convenient dates. Moreover, the convention applied to the final maturity date may differ from the one applied to intermediate dates.

1.3.3 Day-count conventions

A day-count convention defines the year fraction

$$\tau(T_1, T_2) \tag{11}$$

associated with the calendar interval $[T_1, T_2]$. This quantity enters interest accruals and the conversion between discount factors and quoted rates. It is therefore part of the definition of the rate, not a cosmetic detail.

For example, under an Actual/360 convention,

$$\tau(T_1, T_2) = \frac{\text{actual number of days from} T_1 \text{to} T_2}{360}, \tag{12}$$

whereas under Actual/365 Fixed the denominator is 365. Conventions in the 30/360 family replace the actual calendar-day count with a rule-based count in which months are treated as having 30 days. Actual/Actual conventions use the actual number of days but determine the denominator by a convention-specific treatment of calendar years and coupon periods.

Names such as "30/360" and "Actual/Actual" identify families rather than always identifying a unique algorithm. The precise market variant must be specified whenever it affects a calculation.

1.3.4 Payment schedules

A payment schedule is an ordered sequence of dates

$$T_0 < T_1 < \cdots < T_n, \tag{13}$$

together with the conventions used to generate and adjust them. Its construction normally requires:

- an effective or settlement date and a termination date;
- a tenor, such as three or six months;
- a calendar and business-day convention;
- a date-generation rule;
- an end-of-month rule, when applicable.

Dates may be generated forward from the effective date or backward from the termination date. If the contractual horizon is not an integer multiple of the tenor, the schedule contains a short or long first or last period, called a stub. The generation direction therefore matters: it determines where an irregular period appears.

One must also distinguish between unadjusted schedule dates and adjusted payment dates. Adjusting each date independently can make consecutive accrual periods appear irregular even when the underlying contractual schedule is regular. A robust implementation should preserve both sets of dates and apply the appropriate day-count rule to the dates specified in the contract.

The practical construction of a schedule, its accrual factors, and its payment times will be developed separately in the companion notebook.

Worked example

Original pedagogical example: 90 actual days give $90/360 = 0.25$ under Actual/360 and $90/365 \approx 0.2466$ under Actual/365 Fixed. A fixed quoted rate therefore accrues different interest amounts. For a coupon $C_i = NL_i\tau_i$ paid at T_i^p, its value is $NL_i\tau_i P(t, T_i^p)$: the accrual end date need not equal the payment date.

Key idea

Before inserting any number into a rate formula, write down *which date generates the amount* and *which date receives the payment*. Keep unadjusted schedule dates, adjusted payment dates and year fractions separately.

2 Spot quotations and term structures

2.1 From Spot Discount Factors to Spot Rates

A spot discount factor is a price. To represent that price by an interest rate, one must specify:

start date, end date, day-count convention, compounding convention. (14)

For a spot contract, the start date is the valuation date t, the end date is the maturity T, and

$$\tau = \tau(t, T) > 0 \tag{15}$$

is the corresponding year fraction. Once τ and the compounding convention have been fixed, there is a one-to-one correspondence between $P(t, T)$ and the associated spot rate.

2.1.1 Simple compounding

Let $L(t,T)$ denote the simply compounded spot rate for the interval $[t,T]$. Investing one unit at time t produces

$$1 + L(t,T)\tau(t,T) \tag{16}$$

at time T. Therefore,

$$P(t,T) = \frac{1}{1 + L(t,T)\tau(t,T)}, \tag{17}$$

and conversely

$$L(t,T) = \frac{1}{\tau(t,T)}\left(\frac{1}{P(t,T)} - 1\right). \tag{18}$$

The admissibility condition is $1 + L(t,T)\tau(t,T) > 0$, which is equivalent to the positivity of the discount factor.

Under the no-compounding assumption, interest over a single accrual period is proportional to the length of that period. If an annualised simply compounded rate L_i applies to $[T_{i-1}, T_i]$, the interest on notional N is

$$C_i = NL_i\tau_i, \quad \tau_i = \tau(T_{i-1}, T_i). \tag{19}$$

If this amount is paid at T_i^p , its value at time t is

$$V_i(t) = NL_i\tau_i P(t, T_i^p). \tag{20}$$

The accrual dates determine the amount of interest, whereas the payment date determines the discount factor.

2.1.2 Annual compounding

Let $Y(t,T)$ denote an annually compounded spot rate. The accumulation factor over $\tau(t,T)$ years is $(1 + Y(t,T))^{\tau(t,T)}$. Hence

$$P(t,T) = (1 + Y(t,T))^{-\tau(t,T)}, \tag{21}$$

$$Y(t,T) = P(t,T)^{-1/\tau(t,T)} - 1. \tag{22}$$

2.1.3 Compounding m times per year

Let $Y_m(t,T)$ denote the nominal annual spot rate compounded m times per year. The rate applied in each compounding period is $Y_m(t,T)/m$, giving

$$P(t,T) = \left(1 + \frac{Y_m(t,T)}{m}\right)^{-m\tau(t,T)}, \tag{23}$$

$$Y_m(t,T) = m\left(P(t,T)^{-1/[m\tau(t,T)]} - 1\right). \tag{24}$$

The quoted quantity Y_m is a nominal annual rate; it is not the rate applied in each subperiod.

2.1.4 Continuous compounding

Let $R(t,T)$ denote the continuously compounded spot rate. Then

$$P(t,T) = \exp(-R(t,T)\tau(t,T)), \tag{25}$$

$$R(t,T) = -\frac{\log P(t,T)}{\tau(t,T)}. \tag{26}$$

Continuous compounding is particularly convenient analytically because products of discount factors become sums of logarithms.

2.1.5 Equivalent rate quotations

Rates quoted under different conventions are equivalent when they generate the same discount factor for the same dates and day-count convention. Thus,

$$\frac{1}{1+L\tau} = (1+Y)^{-\tau} = \left(1+\frac{Y_m}{m}\right)^{-m\tau} = e^{-R\tau}. \tag{27}$$

The numerical rates L, Y, Y_m , and R are generally different. Comparing them without first aligning dates, day count, and compounding convention is meaningless.

For example, if $P(t,T) = 0.95$ and $\tau(t,T) = 2$, the equivalent annualised rates are approximately

Quotation convention	Equivalent rate
Simple	2.6316%
Annual	2.5978%
Semi-annual nominal	2.5812%
Continuous	2.5647%

All four quotations describe exactly the same price.

For a \$100 maturity payment, all four quotations give a current value of \$95. A smaller numerical rate quote alone does not identify a cheaper price.

Prof notes: a second price. For the handwritten example $P(0,2) = 0.94$ and $\tau = 2$, substitute the new price into the same four quotation rules:

$$\begin{aligned} 0.94(1+2L) &= 1, & L &= \tfrac{1}{2}(1/0.94-1),\\ 0.94(1+Y)^2 &= 1, & Y &= (1/0.94)^{1/2}-1,\\ 0.94(1+Y_2/2)^4 &= 1, & Y_2 &= 2[(1/0.94)^{1/4}-1],\\ 0.94e^{2R} &= 1, & R &= -\log(0.94)/2. \end{aligned}$$

Continuous compounding is the limit as the number of compounding periods tends to infinity.

Key idea

Prices are primary. If two rate quotes appear to disagree, convert both to $P(t,T)$ using the same contract dates before diagnosing an opportunity.

2.2 The Spot Term Structure

Fix a valuation date t. The family of spot discount factors

$$\{P(t,T) : T \geq t\} \tag{28}$$

defines the discount term structure at time t. Choosing a day-count and a compounding convention transforms it into the corresponding family of spot rates

$$\{R(t,T) : T \geq t\}, \tag{29}$$

usually called the zero-rate curve or spot-rate curve.

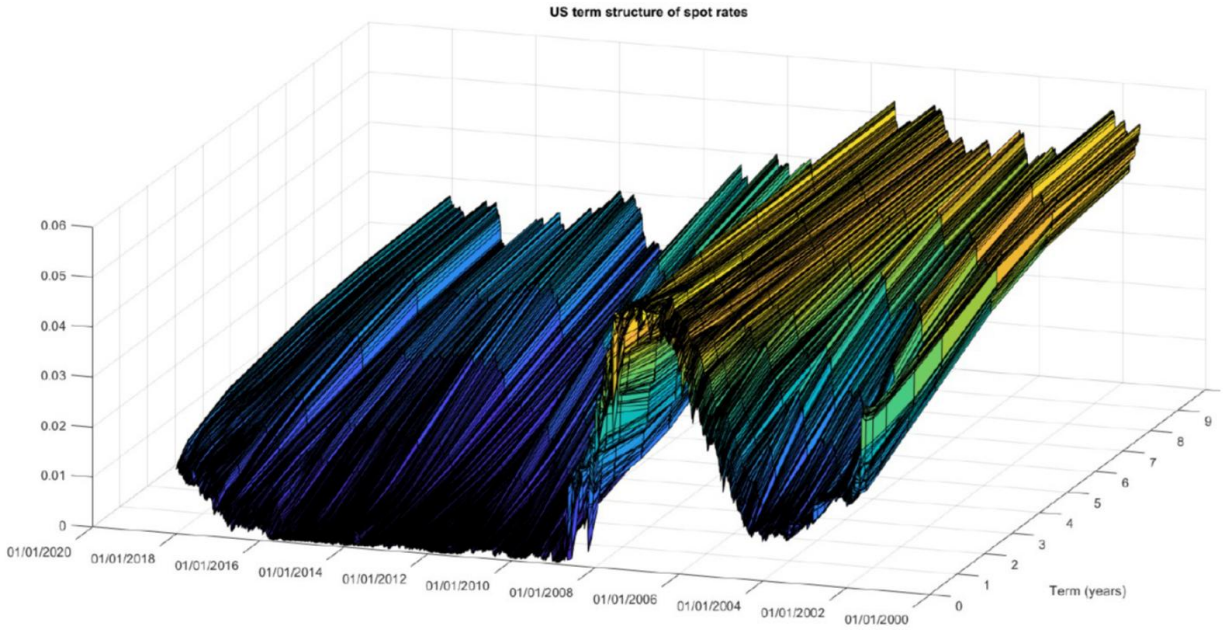


Figure 2: The term structure of spot interest rates at valuation date t.

The discount curve and a zero-rate curve contain the same information once the quotation convention is known. The discount curve is a price object and does not change when the rate convention changes. The numerical zero-rate curve does change. Consequently, two zero-rate curves should not be compared unless their day-count and compounding conventions are aligned.

2.2.1 Cross-section and evolution through time

The term structure is a cross-section across maturities at a fixed valuation date:

$$T \longmapsto P(t,T) \quad \text{or} \quad T \longmapsto R(t,T). \tag{30}$$

The supplied three-dimensional historical zero-rate plot adds a valuation-date axis: it is a sequence of curves, not a single curve with two maturity arguments. This should be distinguished from the evolution through calendar time of a fixed-maturity price or rate, obtained by varying t. Interest-rate models must ultimately describe this evolution, but the present chapter concentrates on the cross-sectional objects observed at one valuation date.

2.2.2 Curve shapes and terminology

A spot-rate curve may be increasing, approximately flat, decreasing, or non-monotone. The labels normal and inverted commonly refer to increasing and decreasing rate curves, respectively. These descriptions concern the selected rate representation; they should not be transferred mechanically to the discount curve. In particular, negative rates may produce discount factors greater than one and locally increasing discount curves.

The expression yield curve is sometimes used loosely for several different objects, including zero rates, par rates, and bond yields. In these notes, spot term structure means the zero-coupon term structure derived from $P(t,T)$. Par yields and yields to maturity are distinct quantities and will not be used as substitutes for spot rates.

2.2.3 Instantaneous spot rate and short rate

The continuously compounded spot rate $R(t,T)$ refers to the finite interval $[t,T]$. If the relevant limit exists, the instantaneous spot rate is defined by

$$r(t) := \lim_{T \downarrow t} R(t,T). \tag{31}$$

This quantity is called the short rate. It is the rate, observed at time t, for an infinitesimal spot contract beginning at t. When the discount curve is differentiable in maturity,

$$r(t) = -\left.\frac{\partial}{\partial T} \log P(t,T)\right|_{T=t}. \tag{32}$$

To obtain the derivative explicitly, let $\Delta > 0$ be a small interval expressed in the same numerical time units as the continuous rate. The finite-period identity is

$$P(t, t+\Delta) = e^{-R(t,t+\Delta)\Delta}, \qquad R(t, t+\Delta) = -\frac{\log P(t, t+\Delta)}{\Delta}.$$

Since $P(t,t) = 1$ and hence $\log P(t,t) = 0$, its limit is the negative right maturity derivative:

$$r(t) = -\lim_{\Delta \downarrow 0} \frac{\log P(t, t+\Delta) - \log P(t,t)}{\Delta}.$$

The valuation date stays fixed while maturity approaches it. Differentiation of a smooth discount curve therefore recovers the short-end rate; it does not assert that the observed finite overnight fixing is literally infinitesimal.

The notation $r(t)$ will be retained when the short rate becomes a stochastic process in the interest-rate models developed later. Unlike a rate associated with a finite market accrual period, the short rate is an infinitesimal mathematical object. It is especially useful as the state variable in short-rate models, in which the dynamics of $r(t)$ are specified and the prices of zero-coupon bonds are then derived from those dynamics.

The general foundations of no-arbitrage pricing are developed in the Derivatives course. Here we shall use an elementary fixed-income argument based on the law of one price and replicating portfolios. Its purpose is to show how spot zero-coupon bonds price deterministic cash flows and generate forward contracts.

Prof notes: short rates and finite market fixings. An overnight fixing is a finite-period observable that may approximate the instantaneous short rate in a stylised model. Its day count, fixing and payment conventions still matter. Differentiating noisy or interpolated prices depends on the chosen smooth curve. An implied forward rate is known today, whereas the short rate observed at its future start is random in a stochastic model; the next chapter develops the maturity-direction derivative at fixed valuation date.

3 No-arbitrage replication and forward rates

3.1 Same Cash Flows, Same Price

Consider two feasible portfolios, A and B. Suppose that they generate exactly the same intermediate cash flows at every future payment date and the same final payoff. The law of one price implies

$$V_A(t) = V_B(t). \tag{33}$$

In words:

$$\boxed{\text{same future and intermediate cash flows } \implies \text{ same current price.}} \tag{34}$$

If, for example, $V_A(t) > V_B(t)$, an arbitrageur can sell Portfolio A and buy Portfolio B. The transaction produces the strictly positive initial cash flow

$$V_A(t) - V_B(t) > 0, \tag{35}$$

while all subsequent cash flows cancel exactly. The opposite strategy applies when $V_A(t) < V_B(t)$. Hence two portfolios with identical future cash flows cannot have different current prices in an arbitrage-free market.

For example, two portfolios paying 100 at the same final date with no intermediate payments cannot cost 80 and 90. Buy the first and short the second to receive 10 now with zero later net flow. Buying pressure raises the cheaper price and selling pressure lowers the dearer one.

This elementary argument is the fixed-income pricing principle used below: construct a portfolio of traded zero-coupon bonds that reproduces the cash flows of the target contract, and equate the two current values. The general theoretical foundations are developed in the Derivatives course.

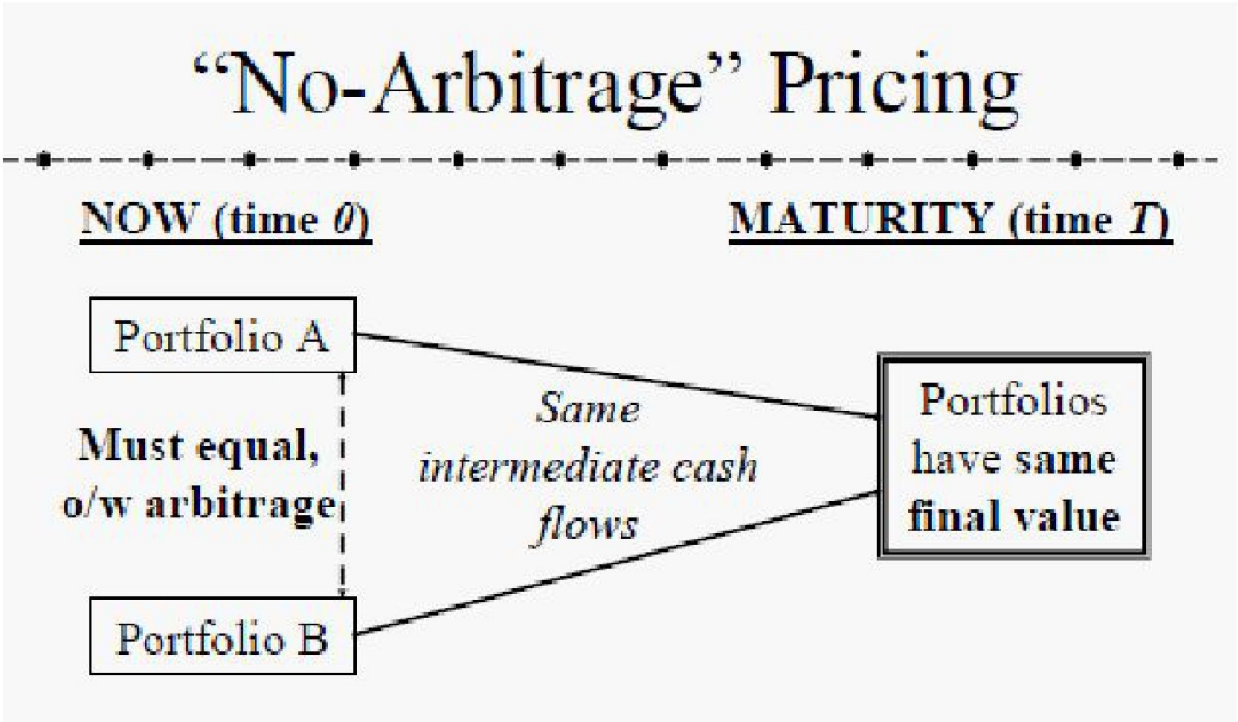


Figure 3: The Jarrow–Turnbull no-arbitrage argument: identical future and intermediate cash flows imply identical current prices.

3.1.1 Working assumptions

The elementary replication arguments in this chapter are developed under the following benchmark assumptions:

1. No market frictions: there are no transaction costs, bid–ask spreads, margin requirements, taxes, or restrictions on short sales.
2. No counterparty risk: all contractual payments used in the replication are honoured in full.
3. Competitive markets: traders are price takers and can trade at the quoted market prices.
4. No-arbitrage prices: prices adjust so that no feasible trading strategy produces an arbitrage opportunity.

These assumptions define a benchmark model. Their role is not to claim that actual markets are frictionless, but to isolate the price relations implied by replication. Frictions and counterparty or funding effects can be introduced later by modifying the relevant contracts and discounting framework.

3.2 Price Ratios and Relative Discounting

For $t \leq T_1 < T_2$, the ratio

$$\frac{P(t,T_2)}{P(t,T_1)} \tag{36}$$

compares the time-t values of unit payments at T_2 and T_1 . As will be shown below, this ratio has a direct forward interpretation. It is therefore essential to distinguish:

- $P(t,T_2)$, which discounts directly from T_2 to t;
- $P(t,T_2)/P(t,T_1)$, which discounts over the future interval $[T_1,T_2]$, as seen at time t.

3.2.1 The forward price of a zero-coupon bond

Fix three dates $t \leq T_1 < T_2$. Consider a forward contract, entered into at time t, under which k units of currency are paid at T_1 in exchange for one unit of currency at T_2 . Equivalently, the underlying asset is a zero-coupon bond maturing at T_2 , delivered at T_1 . The delivery price k is fixed at time t.

Compare the following two portfolios.

	t	T_1	T_2
Portfolio A: forward contract	0	-k	1
Portfolio B: buy one T_2-ZCB	$-P(t,T_2)$	0	1
Portfolio B: short k T_1-ZCBs	$kP(t,T_1)$	-k	0
Portfolio B: total	$-P(t,T_2)+kP(t,T_1)$	-k	1

Portfolios A and B have identical cash flows at T_1 and T_2 . Portfolio A has zero value at inception when k is the fair delivery price. By the law of one price, Portfolio B must then also have zero initial value:

$$-P(t,T_2)+kP(t,T_1)=0. \tag{37}$$

It follows that

$$k=\frac{P(t,T_2)}{P(t,T_1)}. \tag{38}$$

The quantity

$$P(t;T_1,T_2):=\frac{P(t,T_2)}{P(t,T_1)} \tag{39}$$

is the forward price, fixed at time t, of one unit paid at T_2 against payment at T_1 . It is also called the forward discount factor for the future interval $[T_1,T_2]$.

This argument makes the economic content of the formula explicit. The forward discount factor is not introduced as an algebraic ratio to be memorised: it is the unique delivery price that makes the forward contract have zero value at inception and prevents an arbitrage between the contract and its replicating portfolio.

A forward price is not, by definition, a forecast of the future spot price. It is a no-arbitrage delivery price implied by current spot discount factors. Relating it to expectations requires additional probabilistic assumptions and a specified probability measure.

Worked example

Prof notes: the three-month/nine-month example. Let $P(0,0.25)=0.98$ and $P(0,0.75)=0.956$. Buying one nine-month zero coupon and shorting k units of the three-month bond costs $0.956-0.98k$. The fair forward therefore has

$$k=0.956/0.98=0.975510204\ldots.$$

The handwritten 0.975 is rounded. An early verbal interchange of the two prices is corrected by the written example. The replica must match both $-k$ at three months and $+1$ at nine months; buying only the final bond fails to reproduce the intermediate liability. A positive bond price is its acquisition cost, so the buyer's initial cash flow has the opposite sign.

Mispriced forward: all cash flows. If the market delivery price is $k=0.95$, buy the forward, short one nine-month bond and buy 0.95 units of the three-month bond:

Position	0	0.25	0.75
Long market forward	0	-0.95	1
Short 9M zero coupon	0.956	0	-1
Long 0.95 units of 3M zero coupon	$-0.95(0.98)$	0.95	0
Total	0.025	0	0

The initial profit is 0.025 and all later obligations cancel. Buying 0.95/0.98 units of the three-month bond would deliver the wrong amount. Check one column per cash-flow date before identifying an arbitrage.

3.3 From Forward Discount Factors to Forward Rates

The forward discount factor $P(t;T_1,T_2)$ is a price fixed at time t for an exchange over the future interval $[T_1,T_2]$. To represent this price by a forward interest rate, one must again specify

$$\boxed{\text{valuation date } t, \quad \text{start date } T_1, \quad \text{end date } T_2,} \tag{40}$$

The valuation date tells us when the rate is determined. The start and end dates define the interval to which the rate applies. Let

$$\tau_1 = \tau(T_1, T_2) > 0 \tag{41}$$

be its tenor, expressed as a year fraction. Once τ_1 and the compounding convention have been fixed, the forward discount factor and the forward rate are equivalent representations of the same no-arbitrage price.

3.3.1 Simply compounded forward rate

Let $F(t; T_1, T_2)$ denote the simply compounded forward rate, determined at time t, for the future interval $[T_1, T_2]$. It is defined by

$$P(t; T_1, T_2) = \frac{1}{1 + F(t; T_1, T_2)\tau_1}. \tag{42}$$

Using the no-arbitrage expression for the forward discount factor gives

$$F(t; T_1, T_2) = \frac{1}{\tau_1}\left(\frac{1}{P(t; T_1, T_2)} - 1\right) \tag{43}$$

$$= \frac{1}{\tau_1}\left(\frac{P(t, T_1)}{P(t, T_2)} - 1\right). \tag{44}$$

The forward price divides the longer-maturity discount factor by the shorter-maturity one. Inverting that discount factor to obtain accumulation reverses the ratio in the simple-rate formula. The lecturer asks students to derive that relation at least once.

Thus the forward rate is implied by two spot discount factors. It is not an additional independent input once the discount curve is known.

For a notional N, a simple rate $F(t; T_1, T_2)$ applied to the accrual period produces the interest amount

$$NF(t; T_1, T_2)\tau_1. \tag{45}$$

A forward loan from the lender's perspective exchanges $-N$ at T_1 for $N[1 + F\tau_1]$ at T_2; its valuation date is earlier than that future accrual period.

The formula concerns the amount accruing over $[T_1, T_2]$; the contract must separately specify when that amount is paid.

3.3.2 Continuously compounded forward rate

For term-structure analysis and interest-rate modelling, it is useful to represent the same forward discount factor by the continuously compounded forward rate $F_c(t; T_1, T_2)$, defined by

$$P(t; T_1, T_2) = \exp(-F_c(t; T_1, T_2)\tau_1). \tag{46}$$

Hence

$$F_c(t; T_1, T_2) = -\frac{\log P(t; T_1, T_2)}{\tau_1} = \frac{\log P(t, T_1) - \log P(t, T_2)}{\tau_1}. \tag{47}$$

The simple and continuously compounded forward rates are numerically different, but they represent the same forward discount factor:

$$\frac{1}{1 + F(t; T_1, T_2)\tau_1} = \exp(-F_c(t; T_1, T_2)\tau_1). \tag{48}$$

Other compounding conventions could be defined in the same way, but they are not needed for the forward quantities used in these notes.

3.3.3 Relations between spot and forward rates

The fundamental relation is most transparent at the level of prices:

$$P(t,T_2) = P(t,T_1)P(t;T_1,T_2). \tag{49}$$

Rate relations follow by applying a common day-count and compounding convention. Under continuous compounding, write the spot rates as $R(t,T_1)$ and $R(t,T_2)$. Then

$$R(t,T_2)\tau(t,T_2) = R(t,T_1)\tau(t,T_1) + F_c(t;T_1,T_2)\tau_1. \tag{50}$$

When the year fractions are additive, $\tau(t,T_2) = \tau(t,T_1) + \tau_1$, this yields

$$F_c(t;T_1,T_2) = \frac{R(t,T_2)\tau(t,T_2) - R(t,T_1)\tau(t,T_1)}{\tau_1}. \tag{51}$$

This formula should not obscure the logical order: the price identity in Equation (49) is primary; the rate formula depends on the selected conventions.

Worked example

As an original check, let $P(t,T_1) = 0.98$, $P(t,T_2) = 0.94$, and $\delta = 0.5$. The forward discount factor is $0.94/0.98 \approx 0.959184$. The equivalent simple and continuously compounded forward quotes are respectively 8.5106% and 8.3345% per year. Substituting either quote back into its defining formula returns the same forward discount factor; their numerical difference is a convention difference, not an arbitrage.

Key idea

Draw the two dated payoffs first. A forward price is then the ratio of the two spot prices. Only after that should it be expressed as a rate.

3.4 The Forward Term Structure

At a fixed valuation date t, the collection

$$\{P(t;T_1,T_2) : t \le T_1 < T_2\} \tag{52}$$

defines the forward discount structure implied by the spot discount curve. After a day-count and compounding convention have been chosen, it can be expressed as the corresponding collection of forward rates.

Unlike a spot curve, a forward structure has two future date arguments. A one-dimensional forward curve therefore requires one dimension to be fixed. Two common representations are:

- fix the start date T_1 and vary the end date T_2;
- fix a tenor, such as three or six months, and vary the start date T_1, with T_2 determined by the contractual date rules.

The second representation is especially natural for market forward rates. The tenor is not merely a numerical difference between two times: it is a contractual period whose end date, business-day adjustment, and year fraction must be generated using the relevant conventions.

3.4.1 Instantaneous forward rate

If the discount curve is sufficiently smooth in maturity, the continuously compounded instantaneous forward rate is defined by

$$f(t,T) = -\frac{\partial}{\partial T}\log P(t,T). \tag{53}$$

The finite-period derivation starts with a continuously compounded forward for $[T, T+\Delta]$, using consistent numerical year units:

$$\frac{P(t,T+\Delta)}{P(t,T)} = e^{-F_c(t;T,T+\Delta)\Delta}, \qquad F_c(t;T,T+\Delta) = -\frac{\log P(t,T+\Delta) - \log P(t,T)}{\Delta}.$$

Letting $\Delta \downarrow 0$ gives Equation (53). Here t is fixed and T moves along the maturity axis. Conversely, integrating $\partial_T \log P(t,T) = -f(t,T)$ from t to T, with $\log P(t,t) = 0$, recovers the price formula below. This shows both directions of the relationship rather than treating differentiation and integration as unrelated definitions.

It describes the local rate in the maturity direction at T, as seen at time t. Equivalently,

$$P(t,T) = \exp\left(-\int_t^T f(t,u)du\right), \tag{54}$$

and the continuously compounded spot rate is the maturity average

$$R(t,T) = \frac{1}{\tau(t,T)}\int_t^T f(t,u)du, \tag{55}$$

when the time parameter and day-count convention are consistent with the integration variable. Thus the instantaneous forward curve contains the same information as the discount curve under the stated regularity conditions. At the short end of the curve,

$$f(t,t) = r(t), \tag{56}$$

so the short rate is the instantaneous forward rate whose start date is the current date. The instantaneous forward curve is likewise primarily a mathematical modelling object rather than a directly quoted market curve. In forward-rate models, such as the Heath–Jarrow–Morton framework, the dynamics of the family $\{f(t,T) : T \geq t\}$ are modelled directly.

3.4.2 Interpretation and curve shapes

Forward curves may be increasing, decreasing, flat, or non-monotone. Their shape need not coincide with the shape of the spot-rate curve: a spot rate is an average over an interval beginning at t, whereas a forward rate refers to a later subinterval. This difference is already visible in Equation (51).

Finally, an implied forward curve is not automatically a forecast curve. It is the set of forward prices or rates consistent with the current spot curve and no arbitrage. An expectations interpretation requires an explicit model, a probability measure, and attention to the relevant risk premia.

4 Benchmark rates: from a representation to a fixing

A rate read from a discount curve represents a price after dates, day-count and compounding have been selected. A benchmark fixing has a different contractual role: the contract names an index and a fixing date, then pays the number published for that index. The course's sequence is therefore contract, fixing, price, rate representation.

4.1 Two rates enter a derivative price

Under the course's risk-free, single-curve framework,

$$V(t) = \mathbb{E}_t^Q[D(t,T)V(T)].$$

The discount factor depends on the funding rate of the replicating strategy; the payoff of an interest-rate derivative depends on a reference rate. These are different roles even when one curve is assumed to serve both. For a European equity call the payoff is $(S_T - K)^+$; in Black–Scholes the bank account has deterministic rate r and $D(t,T) = e^{-r(T-t)}$. The rate finances the dynamic replication.

Key idea

The first part of this course assumes one curve for funding and reference rates. Keep that assumption visible: a benchmark fixing and a zero rate are not interchangeable merely because both are called interest rates.

Prof notes: risk-free price and valuation adjustments. The annotated page 4 distinguishes the risk-free price studied here from a price including counterparty and other valuation adjustments (collectively XVA). CVA concerns credit, FVA funding and KVA regulatory capital. The schematic sum in the notes is a decomposition, not a universal sign convention for all adjustments. For example, a credit-risk illustration pays 100 at year 1 if the obligor survives that date, with a possible recovery after earlier default. This illustrates credit exposure; it is distinct from the later benchmark-rate payoff discussion. Counterparty risk is excluded in the present pricing framework.

4.2 Fixing, accrual and payment are three different dates

For notional N and accrual period $[T_{i-1}, T_i]$,

$$C_i = N \,\text{Rate}_i \,\tau_i, \qquad \tau_i = \tau(T_{i-1}, T_i).$$

The fixing date determines when Rate_i becomes known; the accrual dates determine the year fraction; the payment date T_i^p determines the discount factor $P(t, T_i^p)$. The contract chooses these dates. The fixing can precede the accrual start and payment can follow its end.

	EURIBOR 6M	eSTR
Period	Six months, term rate	One business day, overnight rate
Known	Set in advance, at the start in the stylised schedule	Daily fixings; whole coupon known at the end
Coupon use	Simple rate $L(T_{i-1}, T_i)$	Compounded rate in arrears $L^{ON}(T_{i-1}, T_i)$

Prof notes: other tenors. The supplied annotations also list EURIBOR term tenors 1W, 1M, 3M, 6M and 12M. Overnight fixings can be converted into a payment over a longer period through daily compounding; they do not thereby become a fixing known at its start. In the stylised six-month example, the EURIBOR coupon is $NL(0, 0.5)\tau(0, 0.5)$, whereas the overnight coupon uses $L^{ON}(0, 0.5)$ learned over the accrual period.

4.3 Compounding overnight fixings in arrears

Let r_k be the fixing for business day k, and δ_k the year fraction for which it applies. Then

$$1 + \tau_i L^{ON}(T_{i-1}, T_i) = \prod_{k=1}^{n} (1 + r_k \delta_k), \qquad L^{ON} = \frac{\prod_k (1 + r_k \delta_k) - 1}{\tau_i}.$$

This is the simple rate equivalent to the accumulated product of daily growth factors. It is neither a geometric mean of the rates nor an arithmetic average. In particular, the Friday fixing can apply for three calendar days.

Worked example

For a Monday-to-Monday period, $\tau = 7/360$, $r = (3.00, 3.00, 3.05, 3.10, 3.10)\%$ and $\delta = (1, 1, 1, 1, 3)/360$. The growth factor is approximately 1.00059596, giving $L^{ON} = 3.0650\%$. The day-weighted average is 3.0643% and the plain average is 3.0500%. Only the compounded calculation gives the contractual coupon in this example.

The benchmark methodology and reform history are left to the source reference: Scaringi and Bianchetti (2020), *No Fear of Discounting: How to Manage the Transition from EONIA to eSTR*. This reference provides background; the pricing definitions and calculations needed above are self-contained.

4.4 Prof notes: benchmark questions

Does single curve mean a constant or flat rate? No. The lecturer distinguishes using one curve for both discounting and forwarding from assuming that its rates are identical at all maturities or unchanged tomorrow. The same curve can slope and can move over time. The announced code keeps discounting and projection as separate objects even when they are initially set equal, allowing a later extension to multiple curves.

Is a six-month benchmark a six-month compounding frequency? No. EURIBOR 6M is an annualised simple fixing applied to a six-month accrual fraction. In the classroom example the start fixing is 3.5%, so for stylised $\tau = 0.5$ the coupon is $N(0.035)(0.5) = 0.0175N$. Actual contractual dates and day count determine the precise τ; a fixing or payment lag does not redefine the accrual period. Knowing today's overnight fixing does not determine the future factors in the compounded product. The total coupon is known only after the required observations, subject to the contract's observation and payment conventions.

5 Risk-neutral pricing revisited: bank account and numeraire

5.1 Information and the bank account

The notes start from a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \geq 0}, Q_0)$, with a Brownian filtration. Q_0 denotes the historical or real-world measure, used to describe observed time series. Historical probability is also commonly written $\mathbb{P}$; the roles

of Q_0 and Q remain distinct. Conditional information at time t is represented by $\mathcal{F}_t$. The bank or money-market account satisfies

$$dB_t = r_t B_t \, dt, \qquad B_0 = 1.$$

There is no dW_t term in this equation: conditional on the current short rate, the infinitesimal interest accrued over $[t, t+dt]$ is $r_t B_t dt$. This is the meaning of *locally deterministic* in the handwritten notes. If r_t is random, the bank account's future value is still random.

$$\frac{dB_t}{B_t} = r_t dt, \qquad d\log B_t = r_t dt, \qquad \log B_T - \log B_t = \int_t^T r_s ds.$$

Consequently,

$$\frac{B_T}{B_t} = \exp\left(\int_t^T r_s ds\right), \qquad D(t,T) = \frac{B_t}{B_T} = \exp\left(-\int_t^T r_s ds\right).$$

The first ratio accumulates one unit from t to T along the realised short-rate path; its inverse discounts along that path. If $r_s = r$ is constant, $D(t,T) = e^{-r(T-t)}$. The source identifies this with $P(t,T)$ in the deterministic-rate case. With stochastic rates the market zero-coupon price instead equals $\mathbb{E}_t^Q[D(t,T)]$.

5.2 Discounted prices are martingales under the pricing measure

In the no-arbitrage framework used in the lecture, there is a pricing measure Q, equivalent to Q_0, under which traded asset prices expressed in bank-account units are martingales. For an asset price N and a claim value V,

$$N(t) = \mathbb{E}_t^Q[D(t,T)N(T)], \qquad \frac{V_t}{B_t} = \mathbb{E}_t^Q\left[\frac{V_T}{B_T}\right].$$

The account is adapted. Its current value is known under today's conditional information; its future value generally is not. Multiplying the second identity by B_t, which is known at time t, gives

$$\boxed{V_t = B_t \mathbb{E}_t^Q[V_T/B_T] = \mathbb{E}_t^Q[D(t,T)V_T].}$$

Key idea

Changing from Q_0 to Q changes the expectation used for pricing. It does not replace the contract's cash flow. No-arbitrage relates the price to the replicating traded assets; a historical forecast alone does not determine that price.

5.3 Why interest-rate claims are difficult

For an interest-rate derivative, V_T is itself a function of rates, for example a floating EURIBOR fixing. The pricing expectation contains both that payoff and the stochastic discount factor. Computing it requires their joint distribution under Q, rather than independent forecasts or the product of two expectations. The handwriting identifies this as the major computational issue. Factoring a conditional expectation of a product requires an appropriate conditional factorisation assumption; an unqualified statement about correlation is insufficient.

5.4 Changing the numeraire

The final handwritten page generalises the bank account to a suitable strictly positive traded numeraire N. Its associated equivalent pricing measure Q^N makes prices relative to N martingales:

$$\boxed{\frac{V_t}{N_t} = \mathbb{E}_t^{Q^N}\left[\frac{V_T}{N_T}\right].}$$

Choosing $N = B$ recovers the previous equation. The payment-date zero coupon provides the useful alternative developed in Section 6. A different numeraire requires its associated measure; one cannot keep the same expectation measure while arbitrarily changing the denominator. These statements use the course's idealised no-arbitrage assumptions and eligible traded numeraires.

5.5 Prof notes: a zero coupon is the simple rate-dependent case

For a unit zero coupon, define $I_{t,T} = \int_t^T r_s ds$. Its constant terminal payoff removes the need for a joint distribution with a future fixing: only the accumulated-rate distribution enters the zero-coupon expectation already derived above. A random

integrand does not prevent the time integral from being defined: it is evaluated along each admissible rate path. The computational programme is to solve the short-rate dynamics, integrate that solution, identify the conditional distribution of $I_{t,T}$, and evaluate the expectation. Integrating a solution containing a Brownian stochastic integral can produce nested integrals; suitable stochastic-Fubini conditions are needed to rearrange them.

The lecturer previews Gaussian short-rate models and the use of their moment-generating functions. **Generated calculation:** if $I_{t,T}$ is conditionally normal under $\mathbb{Q}$ with mean $m_{t,T}$ and variance $v_{t,T}$, the normal moment-generating function gives

$$P(t,T) = \exp(-m_{t,T} + \tfrac{1}{2}v_{t,T}).$$

This is a conditional Gaussian special case, not a statement that every short-rate model has a normal integral. For deterministic rates $v_{t,T} = 0$. For a floating coupon there is an additional random cash flow, and the earlier joint-distribution issue returns. Choosing a more convenient measure simplifies that valuation; the bank-account pricing formula remains valid.

6 Change of numeraire

6.1 The price is invariant; the expectation changes

Work on a finite horizon $[0,T]$ with a filtration $(\mathcal{F}_t)$ satisfying the usual conditions and trivial initial information. Write $\mathbb{E}_t^{\mathbb{Q}}[X] = \mathbb{E}^{\mathbb{Q}}[X \mid \mathcal{F}_t]$. Let B_t be the money-market account and $\mathbb{Q} = \mathbb{Q}^B$ its equivalent pricing measure. For a traded asset V without intermediate payments,

$$V_t = B_t \mathbb{E}_t^{\mathbb{Q}}\left[\frac{V_T}{B_T}\right].$$

Standing integrability assumption. Discounted prices of the relevant assets, including the numeraire, are true martingales under their corresponding pricing measures. Integrability is required; a local martingale alone does not justify all the conditional-expectation equalities below.

A new numeraire N_t must be **strictly positive** throughout the horizon almost surely, **non-dividend-paying** on that horizon, and the price of a **traded asset or self-financing traded portfolio**. The short rate is not an asset price and can be negative. Positivity permits division and gives an equivalent measure; trading and the martingale assumptions justify the density construction.

Key idea

A numeraire is the unit in which we quote values. Dividing an asset's price by a traded unit changes the relative price process. The matching probability measure changes too. The final value expressed in currency remains the same.

6.2 The density and the pricing theorem

Define the terminal Radon–Nikodym derivative and its density process by

$$\left.\frac{d\mathbb{Q}^N}{d\mathbb{Q}}\right|_{\mathcal{F}_T} = Z_T = \frac{N_T/N_0}{B_T/B_0} = \frac{N_T B_0}{B_T N_0}, \qquad Z_t = \frac{N_t B_0}{B_t N_0}.$$

Since N/B is a true $\mathbb{Q}$-martingale, Z is a true martingale, $\mathbb{E}_t^{\mathbb{Q}}[Z_T] = Z_t$, and $\mathbb{E}^{\mathbb{Q}}[Z_T] = Z_0 = 1$. Also $Z_T > 0$ almost surely. Thus

$$\mathbb{Q}^N(A) = \mathbb{E}^{\mathbb{Q}}[Z_T \mathbf{1}_A], \quad A \in \mathcal{F}_T, \qquad \left.\frac{d\mathbb{Q}^N}{d\mathbb{Q}}\right|_{\mathcal{F}_t} = Z_t,$$

defines a probability measure equivalent to $\mathbb{Q}$. Every admissible traded asset without intermediate payments satisfies

$$\mathbb{E}_t^{\mathbb{Q}^N}\left[\frac{V_T}{N_T}\right] = \frac{V_t}{N_t}, \qquad \boxed{V_t = N_t \mathbb{E}_t^{\mathbb{Q}^N}\left[\frac{V_T}{N_T}\right].}$$

6.3 Proof using conditional Bayes' formula

For an $\mathcal{F}_T$-measurable, $\mathbb{Q}^N$-integrable random variable X,

$$\mathbb{E}_t^{\mathbb{Q}^N}[X] = \frac{\mathbb{E}_t^{\mathbb{Q}}[XZ_T]}{\mathbb{E}_t^{\mathbb{Q}}[Z_T]} = \frac{\mathbb{E}_t^{\mathbb{Q}}[XZ_T]}{Z_t}.$$

Apply this identity to $X = V_T/N_T$:

$$\mathbb{E}_t^{\mathbb{Q}^N}\left[\frac{V_T}{N_T}\right] = \frac{\mathbb{E}_t^{\mathbb{Q}}\left[\frac{V_T}{N_T}\frac{N_T B_0}{B_T N_0}\right]}{N_t B_0/(B_t N_0)} = \frac{B_t}{N_t}\mathbb{E}_t^{\mathbb{Q}}\left[\frac{V_T}{B_T}\right] = \frac{B_t}{N_t}\frac{V_t}{B_t} = \frac{V_t}{N_t}.$$

The required terminal integrability follows from

$$\mathbb{E}^{\mathbb{Q}^N}\left[\left|\frac{V_T}{N_T}\right|\right] = \frac{B_0}{N_0}\mathbb{E}^{\mathbb{Q}}\left[\left|\frac{V_T}{B_T}\right|\right] < \infty.$$

The process V/N is the conditional expectation of this integrable terminal variable, hence a true $\mathbb{Q}^N$-martingale by the tower property.

Prof notes: normalisation and equivalent measures. The density compares the growth factors of two traded units, not their raw prices. Its initial normalisation is essential: without B_0/N_0, the candidate density would not in general have expectation one. Multiplying a numeraire by a fixed positive constant changes the quoted relative prices but leaves its associated measure unchanged, because that constant cancels from N_T/N_0. Equivalence means agreement on null events, not equality of all probabilities. The conditional Bayes denominator is the time-t density Z_t, obtained from the martingale property, rather than an extra unconditional expectation of the payoff.

6.4 Moving between two numeraires

For admissible numeraires U and N, apply the theorem separately:

$$V_t = U_t\mathbb{E}_t^{\mathbb{Q}^U}\left[\frac{V_T}{U_T}\right] = N_t\mathbb{E}_t^{\mathbb{Q}^N}\left[\frac{V_T}{N_T}\right].$$

The chain rule for densities gives

$$\left.\frac{d\mathbb{Q}^N}{d\mathbb{Q}^U}\right|_{\mathcal{F}_T} = \frac{d\mathbb{Q}^N/d\mathbb{Q}}{d\mathbb{Q}^U/d\mathbb{Q}} = \frac{(N_T/B_T)(B_0/N_0)}{(U_T/B_T)(B_0/U_0)} = \frac{N_T/N_0}{U_T/U_0} = \frac{N_T U_0}{U_T N_0}.$$

The density is the ratio of *gross returns*, with the new numeraire in the numerator and the old one in the denominator. Its process is a martingale under the *starting* measure $\mathbb{Q}^U$.

6.5 General payoffs, intermediate payments and uniqueness

The same Bayes calculation applies to an integrable terminal payoff whose selected pricing formula is $V_t = B_t\mathbb{E}_t^{\mathbb{Q}}[V_T/B_T]$. Any admissible numeraire gives

$$V_t = \mathrm{num}_t\mathbb{E}_t^{\mathbb{Q}^{\mathrm{num}}}\left[\frac{V_T}{\mathrm{num}_T}\right].$$

For a replicated payoff this is the replicating portfolio's value. In an incomplete market, changing numeraire within a selected pricing measure does **not** establish a unique price for an arbitrary non-replicated claim.

The formulas above apply directly to assets with **no intermediate payments**. Dividend-paying assets require a gains process that includes their distributions. A self-financing portfolio reinvesting distributions may serve as a numeraire if it meets all the other conditions. A coupon bond's unreinvested price alone drops at coupon dates; it cannot simply be treated as a non-dividend-paying asset.

6.6 Money-market, forward and swap measures

Money-market account. $(N, \mathbb{Q}^N) = (B, \mathbb{Q}^B)$ with $\mathbb{Q}^B = \mathbb{Q}$ is the usual risk-neutral pair. In a short-rate model,

$$B_t = B_0\exp\left(\int_0^t r_s\,ds\right), \qquad V_t = \mathbb{E}_t^{\mathbb{Q}}\left[e^{-\int_t^T r_s\,ds}V_T\right].$$

Zero-coupon bond. $N_t = P(t,T)$, $0 \le t \le T$, defines the T-forward measure $\mathbb{Q}^T$. Since $P(T,T) = 1$,

$$\boxed{V_t = P(t,T)\mathbb{E}_t^{\mathbb{Q}^T}[V_T].}$$

The discount factor outside this expectation is observed at time t; it is not the random future discount factor from the risk-neutral representation. On $\mathcal{F}_T$, with $B_0 = 1$,

$$\frac{d\mathbb{Q}^T}{d\mathbb{Q}} = \frac{e^{-\int_0^T r_s\,ds}}{P(0,T)}.$$

If rates are deterministic this density is one and $\mathbb{Q}^T = \mathbb{Q}$. Stochastic rates generally make the measures different.

Prof notes: match the bond to the payment date. If a benchmark fixes at T_1 and its coupon pays at T_2, choose $P(t,T_2)$ as numeraire. The fixing makes the amount known at T_1, but payment still occurs later. Before fixing, the expectation uses today's information under the T_2-forward measure. The floating-coupon calculation and its additional single-curve simplification are developed in Section 8; the general change-of-numeraire theorem itself does not require forwarding and discounting curves to coincide.

Swap annuity. For payment dates $T_1 < \cdots < T_n$, deterministic year fractions $\tau_i > 0$, and swap start $T_0 < T_1$, set

$$N_t = A(t;T_0,T_n) = \sum_{i=1}^{n} \tau_i P(t,T_i), \qquad 0 \le t \le T_0.$$

It is a strictly positive fixed portfolio of zero-coupon bonds with no payments before the end of this horizon. Under the standing assumptions it defines the annuity or swap measure $\mathbb{Q}^S$, used for swaptions. After coupon dates the unreinvested annuity makes payments: the horizon restriction matters and the numeraire needs adjustment or reinvestment.

6.7 Prof notes: a changed expectation needs changed dynamics

The forward-measure representation simplifies the discounting step, but it does not permit drawing the same scenarios under the old measure without an adjustment. If a stochastic differential equation has been specified under a historical or bank-account measure, changing to the chosen payment measure generally changes its drift. A Girsanov calculation relates the Brownian motions when its assumptions hold. Monte Carlo samples must either come from the stated pricing measure or carry the appropriate likelihood weights. Replacing the label on an expectation alone does not change the distribution of simulated paths. The lecturer recalls the no-dividend Black–Scholes analogy: $dS_t = \mu S_t dt + \sigma S_t dW_t^{\mathbb{P}}$ under the historical measure becomes $dS_t = rS_t dt + \sigma S_t dW_t^{\mathbb{Q}}$ under the bank-account measure. The discounted stock has zero drift there, while the stock itself has drift r. A further change to a forward measure has its own drift adjustment; it is not justified by setting every asset's drift to r again.

The lecturer's immediate goal is the general numeraire theorem and its pricing applications. Identifying the expected benchmark fixing with a bond-ratio forward rate introduces the extra single-curve assumption; it should not be imported into a setting with separate forwarding and discounting curves. The same numeraire method then supports later FRA, swap and option pricing.

6.8 Multiple-choice activities and supplied solutions

Each question has exactly one correct answer. Here $F(t;T_{i-1},T_i)$ is the simple forward rate and $L(T_{i-1},T_i)$ the benchmark fixing. An annuity is used only over an admissible horizon before its payments.

Activity 1: Numeraire check.

1.1. Which cannot be a numeraire? (A) Bank account $B(t)$; (B) zero-coupon bond $P(t,T)$, $t \le T$; (C) short rate $r(t)$; (D) annuity $A(t) = \sum_i \tau_i P(t,T_i)$.

1.2. Why cannot a coupon-paying bond's price be used directly? (A) It pays coupons, so its price relative to B is not a $\mathbb{Q}$-martingale; (B) its price can be negative; (C) it is not traded; (D) it is not adapted.

1.3. Which property guarantees that $Z_t = N_t B_0/(B_t N_0)$ is a $\mathbb{Q}$-martingale under the standing assumptions? (A) N is deterministic; (B) N has continuous paths; (C) N grows at rate r; (D) N is a traded asset price with no dividends.

1.4. Price a derivative once with N and once with U. What happens? (A) Two prices, since numeraire is a modelling choice; (B) same price, different expectation; (C) same only for deterministic rates; (D) same only if $U = B$.

Supplied answers: 1.1 C; 1.2 A; 1.3 D; 1.4 B. A numeraire is a positive traded price, whereas the short rate is a rate and may be negative. Coupons must enter the gains process. Trading, absence of dividends and true-martingale integrability support the discounted-price identity; positivity gives equivalent measures. Both numeraire representations reproduce the same currency value. The choice simplifies computation within a fixed pricing setup.

Activity 2: Forward-measure pricing.

2.1. A payoff $H(T)$ is paid at T. Which is its value under $\mathbb{Q}^T$? (A) $\mathbb{E}_t^{\mathbb{Q}^T}[e^{-\int_t^T r_s ds}H(T)]$; (B) $P(t,T)\mathbb{E}_t^{\mathbb{Q}^T}[H(T)]$; (C) $P(t,T)\mathbb{E}_t^{\mathbb{Q}}[H(T)]$; (D) $B(t)\mathbb{E}_t^{\mathbb{Q}^T}[H(T)/B(T)]$.

2.2. Why does the discount factor leave the expectation? (A) Rates are deterministic under $\mathbb{Q}^T$; (B) $\mathbb{Q}^T = \mathbb{Q}$; (C) payoff is independent of rates; (D) $P(T,T) = 1$, leaving only the time-t price $P(t,T)$.

2.3. With $B_0 = 1$, which is $d\mathbb{Q}^T/d\mathbb{Q}$ on $\mathcal{F}_T$? (A) $P(0,T)^{-1}e^{-\int_0^T r_s ds}$; (B) $P(0,T)e^{-\int_0^T r_s ds}$; (C) $P(0,T)^{-1}e^{\int_0^T r_s ds}$; (D) $e^{-\int_0^T r_s ds}$.

2.4. If rates are deterministic: (A) the measures are mutually singular; (B) equal only if $r = 0$; (C) $\mathbb{Q}^T = \mathbb{Q}$; (D) density equals $P(0,T)$.

Supplied answers: 2.1 B; 2.2 D; 2.3 A; 2.4 C. The numeraire formula and $P(T,T) = 1$ give B; the other alternatives mix measures with the wrong discounting unit. The density is the ratio of bond and bank-account growth rates. Dividing by $P(0,T)$ normalises its mean to one. For deterministic rates $P(0,T) = e^{-\int_0^T r_s ds}$, hence density one; rates need not be zero.

Activity 3: Growth-rate densities.

3.1. Which is $d\mathbb{Q}^N/d\mathbb{Q}$ on $\mathcal{F}_T$? (A) N_T/B_T; (B) $(B_T/B_0)/(N_T/N_0)$; (C) $N_T B_T$; (D) $(N_T/N_0)/(B_T/B_0)$.

3.2. Take $N = B$. What is $d\mathbb{Q}^B/d\mathbb{Q}$? (A) B_T; (B) 1 since $\mathbb{Q}^B = \mathbb{Q}$; (C) $1/B_T$; (D) $e^{-\int_0^T r_s ds}$.

3.3. Which is $d\mathbb{Q}^N/d\mathbb{Q}^U$? (A) $(U_T/U_0)/(N_T/N_0)$; (B) $N_T U_T$; (C) $(N_T/N_0)/(U_T/U_0)$; (D) N_T/U_T.

3.4. The process $Z_t = P(t,T_j)P(0,T_i)/[P(t,T_i)P(0,T_j)]$, the density of $\mathbb{Q}^{T_j}$ relative to $\mathbb{Q}^{T_i}$, is a martingale under which measure? (A) $\mathbb{Q}^{T_i}$, the starting measure; (B) $\mathbb{Q}^{T_j}$, the new measure; (C) $\mathbb{Q}$; (D) none, since it is deterministic.

Supplied answers: 3.1 D; 3.2 B; 3.3 C; 3.4 A. Growth-rate ratios require their initial-value normalisation. For identical numeraires the ratio is one. Dividing the two densities relative to $\mathbb{Q}$ cancels the bank account. A density process is a martingale under the starting measure; the bond-price ratio is a traded relative price under $\mathbb{Q}^{T_i}$.

Editorial correction to the supplied explanation of 3.1. Alternative A is the unnormalised ratio, with expectation N_0/B_0; alternative B is the inverse growth-rate ratio. The supplied answer D is correct, but its explanation swaps the descriptions of the distractors. The alternatives above retain their original order.

Reference. H. Geman, N. El Karoui and J.-C. Rochet (1995), "Changes of numéraire, changes of probability measure and option pricing", *Journal of Applied Probability* 32(2), 443–458. DOI: 10.2307/3215299. This is the theorem underlying the measure-change construction.

7 Plain vanilla interest-rate products: a cash-flow map

Let N be notional, $T_0 < \cdots < T_n$ a schedule, $\tau_i = \tau(T_{i-1}, T_i)$ and $L_i = L(T_{i-1}, T_i)$ the fixing set at T_{i-1}. L_i^{ON} is the overnight rate compounded in arrears and K is a fixed strike. Cash flows below are paid at the accrual end unless specified otherwise. They describe contract payoffs, not present prices.

7.1 Linear products

Zero-coupon and fixed-coupon bonds.
A zero coupon pays N at T. A coupon bond pays $Nc\tau_i$ at each T_i and principal N at T_n.

Floating-rate note.
Coupons $N\tau_i L_i$ at each T_i, plus N at T_n; the index can instead be compounded eSTR.

Deposit.
A simple-rate loan: $-N$ at T_0 and $N[1 + \tau(T_0,T)L(T_0,T)]$ at T.

Forward rate agreement.
The source uses end-period settlement: a fixed-rate receiver obtains $N\tau(T_1,T_2)[K - L(T_1,T_2)]$ at T_2. Keep this convention separate from an FRA settled and discounted at T_1.

Interest-rate swap.
For a fixed-rate payer, each T_i payment is $N\tau_i(L_i - K)$. An IRS is a portfolio of period exchanges with the same fixed rate. The source uses 6M EURIBOR as a floating index.

Overnight indexed swap.
The same fixed-versus-floating exchange with compounded overnight coupons: $N\tau_i(L_i^{ON} - K)$ at T_i; in euro the index is eSTR.

Interest-rate future.
The source's long position has cumulative rate payoff $N\tau(T_1, T_2)[R_0 - L(T_1, T_2)]$, delivered through daily margining. Margining creates a convexity adjustment relative to a forward; linearity of the terminal rate exposure does not make the two prices identical.

Basis swap.
Exchanges floating indices or tenors. In the stylised aligned-period example, the payment is $N\tau_i[L_i^{3M} + s - L_i^{6M}]$, with spread s on the 3M leg. In the ideal single-curve framework the fair spread is zero. The supplied material notes that basis swaps existed before 2007 with near-zero quoted spreads; materially nonzero spreads after the credit crunch indicate the need for multiple curves. Actual tenor schedules need not be aligned.

7.2 Products with optionality

With $x^+ = \max(x, 0)$, a caplet pays $N\tau_i(L_i - K)^+$ and a floorlet pays $N\tau_i(K - L_i)^+$ at T_i. A cap or floor is a portfolio of these options over consecutive periods:

$$\text{Cap}: \sum_i N\tau_i(L_i - K)^+, \qquad \text{Floor}: \sum_i N\tau_i(K - L_i)^+.$$

A long cap and short floor form a collar (the opposite positions give its reverse):

$$\sum_i N\tau_i\big[(L_i - K_c)^+ - (K_f - L_i)^+\big], \qquad K_f < K_c.$$

A European swaption gives the right to enter an IRS at T_0. Writing

$$A(T_0) = \sum_{i=1}^{n} \tau_i P(T_0, T_i),$$

for the annuity, and $S(T_0)$ for the underlying swap rate, the source's cash-settled value at exercise is $NA(T_0)(S(T_0) - K)^+$ for a payer and $NA(T_0)(K - S(T_0))^+$ for a receiver. A physically exercised swaption enters the corresponding swap rather than paying that number as an immediate coupon. A call on a unit zero coupon maturing at $S > T$ has payoff $(P(T, S) - K)^+$ at T. The supplied notes connect caps to options on zero coupons and swaptions to options on coupon bonds. A further decomposition of a coupon-bond option into zero-coupon options requires the appropriate model conditions; it is not a general payoff identity in arbitrary multi-factor models.

7.3 Beyond plain vanilla

The source lists Bermudan and American swaptions, constant-maturity swaps (CMS) and options on CMS, range accruals, callable bonds, rate digitals and options on rate spreads. These identify later applications; the present material does not supply a full pricing model for every item.

7.4 Prof notes: the pricing programme

The lecturer distinguishes linear products from optional payoffs. Fixed coupons, floating coupons, FRAs and swaps can first be understood through static cash-flow replication; an IRS collects period exchanges. Optional products require a dynamic/model-based argument because the payoff reacts nonlinearly to rate evolution. The expectation is the mathematical representation of a pricing strategy, not an unrelated forecast. The announced Python work progresses from bonds and floating notes to FRA/swap pricing and then rate options. The lecturer proposes comparing model-based analytical prices with numerical conditional expectations, including Monte Carlo, with models calibrated to market observations. Some spoken model names are not reliably recognised; the recording does not yet supply all the future code or formulas. The product sheet serves as a reference map, and the lecturer does not promise to cover every listed exotic product in this session.

8 Discounting, floating coupons and forward rate agreements

8.1 One curve or several curves?

The valuation of a future payment has two distinct ingredients: the discount factor and the amount of the payment. If the latter depends on a benchmark interest rate, a model must describe that benchmark as well as the discounting rule. Under the money-market pricing measure, the schematic valuation is

$$V(t) = \mathbb{E}_t^{\mathbb{Q}}[D(t, T)V(T)], \qquad D(t, T) = \exp\left(-\int_t^T r_s\, ds\right).$$

Here $D(t,T)$ is a possibly random discount factor, whereas $V(T)$ can be a function of a future reference-rate fixing. The two objects inside the expectation need not be independent.

In the classical **single-curve framework**, commonly used before the 2007 credit crisis, interbank offered rates were treated as close enough to risk-free rates for one curve to perform two jobs. It supplied the funding/discounting rates used to construct discount factors, and the reference rates used to determine floating payments. Forward rates for the floating leg were then inferred from that same discount curve.

The credit crisis made the distinction material. Interbank lending incorporates credit and liquidity effects; offered rates with tenors such as one, three, six or twelve months need not coincide with a risk-free rate. A longer unsecured lending tenor exposes the lender for longer and can carry a larger premium. This is a motivation for separate tenor curves, not a theorem that every observed rate curve must be increasing.

For an ideally collateralised transaction, the collateral arrangement changes the relevant benchmark. With perfect collateralisation remunerated at the overnight rate, the working model discounts using the associated overnight indexed swap (OIS) curve. Collateralisation makes counterparty exposure negligible in this idealisation; it does not make the underlying interbank benchmark itself risk free.

	Discounting curve	Forwarding curve
Purpose	Convert a payment at its payment date to present value	Project the benchmark fixing that determines a floating payment
Ideal collateralised benchmark	OIS/overnight curve	Reference-rate curve for the contractual tenor
Tenor dependence	Overnight collateral remuneration	Separate three-, six- or twelve-month curves when required
Risk represented	Collateral funding convention	The benchmark's tenor-specific credit/liquidity component

The lecture also flags the LIBOR reform, dated 2019 in the notes. This heading introduces the transition away from the old benchmark framework; it does not by itself specify replacement conventions or a complete reform timetable. The calculations below first isolate the single-curve assumption so that its consequences, and its limits, are explicit.

Prof notes: keep the two roles visible in the implementation. The lecturer motivates this distinction before the pricing laboratory. Even when a first implementation supplies the same numerical curve to both tasks, discounting a payment and projecting its benchmark fixing should remain separate operations. This makes the single-curve assumption visible and allows one input to be replaced when moving to several curves. A semiannual benchmark coupon calls for its corresponding six-month forwarding curve, while a quarterly benchmark calls for the three-month curve; the choice follows the contractual index, not the name of the product alone. The crisis comparison is between lending for six months at once and lending for three months with a later rollover: their economic exposures need not be identical even though the final horizon is the same.

8.2 A simple forward rate is a forward-measure martingale

Fix valuation and accrual dates $t \le T_1 < T_2$, and a positive year fraction $\tau = \tau(T_1, T_2)$. The discount-curve-implied simple forward rate is

$$F_D(t; T_1, T_2) = \frac{1}{\tau}\left[\frac{P(t,T_1)}{P(t,T_2)} - 1\right] = \frac{P(t,T_1) - P(t,T_2)}{\tau P(t,T_2)}.$$

It is the rate for lending or borrowing over the future interval $[T_1, T_2]$, agreed at the earlier valuation date. The subscript D emphasises that this rate is constructed from the discount curve. Its no-arbitrage interpretation follows from zero-coupon-bond replication.

Use the strictly positive T_2 zero coupon as numeraire, with associated measure $\mathbb{Q}^{T_2}$. The numerator $P(t,T_1) - P(t,T_2)$ is the value of a portfolio long one T_1 zero coupon and short one T_2 zero coupon. Up to T_1, neither bond has made an intermediate payment. Therefore its value divided by $P(t,T_2)$ is a martingale under $\mathbb{Q}^{T_2}$, provided the relevant discounted prices are true, integrable martingales. Multiplication by the deterministic constant $1/\tau$ preserves this property:

$$\boxed{\mathbb{E}_t^{\mathbb{Q}^{T_2}}[F_D(u; T_1, T_2)] = F_D(t; T_1, T_2), \qquad t \le u \le T_1.}$$

The time interval matters. The process starts at the valuation date and reaches its fixing at T_1; it is not an unfixed rate evolving through the payment date T_2.

Prof notes: distinguish the traded numerator from the numeraire. The long–short portfolio in the numerator is the traded asset being expressed in relative units. The numeraire is the *denominator*, the T_2 bond. The forward is therefore a martingale under the T_2-forward measure, not automatically under the T_1 measure or the bank-account measure. When

working with several forwards having different payment dates, each naturally has a different associated forward measure. Their individual martingale statements cannot all be transferred to a common measure without another change-of-measure calculation.

An analogous instantaneous-rate identity is

$$\mathbb{E}_t^{\mathbb{Q}^T}[r_T] = f(t,T),$$

where r_T is the short rate at T and $f(t,T)$ is the instantaneous forward rate seen at t. This identity needs a sufficiently regular short-rate/discount-bond model and justification for differentiation under the conditional expectation. Indeed, if

$$P(t,T) = \mathbb{E}_t^{\mathbb{Q}}[D(t,T)],$$

then under those regularity assumptions

$$-\partial_T P(t,T) = \mathbb{E}_t^{\mathbb{Q}}[D(t,T)r_T], \qquad f(t,T) = -\partial_T \log P(t,T) = \frac{\mathbb{E}_t^{\mathbb{Q}}[D(t,T)r_T]}{P(t,T)}.$$

Conditional change of measure identifies the last ratio with the T-forward expectation of r_T. This is an expectation under a specified pricing measure, not a prediction under the physical probability measure.

8.3 A floating coupon: fixing date, payment date and value

Let $L(T_1,T_2)$ be a simple reference rate observed at T_1 for the accrual period ending at T_2. On notional N, a pure floating interest payment at T_2 is

$$V(T_2) = N\tau L(T_1,T_2).$$

The rate becomes known at the **fixing/reset date** T_1, while the money is paid at the later **payment date** T_2. The year fraction translates the annualised quote into interest for that period. There is no principal repayment in this particular payoff.

Because payment occurs at T_2, the natural numeraire is $P(t,T_2)$, which pays one at that date. For $t \le T_1$ the general valuation is

$$\boxed{V(t) = P(t,T_2)\mathbb{E}_t^{\mathbb{Q}^{T_2}}[N\tau L(T_1,T_2)]\,.}$$

Once the fixing has occurred, $T_1 \le t \le T_2$, the amount is known and its value is simply $N\tau L(T_1,T_2)P(t,T_2)$.

8.3.1 Single-curve valuation by change of measure

The defining extra assumption is that the benchmark fixing is obtained from the same discount curve:

$$L(T_1,T_2) = \frac{1}{\tau}\left[\frac{1}{P(T_1,T_2)} - 1\right] = F_D(T_1;T_1,T_2),$$

using $P(T_1,T_1) = 1$. The forward-measure martingale property just proved gives

$$\begin{aligned} V(t) &= N\tau P(t,T_2)\mathbb{E}_t^{\mathbb{Q}^{T_2}}[F_D(T_1;T_1,T_2)] \\ &= N\tau P(t,T_2)F_D(t;T_1,T_2) \\ &= N\tau P(t,T_2)\frac{1}{\tau}\left[\frac{P(t,T_1)}{P(t,T_2)} - 1\right] \\ &= \boxed{N[P(t,T_1) - P(t,T_2)].} \end{aligned}$$

Thus a future floating coupon has the same present value as the difference of two zero coupons. The cancellation of the denominator is a consequence of the single-curve identification, rather than an identity valid for every reference benchmark.

Prof notes: replacing a random coupon by a known forward. The pricing calculation can be read as replacing the future floating fixing by today's corresponding forward *inside its payment-date pricing expectation*. Under the single-curve assumption this produces the same present value as a fixed coupon whose rate equals $F_D(t;T_1,T_2)$. It does not mean the realised fixing will equal today's forward. The expectation step uses both the chosen measure and the martingale property; the later cancellation uses the equality of the forwarding and discounting curves. Keeping those two steps separate shows exactly which simplification fails when the curves differ.

8.3.2 The same result by cash-flow replication

Take $N = 1$ first. Buy one T_1 zero coupon and sell one T_2 zero coupon. The initial cost is $P(t, T_1) - P(t, T_2)$. At T_1, invest the unit received from the first bond in $1/P(T_1, T_2)$ units of the T_2 bond. The reinvestment uses exactly the received unit, so it is self financing. The full cash-flow account is

Position	t	T_1	T_2
Buy one T_1 zero coupon	$-P(t, T_1)$	$+1$	0
Reinvest at T_1 in T_2 bonds	0	-1	$1/P(T_1, T_2)$
Sell one T_2 zero coupon	$+P(t, T_2)$	0	-1
Total	$-[P(t, T_1) - P(t, T_2)]$	0	$\tau L(T_1, T_2)$

The last column follows from $1/P(T_1, T_2) - 1 = \tau L(T_1, T_2)$. Multiplying all positions by N replicates the general coupon. Matching the payment and the absence of an intermediate net flow proves the same value by no arbitrage.

In a multi-curve setting write P^d for the discounting bond and define the relevant projected benchmark by

$$F^L(t; T_1, T_2) := \mathbb{E}_t^{\mathbb{Q}^{d,T_2}}[L(T_1, T_2)].$$

Then the coupon value is $N\tau P^d(t, T_2)F^L(t; T_1, T_2)$. It generally cannot be simplified to $N[P^d(t, T_1) - P^d(t, T_2)]$: that would impose precisely the single-curve relationship that is now being relaxed.

8.4 Money-market instruments and curve construction

The market curve is inferred from observable instruments. The lecture groups these into **deposits**, **FRAs/futures** and **swaps**, and connects their quotes to the bootstrapping of an interest-rate curve. A bootstrap successively solves instrument-pricing equations for curve quantities; market quotes are inputs and the discount/forward curve is the calibrated object. In a multi-curve framework, one must specify which curve each quote constrains. Futures also require their own margining/convexity treatment; they cannot automatically be substituted for FRAs.

Prof notes: pricing and bootstrapping run in opposite directions. For the first pricing exercises, take the curve as given and compute the instrument value. In the market, the directly observed objects are instrument quotes, so calibration reverses that direction: choose the curve so that those instruments reprice to their quoted values. The lecturer distinguishes a government-bond curve inferred from coupon-bond prices from the interbank curve inferred from deposits, FRA/futures and swaps. The instruments, credit exposure and intended use of the curve matter. Bootstrapping also needs choices between quoted maturities, so reproducing a list of observed quotes does not uniquely determine every possible interpolated curve without further conventions.

Liquidity guides which instruments provide reliable information at each maturity. The lecture associates deposits with the very short end, FRA/futures with intermediate maturities (illustratively from three or six months towards two years), and swaps with longer horizons, mentioning quotes extending as far as fifty years. These are indicative market segments rather than universal cutoffs: the liquid instruments available for a particular currency and date determine the actual calibration set. Inferring a curve from an illiquid price can import a poor or stale quote into all subsequent valuations.

8.4.1 Deposits

A spot deposit exchanges $-N$ today for a single repayment

$$N[1 + L(t, T)\tau(t, T)]$$

at maturity T. There is no intermediate interest compounding in this simple-rate contract. It is the money-market analogue of a single-payment zero-coupon investment. In the single-curve idealisation, equality of the initial investment and the discounted repayment gives

$$N = P(t, T)N[1 + L(t, T)\tau(t, T)], \qquad P(t, T) = \frac{1}{1 + L(t, T)\tau(t, T)}.$$

The relevant year fraction follows the contract's convention, indicated as ACT/360 for these money-market rates. A six-month tenor must therefore be converted using its actual dates; using 0.5 is the simplified half-year convention in the numerical illustration.

Worked example

For the quoted six-month deposit $L(0, 6\text{m}) = 2\%$, taking $\tau = 0.5$ gives

$$P(0, 6\text{m}) = \frac{1}{1 + 0.02 \times 0.5} = \frac{1}{1.01} = 0.9900990099\ldots.$$

An investment of 100 repays 101 in this illustration. With actual ACT/360 dates, replace 0.5 by the actual day count divided by 360.

8.5 Forward rate agreements and their settlement convention

A **forward rate agreement** (FRA) is agreed at t on a future reference rate, such as EURIBOR, fixing at T_1 for the interval $[T_1, T_2]$. The notional N, fixed contractual rate K, fixing date, payment date and accrual fraction τ must all be specified. In the end-period settlement convention used here, there is no exchange of principal, and the two interest amounts are paid at T_2.

The **buyer/long position** is the fixed-rate payer and floating-rate receiver. The **seller/short position** is the fixed-rate receiver and floating-rate payer. Their payment map is

	Buyer pays seller	Seller pays buyer
At inception t	No initial exchange for a fair FRA	No initial exchange for a fair FRA
At fixing T_1	Fixing becomes known	Fixing becomes known
At payment T_2	$N\tau K$	$N\tau L(T_1, T_2)$

The net cash flow received by the buyer is therefore

$$\Pi_{\text{payer}}(T_2) = N\tau[L(T_1, T_2) - K], \qquad \Pi_{\text{receiver}}(T_2) = N\tau[K - L(T_1, T_2)].$$

These signs agree with the fixed-receiver convention used in the earlier product map; changing which side is being valued changes the sign. The quoted fair FRA rate is the fixed K that makes the contract's initial value zero. An existing contract with a previously fixed K can subsequently have a nonzero value.

Worked example

The supplied example has notional $N = 100$, inception at zero, reset at three months and payment at nine months. The six-month accrual is represented by $\tau = 0.5$, and the agreed fixed rate is $K = 4.2\%$. The buyer pays fixed interest

$$100 \times 0.5 \times 0.042 = 2.10$$

and receives $50L(3\text{m}, 9\text{m})$, where the future fixing is expressed as a decimal annualised rate. The buyer's net payment received is $50[L(3\text{m}, 9\text{m}) - 0.042]$. The fixing is random before three months; the contractual fixed amount is already known.

Numerical annotation. The handwritten rate heading reads 4%, while the fixed-payment arrow is labelled 4.2%. The calculation above follows the cash-flow label. Using the heading's 4% instead would give fixed interest $100 \times 0.5 \times 0.04 = 2.00$. The pricing argument is unchanged, but the two inputs must not be silently mixed.

For $t \le T_1$, the payer's value under the chosen discounting convention is

$$V_{\text{payer}}(t) = N\tau P^d(t, T_2)\left[F^L(t; T_1, T_2) - K\right].$$

Hence $K_{\text{fair}} = F^L(t; T_1, T_2)$ at inception. In the single-curve special case this becomes

$$V_{\text{payer}}(t) = N\{P(t, T_1) - [1 + \tau K]P(t, T_2)\}, \qquad K_{\text{fair}} = F_D(t; T_1, T_2).$$

This follows by subtracting the present value of the fixed coupon, $N\tau KP(t, T_2)$, from the floating-coupon value derived above. It is a valuation of the end-date payoff. For an FRA instead settled early at T_1, the corresponding amount must be converted to that earlier date according to the contract, often using $1 + \tau L(T_1, T_2)$ in the denominator. Mixing early settlement with an undiscounted end-period payoff would value a different cash-flow stream.

8.6 Hedging a future borrower

A borrower exposed to a future floating-rate loan fears an unexpected increase in $L(T_1, T_2)$. The borrower takes the FRA buyer's position: pay fixed $N\tau K$ and receive floating $N\tau L(T_1, T_2)$. At the payment date the loan interest and the FRA's

floating receipt cancel:

$$\underbrace{-N\tau L(T_1,T_2)}_{\text{loan interest}}+\underbrace{N\tau[L(T_1,T_2)-K]}_{\text{FRA receipt}}=-N\tau K.$$

The combined interest cost is fixed. This exact cancellation requires matching notional, reference benchmark, fixing date, accrual period, day-count convention and payment date. Principal cash flows belong to the loan and have not been removed by the interest-rate hedge. The receiver side gives the opposite hedge, appropriate to an exposure benefiting from a high floating receipt and endangered by falling rates.

Prof notes: assess the hedge together with the exposure. If rates rise, the payer FRA gains while the loan becomes more expensive; if rates fall, the FRA loses while the loan becomes cheaper. The object is a stable combined cost, not a guaranteed gain on the derivative in isolation. Entering the same FRA without the corresponding borrowing exposure leaves a directional rate position rather than removing one. A business can therefore choose a known financing cost even though it may later turn out that an unhedged floating loan would have been cheaper.

8.7 A swap is a portfolio of period exchanges

An interest-rate swap collects FRA-type fixed-versus-floating exchanges over a schedule

$$t\le T_0<T_1<\cdots<T_n.$$

If $T_0>t$, the swap is forward starting. For period i, the reset date is T_{i-1}, the payment date is T_i, the accrual fraction is $\tau_i=\tau(T_{i-1},T_i)$, and the floating benchmark is $L(T_{i-1},T_i)$. The final date T_n is maturity. The notional sets the scale of each interest payment.

For a fixed payer, period i has net receipt

$$N\tau_i[L(T_{i-1},T_i)-K].$$

The fixed rate K is the same for the swap's periods, whereas individual fair FRAs for those periods would generally have different quoted fixed rates. The swap rate is chosen to balance the whole collection of payments, not to make every constituent exchange separately worth zero.

The schedule above uses coinciding fixed and floating payment dates for clarity. The lecturer explicitly notes that the two legs can instead have different payment frequencies. In that case generate each leg's reset dates, payment dates and year fractions separately; do not force a fixed coupon onto the floating leg's schedule. For floating dates T_i with accruals τ_i, and fixed payment dates S_j with accruals α_j, the payer value before the first reset is

$$N\left\{\sum_i\tau_iP^d(t,T_i)F_i^L(t)-K\sum_j\alpha_jP^d(t,S_j)\right\}.$$

Its fair fixed rate has the floating-leg present-value sum in the numerator and the actual fixed-leg annuity $\sum_j\alpha_jP^d(t,S_j)$ in the denominator. The aligned-schedule formulas below are the special case $S_i=T_i$ and $\alpha_i=\tau_i$.

Before the first reset, write $F_i^L(t)=\mathbb{E}_t^{\mathbb{Q}^{d,T_i}}[L(T_{i-1},T_i)]$. Summing the period values gives

$$V_{\text{payer}}^{\text{swap}}(t)=N\sum_{i=1}^n\tau_iP^d(t,T_i)[F_i^L(t)-K].$$

Therefore, when $\sum_i\tau_iP^d(t,T_i)>0$, the fair fixed swap rate is

$$K_{\text{swap}}(t)=\frac{\sum_{i=1}^n\tau_iP^d(t,T_i)F_i^L(t)}{\sum_{i=1}^n\tau_iP^d(t,T_i)}.$$

In the single-curve case, the floating terms telescope:

$$\sum_{i=1}^n\tau_iP(t,T_i)F_D(t;T_{i-1},T_i)=\sum_{i=1}^n[P(t,T_{i-1})-P(t,T_i)]=P(t,T_0)-P(t,T_n).$$

Thus $K_{\text{swap}}(t)=[P(t,T_0)-P(t,T_n)]/\sum_i\tau_iP(t,T_i)$. This final formula makes explicit how the single-curve coupon identity links FRA valuation, swap valuation and curve bootstrapping.

8.7.1 Prof notes: the first unpaid coupon may already be fixed

A pricing schedule needs one row per accrual period, carrying its start/reset date, end/payment date and contractual year fraction. Year fractions need not all be equal, and a calendar date difference is not a year fraction until the day-count convention is applied. The first unpaid row deserves particular attention.

Suppose valuation takes place between a reset and its payment, $T_0 < t < T_1$. The next floating fixing $L(T_0, T_1)$ was observed in the past. Its coupon is therefore known and must be discounted directly; it is not a new forward-looking expectation of an unfixed benchmark. For the aligned schedule used above, the remaining payer swap value is

$$V^{\text{swap}}_{\text{payer}}(t) = N\tau_1 P^d(t, T_1)[L(T_0, T_1) - K] + N \sum_{i=2}^{n} \tau_i P^d(t, T_i)[F^L_i(t) - K].$$

The known fixing replaces the projected rate only in the first term; later coupons still use their respective forward expectations. Payments that have already occurred are excluded from the remaining-cash-flow value. With separate fixed and floating schedules, apply the same rule to the first unpaid floating coupon and value each outstanding fixed coupon on its own date. In code, automatically projecting every row from today's curve would overwrite an already known cash flow and generally misprice the contract.

9 Laboratory foundations: dates, curve validation and settlement pricing

This chapter integrates the conceptual material developed in the dates-and-conventions notebook and in the pricing-library laboratory. These are the theoretical foundations of the practical work in Part II. Definitions of discount factors, equivalent rates and forwards remain at their canonical locations in Sections 1–3; the laboratory adds the operational assumptions, validation criteria and settlement-price interpretation needed to use them correctly.

9.1 Dates and schedules in the laboratory

The evaluation date is the date of valuation information; the trade date is the agreement date. The settlement date is the payment/delivery date of the transaction, sometimes called its value date. The curve reference date is the origin of the factors returned by the curve. These dates need not coincide. QuantLib's global evaluation date must be set explicitly; it is not automatically changed by constructing a new curve or schedule.

For the first notebook the trade date is 24 August 2026 and the TARGET calendar advances it by two *business* days to settlement. Adding two calendar days is a different operation in general. The calendar identifies valid business dates, while a rolling convention selects how an invalid contractual date is adjusted. Apply the Following, Modified Following, Preceding or Unadjusted definitions in Section 1.3.2. The end-of-month flag controls date generation from an end-of-month anchor; it is distinct from subsequently rolling a date.

The schedule's `effectiveDate` means its contractual start, which may lie in the past and need not equal trade, evaluation or settlement. For an existing regular bond, the current coupon period and future grid can be recovered backwards from maturity and coupon frequency without knowing the original issue date. The start of the retained current period must precede settlement when interest has already begun to accrue. For a forward-starting contract supply its actual effective date. A temporary lookback boundary used to recover a regular period is not the original effective date.

The initial notebook permits students to examine stubs by changing schedule rules. The later pricing library deliberately supports *regular contractual periods only*: its schedule builder rejects relevant irregular periods. Different month lengths and business-day rolling can still produce unequal coupon year fractions without creating a contractual stub. In this teaching library accrual end and payment dates coincide; there is no fixing/payment lag, ex-coupon period, amortisation or issuer-credit model. These restrictions delimit the examples and must not be inferred for all bonds or swaps.

The curve time convention and coupon accrual convention perform different jobs. The synthetic curve uses Actual/365 Fixed: a 366-day interval has coordinate 366/365. Coupon cash amounts and simple forwards use the contractual year fraction for their own period. The first notebook compares Actual/360, Actual/365 Fixed, Actual/Actual AFB and 30/360 Bond Basis. Specifying only "Actual/Actual" is insufficient when the variant affects the result. The AFB choice in the market-data validation below is empirically tied to that supplied export.

9.2 The market-curve convention audit

The first laboratory uses the supplied 24 August 2026 LSEG EUR deposits and IRS versus EURIBOR 6M curve. Its settings are collateralisation **Off**, discounting tenor **6M**, **Cubic Discount** interpolation, **no extrapolation**, and **Mid** prices. Zero-rate calculation is Actual/Actual compounded; forward-rate style is FRA Actual/360. It is a pedagogical single-curve representation, rather than the overnight discounting curve of an ideally collateralised euro derivative.

The exported discount factors are reproduced with **Actual/Actual AFB and annual compounding**. Apply the annual-compounding transformation in Section 2.1.2, converting percentage rates to decimals first. The label EURIBOR 6M specifies the index tenor and does not imply semiannual compounding of its exported zero rates. A negligible reconstruction error validates this numerical convention match for the supplied observations; it does not certify a new model or every hidden vendor convention. An assertion is useful because a changed dataset or convention should trigger investigation before prices are reused.

Worked example

An independent calculation from the supplied workbook gives the following observations. These values are retained as dataset-specific validation, rather than new market quotes.

Tenor	Zero rate (%)	AFB year fraction	Discount factor
5Y	3.125158602	$5 + 2/365$	0.8572423570
10Y	3.288958602	$10 + 2/365$	0.7234091624
30Y	3.339460615	$30 + 4/365$	0.3731280911

Their maturity dates are 26 August 2031, 26 August 2036 and 28 August 2056. The label 5Y is not an instruction to ignore the adjusted date. The exercise also asks for 1Y, but the supplied export has no exact 1Y/12M row. Its short nodes include futures labels such as `cm2` and `cm3`. Obtain the missing matching export or explicitly introduce an interpolation rule; a futures row is not automatically a supplied one-year observation.

For this dataset the zero rate peaks at 20Y, about 3.477254%, then declines to approximately 3.339461% at 30Y, 3.123653% at 40Y and 2.872986% at 50Y. The corresponding discount factors still decrease: a longer accrual horizon can more than offset a lower annualised rate. This is a property of the observations, not a universal no-arbitrage monotonicity theorem; the negative-rate qualification in Section 1.2.3 still applies.

The node-to-node forward calculation uses consecutive exported dates and Actual/360 accruals. Those periods need not be six months. The vendor's standard 6M forward analysis instead supplies its own start and end dates. Comparing them requires identical endpoints, day count, compounding, interpolation and underlying curve object. A sparse table of zero-curve nodes alone generally does not reproduce the vendor's full interpolation and tenor-specific machinery. A discrepancy is a model-validation question to explain, not a numerical result to conceal.

9.3 A synthetic curve with an explicit date interface

The second laboratory is a different example: synthetic discount-factor pillars referenced to 19 October 2026, with Actual/365 Fixed time coordinates. They are not current market quotes or a calibrated market curve. The curve is a function returning one dimensionless discount factor for one QuantLib calendar date. Calendar dates and numerical pillar times are distinct inputs; strings, pandas timestamps or Python dates must be converted to the required `ql.Date` interface before calling the library.

Let x_i be strictly increasing pillar times and $D_i > 0$ their factors. For $x_i \le x \le x_{i+1}$ the demonstration interpolator uses

$$w = \frac{x - x_i}{x_{i+1} - x_i}, \qquad \log D(x) = (1 - w)\log D_i + w \log D_{i+1}, \qquad D(x) = D_i^{1-w} D_{i+1}^{w}.$$

This preserves positivity but imposes no monotonicity; discount factors above one and negative rates are allowed. Instantaneous forwards on the interior of a pillar interval are constant,

$$f(x) = -\frac{\log D_{i+1} - \log D_i}{x_{i+1} - x_i},$$

and may jump at pillars. Smoothness is not the aim of this first curve. Queries outside the pillar interval are rejected: no extrapolation is performed.

The origin $(0, 1)$ is a normalisation identity, not a market quote. Add it if an input file begins after zero, without discarding its first market pillar. The constructor requires matching one-dimensional arrays of at least two finite entries, times starting at zero and strictly increasing, positive finite factors, and an initial factor within absolute tolerance 10^{-12} of one. It then fixes that first factor to exactly one. It copies the input arrays; later changes to the caller's arrays do not update the curve. Construct a new curve when the pillars change. These are input-consistency checks, not evidence that market instruments have been repriced successfully.

A pricing function should request factors by date without knowing whether they came from this interpolator, a bootstrap, or a compatible QuantLib curve. Changing curve construction therefore need not change the pricing formulas. Keep file reading, plotting and export in the notebook, and supply the curve as a callable rather than a filename.

9.4 Settlement values, dirty prices and accrued interest

Let $D_r(T)$ denote the discount factor returned by today's curve, whose reference date is r. For the buyer's known cash flows C_i paid strictly after settlement s, the library's settlement-value convention is

$$D_s(T_i) = \frac{D_r(T_i)}{D_r(s)}, \qquad \text{Dirty}(s) = \sum_{T_i > s} C_i D_s(T_i).$$

If $r = t$, this divides the time-t present value by $P(t,s)$. If $r = s$, the denominator is one; valuation at an earlier t additionally needs discounting from t to s. If settlement precedes the curve's reference date, obtain the missing short-end information or a suitable curve rather than silently extrapolating backwards. A settlement value calculated with today's information is not the random market price that will later be observed at s.

The example library excludes payments on settlement and has no ex-coupon convention. Values are **currency amounts**, not automatically prices per 100. For notional N, convert a currency value V to a quote per 100 by $100V/N$. Rates, fixings and spreads are annual decimals: 0.03 is 3%, and $10/10000 = 0.001$ is ten basis points.

For a fixed-rate bond the principal is added once, with the final coupon:

$$\text{Dirty}_{\text{CB}}(s) = NK \sum_{T_i > s} \tau_i D_s(T_i) + N D_s(T_n).$$

Under the lab's simple-accrual convention, if the current period starts at $T_0 < s$, accrued interest is $NK\tau(T_0, s)$, measured using the contract's coupon convention. Clean value equals dirty value minus accrued interest in the same units. The full current coupon year fraction belongs in the future coupon; replacing it by the remaining fraction would confuse the dirty cash-flow value with accrued-interest subtraction.

9.5 Observed fixings, spreads and floating-rate identities

Keep discounting and projection as separate arguments even when the demonstration passes the same curve to both. Projection factors give an unfixed benchmark through

$$F_i = \frac{D^{\text{proj}}(T_{i-1})/D^{\text{proj}}(T_i) - 1}{\tau_i}.$$

The accrual denominator is the year fraction of that same contract period, rather than the curve's time-coordinate convention. A period already started needs its observed benchmark fixing; today's curve cannot reconstruct a historical fixing. The supplied `current_fixing` is the annual benchmark *before* spread. If a period begins exactly at settlement, the library uses a supplied current fixing, or otherwise projects it. This interface carries one current fixing, not an arbitrary date-indexed fixing history.

Add the contractual spread b once to every benchmark coupon, including an already fixed first coupon. With $\widehat{L}_i$ denoting either the known fixing or the projected benchmark as appropriate,

$$\text{Dirty}_{\text{FRN}}(s) = N \sum_{T_i > s} \tau_i (\widehat{L}_i + b) D_s(T_i) + N D_s(T_n).$$

An FRN and a coupon bond repay principal; a standard interest-rate swap does not. For an aligned reset-date schedule with zero spread, a single curve and first fixing set to that curve's implied forward, telescoping gives

$$N \sum_{i=1}^{n} \tau_i F_i \frac{D(T_i)}{D(s)} = N \frac{D(s) - D(T_n)}{D(s)}, \qquad \text{Dirty}_{\text{FRN}}(s) = N.$$

This par identity is a conditional test of the stated setup. A seasoned FRN with an old fixing, a spread, or different projection and discount curves need not equal par.

For separate fixed and floating schedules, the swap formula in Section 8 applies at settlement using D_s. Include any contractual floating spread in the floating-leg value and any known first fixing in $\widehat{L}_i$. Writing $A_s = \sum_j \alpha_j D_s(S_j)$ and $V_{\text{float}} = N \sum_i \tau_i (\widehat{L}_i + b) D_s(T_i)$,

$$K_{\text{par}} = \frac{V_{\text{float}}}{N A_s}, \qquad V_{\text{payer}} = V_{\text{float}} - NKA_s = NA_s(K_{\text{par}} - K).$$

Thus, for $N, A_s > 0$, a payer with contractual fixed rate above par has negative value; a receiver has the opposite value. Detailed leg tables show unsigned amounts for each leg; apply the payer/receiver sign when combining them. No principal is added to either swap leg.

9.6 From market quotes to risk and model-based valuation

The input–output legend distinguishes functions already implemented from possible later teaching blocks. The latter are proposed responsibilities, not working functions or extra homework requirements. Market-curve bootstrapping turns reference dates, benchmark quotes, instrument conventions and construction settings into a date-accessible curve plus repricing diagnostics. Portfolio valuation then combines positions, contract data, settlement, discount/projection curves and required fixings to give position values, a total and, when requested, detailed tables.

Benchmark sensitivities require bumping the selected market quote, rebuilding the curve and repricing. They give signed currency changes per basis point for individual benchmarks or a simultaneous quote shift. Directly shifting zero rates is a

different risk convention. In the initial single-curve setting, pass each rebuilt curve to both discounting and projection. A swap hedge/scenario block would use portfolio and hedge-swap specifications, baseline quotes, sensitivities and scenario shocks to return hedge direction/notional, residual exposures and original versus hedged value changes.

The planned caplet/cap interface takes dates, year fractions, notional, strike, both curves, model and volatility *with its convention*; its outputs are caplet prices and a total cap value, with intermediate forwards and discount factors. Black and Bachelier are named as later pricing choices rather than interchangeable volatility conventions.

The proposed G1/Hull–White one-factor analytical bond function would take the initial discount curve, model parameters, valuation time, maturity and state, and must reproduce the initial curve at time zero. G1 simulation would additionally specify the probability measure, time grid, path count and random seed, returning model-state/short-rate paths and, if generated directly, path discount factors. Monte Carlo valuation then averages appropriately discounted scenario payoffs, reporting an estimated price, standard error and confidence interval. Curve construction, a model's dynamic law, simulation and Monte Carlo averaging are distinct operations. Optional Nelson–Siegel–Svensson fitting could supply another date-based curve representation; G1 calibration is a separate possible task and is not assumed by these interfaces.

10 Activities with worked reasoning

10.1 Supplied questions and generated study answers

These activities can be discussed in pairs before a class comparison. The aim is to identify the contract and its timeline before selecting a formula or a rate convention.

The questions below are retained from Notes. Each following answer is an original study aid, not a supplied model solution.

10.2 Activity 1 — Spot or forward?

For each object below, identify the valuation date, the start date and the end date of the relevant interval. Then classify it as a price or a convention-dependent rate representation:

$$P(t,T), \qquad R(t,T), \qquad P(t;T_1,T_2), \qquad F(t;T_1,T_2), \qquad f(t,T).$$

Draw the timeline for each forward object. Explain why the distinction between spot and forward concerns the underlying contracts before it concerns their rate quotations.

Generated study answer

Expression	Interpretation
$P(t,T)$	Spot price at t of one unit at T.
$R(t,T)$	Spot/zero rate from t to T, under a stated convention.
$P(t,T_2)/P(t,T_1)$	Forward discount factor for $[T_1,T_2]$.
$F_s(t;T_1,T_2)$	Simple forward rate for that interval.
$f(t,T)$	Instantaneous forward rate at maturity T.

All are assessed at t. For the spot objects the relevant interval starts at t and ends at T. For each finite forward object the interval starts at T_1 and ends at T_2; for $f(t,T)$ it is the infinitesimal interval at T. None is a realised future interest rate merely because it contains a future date.

10.3 Activity 2 — One price, several rate quotations

A discount factor $P(t,T)$ and the dates t,T are given.

1. List the additional information required to convert $P(t,T)$ into a numerical spot rate.
2. Two dealers quote different numerical rates for the same dates. Give conditions under which both quotations may nevertheless represent the same price.
3. A student concludes that the dealer quoting the lower rate offers the cheaper zero-coupon contract. Explain why the conclusion cannot be drawn from the rates alone.
4. State which object remains unchanged when the compounding convention is changed.

Generated study answer

Before computing a rate, specify the start and end dates, day-count/year fraction, and compounding convention. For a given P and τ, then solve $L = (P^{-1} - 1)/\tau$, $Y = P^{-1/\tau} - 1$, or $R = -\log P/\tau$ according to that convention. To compare two dealer quotes, reconstruct the discount factor from each on identical contractual terms. Equal discount factors mean equal economic value even if the percentages differ; the lower numerical rate is not necessarily the cheaper contract. The dated price $P(t, T)$ remains unchanged when only the quotation convention changes. The conventions in Section 2 supply all four conversion pairs.

10.4 Activity 3 — Complete the replicating strategy

A forward contract entered into at t exchanges k units at T_1 for one unit at T_2 , where $t < T_1 < T_2$. Without consulting the derivation in the text:

1. construct the replicating portfolio using T_1 - and T_2 -zero-coupon bonds;
2. prepare a cash-flow table with columns t, T_1, T_2 ;
3. determine the value of k that gives both portfolios the same current price;
4. if the quoted delivery price is higher than this value, specify which portfolio should be bought and which should be sold. Verify every cash flow of the resulting arbitrage strategy.

Identify which working assumptions are needed for the argument.

Generated study answer

An at-market forward to pay K at T_1 and receive one unit at T_2 has future vector $(-K, +1)$. The bond portfolio long one T_2-bond and short K T_1-bonds matches this vector exactly:

Position	t cash flow	T_1	T_2
Long forward	0	$-K$	$+1$
Long T_2-bond, short K T_1-bonds	$KP(t, T_1) - P(t, T_2)$	$-K$	$+1$

Setting the portfolio's initial cash flow to zero gives the fair K. If a zero-premium quoted contract instead has $K > P(t, T_2)/P(t, T_1)$, take the bond portfolio and *short* the forward: the future cash flows cancel and $KP(t, T_1) - P(t, T_2) > 0$ is received now. For K below fair, reverse both positions. This idealised arbitrage presumes the quoted forward and bond positions are executable on matching terms.

10.5 Activity 4 — From spot prices to a forward rate

Suppose that $P(t, T_1)$, $P(t, T_2)$, and $\tau_1 = \tau(T_1, T_2)$ are known.

1. Derive the forward discount factor from spot prices.
2. Obtain the simply compounded forward rate $F(t; T_1, T_2)$.
3. Obtain the continuously compounded forward rate $F_c(t; T_1, T_2)$.
4. Explain why the two numerical rates differ although no new price information has been introduced.
5. Explain why neither rate is, by definition, a forecast of the future spot rate prevailing at T_1 .

Generated study answer

Given $P_1 = P(t, T_1)$, $P_2 = P(t, T_2)$ and $\delta = \tau(T_1, T_2)$, replication gives $P(t; T_1, T_2) = P_2/P_1$. Solving $P_2/P_1 = (1 + F_s\delta)^{-1}$ yields $F_s = (P_1/P_2 - 1)/\delta$; solving $P_2/P_1 = e^{-F_c\delta}$ yields $F_c = \log(P_1/P_2)/\delta$. The rates differ numerically because their compounding definitions differ, although they quote the same forward price. Neither, by definition, forecasts the spot rate that will prevail at T_1; such an interpretation requires assumptions about dynamics and risk premia.

10.6 Activity 5 — Read the term structures

Discuss and correct each statement where necessary:

1. "The discount curve and the zero-rate curve are two independent market inputs."
2. "If the spot-rate curve is increasing, every forward rate must exceed the spot rate with the same end date."
3. "A forward curve is uniquely defined once the valuation date is fixed."
4. "The short rate and the instantaneous forward rate are unrelated modelling objects."
5. "A discount factor greater than one necessarily creates an arbitrage."

For the third statement, describe two different one-dimensional forward curves that can be extracted from the same two-date forward structure.

Generated study answer

1. The discount and zero curves cannot be selected independently once dates and compounding are fixed; they encode the same positive prices.
2. An upward-sloping spot curve does not license a convention-free comparison with a forward quote. Under a common continuous, additive clock, a strictly increasing spot rate between T_1 and T_2 does imply that the interval forward exceeds the spot rate to T_2; the hypotheses matter.
3. A forward curve is not uniquely a one-argument curve without another rule. One may fix T_1 and vary T_2, or fix a three-month tenor and vary T_1 while generating T_2 contractually. The *instantaneous* forward curve at fixed t is one-dimensional in T.
4. The short rate is the boundary value $f(t,t)$, not an unrelated additional price input when the required limit exists.
5. A discount factor greater than one may reflect a negative implied rate. It is not, by itself, an arbitrage.

Key idea

Exam check: label each quantity as a dated *price*, a *rate quote*, or a *future realised rate*. Many incorrect arguments silently exchange these categories.

10.7 Optional AI-supported discussion

After attempting an activity, ask an AI tutor: "Check whether I have correctly distinguished contracts, prices and rate representations in this problem. Ask me one question at a time if my reasoning is incomplete. Keep the notation of these notes, in particular $\tau_i = \tau(T_{i-1}, T_i)$, and do not introduce models or instruments beyond this chapter." Compare its feedback with the timelines, definitions and no-arbitrage relations above; resolve disagreements by returning first to the contractual cash flows.

Part II

Code

11 Notebook 01: practical work on dates, conventions and market curves

The cells below follow the supplied execution order. Conceptual markdown is integrated in Part I and replaced here by internal references; exercises, questions, code and all recorded outputs remain. Outputs are supplied observations, not a new execution. Original notebooks and workbooks are preserved.

11.1 Practical objectives

Reference dataset: 24 August 2026. Work through dates, conventions, schedules, rate conversions and market-curve validation. The conceptual account and the dataset conventions are in Section 9. The original notebook remains available unchanged in the project.

11.1.1 How we will work

Each cycle follows the same sequence:

1. **Guided example:** we interpret and run an existing block of code.
2. **Your turn:** you modify the example or build a small extension.
3. **Discussion checkpoint:** we compare results and identify the relevant financial mechanism.
4. **A possible solution:** discussed in class and provided separately after the attempt.

The objective is not to copy code, but to learn how to **read, modify, validate and interpret** it.

11.1.2 0. Setup

The notebook uses standard scientific Python packages and QuantLib. If QuantLib is not available in the active Anaconda environment, install it once from **Anaconda Prompt** with:

```
conda install -c conda-forge quantlib-python
```

After installation, restart the Jupyter kernel.

```
from pathlib import Path
Path.cwd()
```

```
WindowsPath('C:/Users/User/OneDriveUniTo/2026-FIXED-INCOME/notebooks/01-dates-and-conventions')
```

```
# Import tools for paths, dates, numerical calculations, tables, plots, and financial conventions.
# Set the display format and plotting style, and define the folder containing the raw data.

from pathlib import Path          # File and folder path management
import datetime as dt             # Date and time-interval management in Python

import numpy as np                # Numerical calculations and operations on vectors and arrays
import pandas as pd               # Management and analysis of tabular data
import matplotlib.pyplot as plt   # Plot creation
import QuantLib as ql             # Financial tools: dates, calendars, conventions, curves, and instruments

pd.set_option("display.float_format", lambda x: f"{x:,.8f}")
plt.style.use("seaborn-v0_8-whitegrid")

DATA_DIR = Path("data/raw")
```

11.2 Cycle 1: dates and calendar conventions

11.2.1 Guided example 1A: date representations

Use Section 9.1 to interpret the Python, pandas and QuantLib date objects below.

```
trade_date_py = dt.date(2026, 8, 24)
trade_date_pd = pd.Timestamp("2026-08-24")
trade_date_ql = ql.Date(24, ql.August, 2026)

print("Python date:   ", trade_date_py, type(trade_date_py))
print("pandas date:   ", trade_date_pd, type(trade_date_pd))
print("QuantLib date: ", trade_date_ql, type(trade_date_ql))
```

```
Python date:    2026-08-24 <class 'datetime.date'>
pandas date:    2026-08-24 00:00:00 <class 'pandas._libs.tslibs.timestamps.Timestamp'>
QuantLib date:  August 24th, 2026 <class 'QuantLib.QuantLib.Date'>
```

```
# Print Python dates

print(trade_date_py.strftime("%d/%m/%Y"))  # 'DAY/MONTH/YEAR'
print(trade_date_py.strftime("%Y-%m-%d"))  # 'YEAR-MONTH-DAY'
print(trade_date_py.strftime("%a"))        # 'WEEKDAY'
```

```
24/08/2026
2026-08-24
Mon
```

```
def py_to_ql(date):
    # Convert datetime.date or pandas.Timestamp to QuantLib.Date.
    date = pd.Timestamp(date)
    return ql.Date(date.day, date.month, date.year)


def ql_to_py(date):
    # Convert QuantLib.Date to datetime.date.
    return dt.date(date.year(), date.month(), date.dayOfMonth())

# Check that converting Python → QuantLib → Python preserves the original date
assert ql_to_py(py_to_ql(trade_date_py)) == trade_date_py
```

```
ql_to_py(py_to_ql(trade_date_py)) == trade_date_py
```

```
True
```

11.2.2 Evaluation and settlement: implementation

Apply the date roles in Section 9.1; inspect the explicit evaluation-date setting and business-day settlement advance in the next cell.

```
# Set the global evaluation date in QuantLib
ql.Settings.instance().evaluationDate = trade_date_ql

calendar = ql.TARGET()  # Euro-area TARGET business calendar
settlement_days = 2
settlement_date = calendar.advance(trade_date_ql, settlement_days, ql.Days)

print("Trade date:      ", trade_date_ql)
print("Settlement date: ", settlement_date)
```

```
Trade date:       August 24th, 2026
Settlement date:  August 26th, 2026
```

```
# Check if target date
print(calendar.isBusinessDay(trade_date_ql))
print(calendar.isBusinessDay(settlement_date))
```

```
True
True
```

11.2.3 Guided example 1B: business-day adjustments

Apply the rolling conventions in Section 1.3.2 and compare the recorded weekdays.

```
unadjusted_date = ql.Date(31, ql.October, 2026)  # Saturday

adjustments = {
    "Unadjusted": ql.Unadjusted,
    "Following": ql.Following,
    "Modified Following": ql.ModifiedFollowing,
    "Preceding": ql.Preceding,
}

adjustment_table = pd.DataFrame(
    {
        "Convention": name,
        "Unadjusted Date": ql_to_py(unadjusted_date),
        "Unadjusted Weekday": ql_to_py(unadjusted_date).strftime("%a"),
        "Adjusted Date": ql_to_py(calendar.adjust(unadjusted_date, convention)),
        "Adjusted Weekday": ql_to_py(
            calendar.adjust(unadjusted_date, convention)
        ).strftime("%a"),
    }
    for name, convention in adjustments.items()
)

adjustment_table
```

Recorded source output

	Convention	Unadjusted Date	Unadjusted Weekday	Adjusted Date	Adjusted Weekday
0	Unadjusted	2026-10-31	Sat	2026-10-31	Sat
1	Following	2026-10-31	Sat	2026-11-02	Mon
2	Modified Following	2026-10-31	Sat	2026-10-30	Fri
3	Preceding	2026-10-31	Sat	2026-10-30	Fri

11.2.4 End-of-month check

The end-of-month rule is discussed in Section 9.1; inspect which dates the calendar recognises as month-end.

```
sample_dates = [
    ql.Date(28, ql.August, 2026),
    ql.Date(31, ql.August, 2026),
    ql.Date(30, ql.September, 2026),
]

eom_table = pd.DataFrame({
    "Date": [ql_to_py(d) for d in sample_dates],
    "Is business day": [calendar.isBusinessDay(d) for d in sample_dates],
    "Is end of month": [calendar.isEndOfMonth(d) for d in sample_dates],
    "Calendar end of month": [ql_to_py(calendar.endOfMonth(d)) for d in sample_dates],
})

eom_table
```

Recorded source output

	Date	Is business day	Is end of month	Calendar end of month
0	2026-08-28	True	False	2026-08-31
1	2026-08-31	True	True	2026-08-31
2	2026-09-30	True	True	2026-09-30

Your turn 1 — Business-day conventions Choose a date other than 31 October 2026 for which `Following` and `Modified Following` produce different adjusted dates.

1. Apply `Following`, `Modified Following`, `Preceding` and `Unadjusted`.
2. Report the weekday of each adjusted date.
3. Be ready to explain why the two following conventions give different results.

Suggested time: 10 minutes.

```
# Write your code here
```

Discussion checkpoint

- Which convention crosses the month boundary?
- Which adjustment would you expect before running the code?
- Is the result determined by the calendar, the convention, or both?

11.3 Cycle 2: day counts and compounding

11.3.1 Guided example 2A: year fractions

Use the day-count definitions in Section 1.3.3 and the AFB choice explained in Section 9.2. Compare the same interval under four conventions.

```
start_date = ql.Date(24, ql.August, 2026)
end_date = ql.Date(28, ql.August, 2028)

day_counts = {
    "Actual/360": ql.Actual360(),
    "Actual/365 Fixed": ql.Actual365Fixed(),
    "Actual/Actual (AFB)": ql.ActualActual(ql.ActualActual.AFB),
    "30/360 (Bond Basis)": ql.Thirty360(ql.Thirty360.BondBasis),
}

day_count_table = pd.DataFrame(
    {
        "Convention": name,
        "Start Date": ql_to_py(start_date),
        "End Date": ql_to_py(end_date),
        "Calendar Days": end_date - start_date,
        "Year Fraction": dc.yearFraction(start_date, end_date),
    }
    for name, dc in day_counts.items()
)

day_count_table
```

Recorded source output

	Convention	Start Date	End Date	Calendar Days	Year Fraction
0	Actual/360	2026-08-24	2028-08-28	735	2.04166667
1	Actual/365 Fixed	2026-08-24	2028-08-28	735	2.01369863
2	Actual/Actual (AFB)	2026-08-24	2028-08-28	735	2.01095890
3	30/360 (Bond Basis)	2026-08-24	2028-08-28	735	2.01111111

Your turn 2 — Day-count conventions Choose a start date and an end date such that the interval includes 29 February. Compute the year fraction under:

- `Actual/360`;
- `Actual/365 Fixed`;
- `Actual/Actual (AFB)`;
- `30/360 (Bond Basis)`.

Present the results in a `DataFrame` and briefly explain why they differ.

Suggested time: 12 minutes.

```
# Write your code here
```

Discussion checkpoint

- Which convention produces the largest year fraction?
- Where does the leap day enter the calculation?
- Why can a small difference in τ matter in pricing?

Use the coupon-period notation and rate-definition qualification in Section 9.1 when interpreting the previous comparison.

11.3.2 Guided example 2B: equivalent quotations

Use the simple, periodic and continuous transformations in Section 2. Inspect the common discount factor rather than comparing the rates without their conventions.

```
def discount_from_rate(rate, tau, compounding="continuous", frequency=1):
    if compounding == "simple":
        return 1.0 / (1.0 + rate * tau)
    if compounding == "periodic":
        return (1.0 + rate / frequency) ** (-frequency * tau)
    if compounding == "continuous":
        return np.exp(-rate * tau)
    raise ValueError("Unknown compounding convention")


def rate_from_discount(discount_factor, tau, compounding="continuous", frequency=1):
    if tau <= 0:
        return np.nan
    if compounding == "simple":
        return (1.0 / discount_factor - 1.0) / tau
    if compounding == "periodic":
        return frequency * (discount_factor ** (-1.0 / (frequency * tau)) - 1.0)
    if compounding == "continuous":
        return -np.log(discount_factor) / tau
    raise ValueError("Unknown compounding convention")
```

```
tau_example = 5.0
discount_example = 0.85

equivalent_rates = pd.DataFrame({
    "Convention": ["Simple", "Annual", "Semiannual", "Continuous"],
    "Rate": [
        rate_from_discount(discount_example, tau_example, "simple"),
        rate_from_discount(discount_example, tau_example, "periodic", 1),
        rate_from_discount(discount_example, tau_example, "periodic", 2),
        rate_from_discount(discount_example, tau_example, "continuous"),
    ],
})
equivalent_rates["Rate Percent"] = 100 * equivalent_rates["Rate"]
equivalent_rates
```

Recorded source output

	Convention	Rate	Rate Percent
0	Simple	0.03529412	3.52941176
1	Annual	0.03303780	3.30378041
2	Semiannual	0.03276935	3.27693466
3	Continuous	0.03250379	3.25037859

Your turn 3 — Equivalent interest rates For a five-year discount factor $P(t,T) = 0.85$, calculate the equivalent:

- simply compounded zero rate;
- annually compounded zero rate;
- semiannually compounded zero rate;
- continuously compounded zero rate.

Verify numerically that all four rates reproduce the same discount factor.

Suggested time: 12 minutes.

```
# Write your code here
```

Discussion checkpoint

- Why are the four rates different?
- Which quantity remains unchanged?
- Which rate is the largest in this example, and why?

11.4 Cycle 3: coupon schedules

11.4.1 Guided schedule construction

Apply the effective-date, backward-generation and current-period distinctions in Section 9.1. The next cell keeps the construction visible for modification.

```
previous_coupon_date = ql.Date(15, ql.June, 2026)
maturity_date = ql.Date(15, ql.December, 2029)
tenor = ql.Period(6, ql.Months)
day_rolling = ql.ModifiedFollowing
termination_day_rolling = ql.ModifiedFollowing
date_generation = ql.DateGeneration.Backward
end_of_month = False

schedule = ql.Schedule(
    previous_coupon_date,  # Start of the current coupon period
    maturity_date,
    tenor,
    calendar,
    day_rolling,
    termination_day_rolling,
    date_generation,
    end_of_month,
)

schedule_dates = list(schedule)
accrual_day_count = ql.Actual360()

schedule_table = pd.DataFrame({
    "Period": np.arange(1, len(schedule_dates)),
    "Coupon Period Start": [ql_to_py(d) for d in schedule_dates[:-1]],
    "Coupon Period End": [ql_to_py(d) for d in schedule_dates[1:]],
    "Tau_i": [
        accrual_day_count.yearFraction(start, end)
        for start, end in zip(schedule_dates[:-1], schedule_dates[1:])
    ],
})

schedule_table
```

Recorded source output

	Period	Coupon Period Start	Coupon Period End	Tau_i
0	1	2026-06-15	2026-12-15	0.50833333
1	2	2026-12-15	2027-06-15	0.50555556
2	3	2027-06-15	2027-12-15	0.50833333
3	4	2027-12-15	2028-06-15	0.50833333
4	5	2028-06-15	2028-12-15	0.50833333
5	6	2028-12-15	2029-06-15	0.50555556
6	7	2029-06-15	2029-12-17	0.51388889

Your turn 4 — Coupon schedules Change **one** input in the schedule: maturity date, end-of-month rule or business-day convention. Keep backward generation. Then:

1. generate the new schedule;
2. calculate the accrual fractions τ_i;
3. identify any stub period;
4. compare the result with the original schedule.

Different groups may choose different inputs.

Suggested time: 15 minutes.

```
# Write your code here
```

Discussion checkpoint

1. Is the first coupon period regular or a stub?
2. Are all the accrual fractions τ_i equal?
3. Why does the schedule begin before the settlement date?
4. What changes when the maturity date falls on a weekend?

11.5 Cycle 4: reading and validating a market curve

11.5.1 Guided example 4A: import the supplied LSEG curve

The full market settings and single-curve qualification are collected once in Section 9.2. Inspect the imported columns before proceeding.

```
ZERO_CURVE_FILE = DATA_DIR / "lseg_euribor_6m_zero_curve_2026-08-24.xlsx"
FORWARD_CURVE_FILE = DATA_DIR / "lseg_euribor_6m_forward_curve_2026-08-24.xlsx"

raw_curve = pd.read_excel(
    ZERO_CURVE_FILE,
    sheet_name="Table",
    header=1,
    usecols="D:I",
)

raw_curve.head()
```

Recorded source output

	Tenor	End Date	Rate Percent	Discount Factor	Instrument Code 1	Mid 1
0	0D	2026-08-24	2.30739657	1.00000000	NaN	NaN
1	1D	2026-08-25	2.30739657	0.99993750	EUROND=	2.25000000
2	1W	2026-09-02	2.24630482	0.99945240	EURIBORSWD=	NaN
3	1M	2026-09-28	2.27658346	0.99784378	EURIBOR1MD=	NaN
4	3M	2026-11-26	2.59043047	0.99343534	EURIBOR3MD=	NaN

```
curve = raw_curve.rename(columns={
    "Tenor": "Tenor",
    "End Date": "Maturity Date",
    "Rate Percent": "Zero Rate Percent",
    "Discount Factor": "LSEG Discount Factor",
    "Instrument Code 1": "Instrument Code",
    "Mid 1": "Instrument Mid",
}).copy()

curve = curve.dropna(subset=["Maturity Date", "LSEG Discount Factor"])
curve["Maturity Date"] = pd.to_datetime(curve["Maturity Date"])
curve["Zero Rate"] = curve["Zero Rate Percent"] / 100.0
curve = curve.reset_index(drop=True)
```

```
curve.head(10)
```

Recorded source output

	Tenor	Maturity Date	Zero Rate Percent	LSEG Discount Factor	Instrument Code	Instrument Mid	Zero Rate
0	0D	2026-08-24	2.30739657	1.00000000	NaN	NaN	0.02307397
1	1D	2026-08-25	2.30739657	0.99993750	EUROND=	2.25000000	0.02307397
2	1W	2026-09-02	2.24630482	0.99945240	EURIBORSWD=	NaN	0.02246305
3	1M	2026-09-28	2.27658346	0.99784378	EURIBOR1MD=	NaN	0.02276583
4	3M	2026-11-26	2.59043047	0.99343534	EURIBOR3MD=	NaN	0.02590430
5	4M	2026-12-16	2.65325468	0.99185451	NaN	NaN	0.02653255
6	cm2	2027-03-16	2.75197777	0.98494145	FEIcm2	NaN	0.02751978
7	cm3	2027-06-17	2.83459838	0.97751246	FEIcm3	NaN	0.02834598
8	cm4	2027-09-16	2.89317644	0.97013663	FEIcm4	NaN	0.02893176
9	cm5	2027-12-15	2.93012344	0.96288515	FEIcm5	NaN	0.02930123

11.5.2 Guided example 4B: reconstruct discount factors

Use annual compounding and Actual/Actual AFB as explained in Section 9.2. Convert percent quotes to decimal rates and compare against the supplied factors.

```
lseg_zero_day_count = ql.ActualActual(ql.ActualActual.AFB)

curve["Tau Act/Act AFB"] = curve["Maturity Date"].apply(
    lambda maturity: lseg_zero_day_count.yearFraction(
        trade_date_ql, py_to_ql(maturity)
    )
)

curve["Python Discount Factor"] = np.where(
    curve["Tau Act/Act AFB"] == 0,
    1.0,
    discount_from_rate(
        curve["Zero Rate"],
        curve["Tau Act/Act AFB"],
        compounding="periodic",
        frequency=1,
    ),
)

curve["DF Error"] = (
    curve["Python Discount Factor"] - curve["LSEG Discount Factor"]
)

curve[[
    "Tenor", "Maturity Date", "Tau Act/Act AFB", "Zero Rate Percent",
    "LSEG Discount Factor", "Python Discount Factor", "DF Error"
]].head(12)
```

Recorded source output

	Tenor	Maturity Date	Tau Act/Act AFB	Zero Rate Percent	LSEG Discount Factor	Python Discount Factor	DF Error
0	0D	2026-08-24	0.00000000	2.30739657	1.00000000	1.00000000	0.00000000
1	1D	2026-08-25	0.00273973	2.30739657	0.99993750	0.99993750	-0.00000000
2	1W	2026-09-02	0.02465753	2.24630482	0.99945240	0.99945240	-0.00000000
3	1M	2026-09-28	0.09589041	2.27658346	0.99784378	0.99784378	0.00000000
4	3M	2026-11-26	0.25753425	2.59043047	0.99343534	0.99343534	0.00000000
5	4M	2026-12-16	0.31232877	2.65325468	0.99185451	0.99185451	-0.00000000
6	cm2	2027-03-16	0.55890411	2.75197777	0.98494145	0.98494145	-0.00000000
7	cm3	2027-06-17	0.81369863	2.83459838	0.97751246	0.97751246	0.00000000
8	cm4	2027-09-16	1.06301370	2.89317644	0.97013663	0.97013663	-0.00000000
9	cm5	2027-12-15	1.30958904	2.93012344	0.96288515	0.96288515	-0.00000000
10	cm6	2028-03-15	1.55616438	2.95673173	0.95566819	0.95566819	-0.00000000
11	2Y	2028-08-28	2.01095890	3.07776329	0.94086162	0.94086162	-0.00000000

```
print(f"Maximum absolute DF error: {curve['DF Error'].abs().max():.3e}")

assert curve["DF Error"].abs().max() < 1e-12
```

```
Maximum absolute DF error: 9.437e-16
```

Your turn 5 — Validate selected LSEG discount factors Select the maturities 1Y, 5Y, 10Y and 30Y. For each maturity:

1. report $\tau(t,T)$ under `Actual/Actual (AFB)`;
2. reconstruct the discount factor from the LSEG zero rate;
3. compute the absolute pricing error;
4. verify that the error is negligible.

Suggested time: 15 minutes.

```
# Write your code here
```

Discussion checkpoint

- What does a negligible error validate?
- Would the same result hold under `Actual/360` or semiannual compounding?
- Why is an automated assertion useful here?

Interpret the assertion using the reproducibility criterion in Section 9.2; investigate a failure before proceeding.

11.5.3 Guided example 4C: compare representations

Use the equivalent-rate relations in Section 2.1.5. The following table and plot are the notebook's supplied outputs.

```
positive_tau = curve["Tau Act/Act AFB"] > 0

curve["Simple Zero Rate"] = np.nan
curve["Continuous Zero Rate"] = np.nan

curve.loc[positive_tau, "Simple Zero Rate"] = (
    1.0 / curve.loc[positive_tau, "LSEG Discount Factor"] - 1.0
) / curve.loc[positive_tau, "Tau Act/Act AFB"]

curve.loc[positive_tau, "Continuous Zero Rate"] = (
    -np.log(curve.loc[positive_tau, "LSEG Discount Factor"])
    / curve.loc[positive_tau, "Tau Act/Act AFB"]
)

selected_tenors = ["1Y", "2Y", "5Y", "10Y", "20Y", "30Y"]
comparison = curve.loc[curve["Tenor"].isin(selected_tenors), [
    "Tenor", "Maturity Date", "LSEG Discount Factor", "Zero Rate",
    "Simple Zero Rate", "Continuous Zero Rate"
]].copy()

comparison
```

Recorded source output

	Tenor	Maturity Date	LSEG Discount Factor	Zero Rate	Simple Zero Rate	Continuous Zero Rate
11	2Y	2028-08-28	0.94086162	0.03077763	0.03125651	0.03031350
14	5Y	2031-08-26	0.85724236	0.03125159	0.03326979	0.03077320
19	10Y	2036-08-26	0.72340916	0.03288959	0.03821342	0.03236030
25	20Y	2046-08-27	0.50463819	0.03477254	0.04906073	0.03418163
27	30Y	2056-08-28	0.37312809	0.03339461	0.05598104	0.03284912

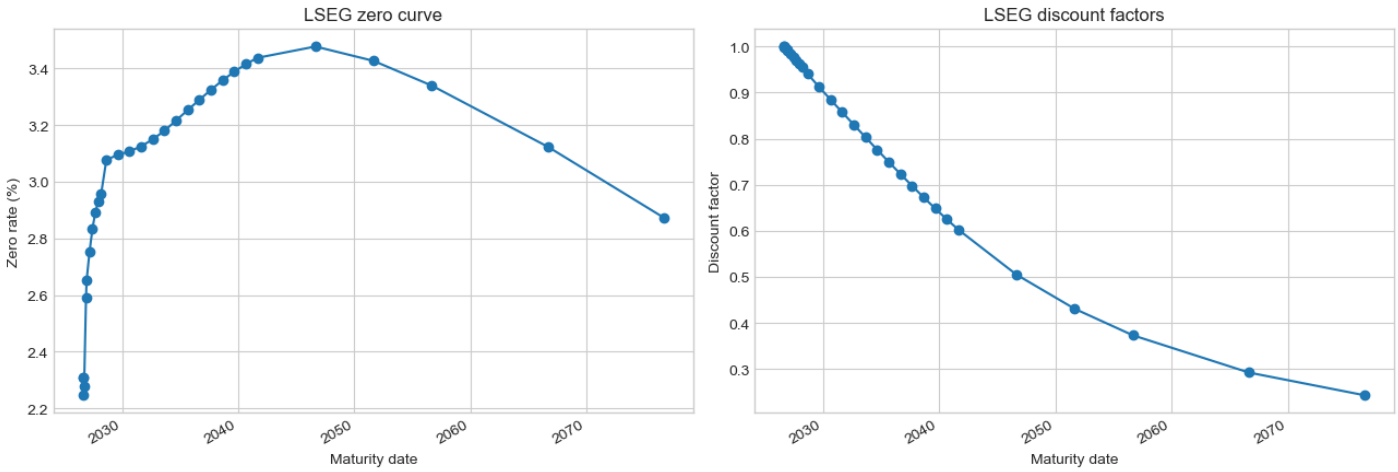
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].plot(curve["Maturity Date"], 100 * curve["Zero Rate"], marker="o")
axes[0].set_title("LSEG zero curve")
axes[0].set_ylabel("Zero rate (%)")
axes[0].set_xlabel("Maturity date")

axes[1].plot(curve["Maturity Date"], curve["LSEG Discount Factor"], marker="o")
axes[1].set_title("LSEG discount factors")
axes[1].set_ylabel("Discount factor")
axes[1].set_xlabel("Maturity date")

fig.autofmt_xdate()
fig.tight_layout()
plt.show()
```

Recorded source output



Your turn 6 — Interpret the term structure Using the LSEG data:

1. identify the maturity at which the zero rate reaches its maximum;
2. describe the short, medium and long ends of the curve;
3. explain why discount factors generally decrease with maturity;
4. discuss the unusual behaviour beyond 30 years.

Write four to six lines of economic interpretation. Do not merely describe the mechanics of the graph.

Suggested time: 10 minutes, or complete after class.

Plot both zero rates and discount factors before writing the requested economic interpretation.

> **Your interpretation:**
> Write here.

11.5.4 Guided example 4D: node-to-node forwards

Apply the simple forward formula in Section 3.3.1 to the consecutive exported dates, with the interval qualification in Section 9.2.

```
forward_day_count = ql.Actual360()

node_forwards = curve.loc[positive_tau, [
    "Tenor", "Maturity Date", "LSEG Discount Factor"
]].copy().reset_index(drop=True)

node_forwards["Previous Date"] = node_forwards["Maturity Date"].shift(1)
node_forwards["Previous Discount Factor"] = node_forwards["LSEG Discount Factor"].shift(1)

node_forwards["Tau_i"] = node_forwards.apply(
    lambda row: np.nan if pd.isna(row["Previous Date"]) else
    forward_day_count.yearFraction(
        py_to_ql(row["Previous Date"]), py_to_ql(row["Maturity Date"])
    ),
    axis=1,
)

node_forwards["Forward Rate"] = (
    node_forwards["Previous Discount Factor"]
    / node_forwards["LSEG Discount Factor"] - 1.0
) / node_forwards["Tau_i"]

node_forwards.head(12)
```

Recorded source output

	Tenor	Maturity Date	LSEG Discount Factor	Previous Date	Previous Discount Factor	Tau_i	Forward Rate
0	1D	2026-08-25	0.99993750	NaT	NaN	NaN	NaN
1	1W	2026-09-02	0.99945240	2026-08-25	0.99993750	0.02222222	0.02184182
2	1M	2026-09-28	0.99784378	2026-09-02	0.99945240	0.07222222	0.02232126
3	3M	2026-11-26	0.99343534	2026-09-28	0.99784378	0.16388889	0.02707671
4	4M	2026-12-16	0.99185451	2026-11-26	0.99343534	0.05555556	0.02868866
5	cm2	2027-03-16	0.98494145	2026-12-16	0.99185451	0.25000000	0.02807500
6	cm3	2027-06-17	0.97751246	2027-03-16	0.98494145	0.25833333	0.02941892
7	cm4	2027-09-16	0.97013663	2027-06-17	0.97751246	0.25277778	0.03007735
8	cm5	2027-12-15	0.96288515	2027-09-16	0.97013663	0.25000000	0.03012396
9	cm6	2028-03-15	0.95566819	2027-12-15	0.96288515	0.25277778	0.02987500
10	2Y	2028-08-28	0.94086162	2028-03-15	0.95566819	0.46111111	0.03412897
11	3Y	2029-08-27	0.91238480	2028-08-28	0.94086162	1.01111111	0.03086843

11.5.5 Read the separate 6M forward analysis

Compare its start and end dates with the previous calculation, using the vendor-comparison criteria in Section 9.2.

```
raw_forward = pd.read_excel(FORWARD_CURVE_FILE, sheet_name="Forward Curve Analysis", header=None)

# Columns in the LSEG export:
# D = start interval, E = start date, J = 6M end date, K = 6M forward rate.
lseg_forward_6m = raw_forward.iloc[6:, [3, 4, 9, 10]].copy()
lseg_forward_6m.columns = ["Start Interval", "Start Date", "End Date", "LSEG Forward Percent"]
lseg_forward_6m = lseg_forward_6m.dropna(subset=["Start Date", "End Date", "LSEG Forward Percent"])
lseg_forward_6m["Start Date"] = pd.to_datetime(lseg_forward_6m["Start Date"])
lseg_forward_6m["End Date"] = pd.to_datetime(lseg_forward_6m["End Date"])
lseg_forward_6m["Tau Act/360"] = lseg_forward_6m.apply(
    lambda row: forward_day_count.yearFraction(
        py_to_ql(row["Start Date"]), py_to_ql(row["End Date"])
    ),
    axis=1,
)

lseg_forward_6m.head(10)
```

Recorded source output

	Start Interval	Start Date	End Date	LSEG Forward Percent	Tau Act/360
6	2026-08-24 00:00:00	2026-08-26	2027-02-26	2.64454721	0.51111111
7	1W	2026-09-02	2027-03-02	2.66695967	0.50277778
8	1M	2026-09-28	2027-03-30	2.76121469	0.50833333
9	2M	2026-10-26	2027-04-26	2.79915159	0.50555556
10	3M	2026-11-26	2027-05-26	2.83260672	0.50277778
11	4M	2026-12-28	2027-06-28	2.90023330	0.50555556
12	5M	2027-01-26	2027-07-26	2.93518477	0.50277778
13	6M	2027-02-26	2027-08-31	2.96520508	0.51666667
14	9M	2027-05-26	2027-11-26	3.01038844	0.51111111
15	1Y	2027-08-26	2028-02-28	2.99410998	0.51666667

Diagnostic checkpoint. Explain any difference using Section 9.2; do not treat sparse-node forwards as an identical vendor 6M instrument. A later explicit curve-object comparison must control every convention.

11.6 Suggested experiments

Change the day-count convention and measure the effect on reconstructed factors. Replace annual compounding by semiannual compounding. Change schedule maturity and inspect the stub. Restrict plots to 30 years and discuss the longer end separately. Use the conclusions of Section 9 to explain the results.

11.6.1 Homework — A convention audit

Prepare a short, reproducible notebook that documents the convention choices for the following fixed-income problem.

Part A — Settlement and adjusted dates Use 24 August 2026 as the trade date and the TARGET calendar.

1. Compute the two-business-day settlement date.
2. Choose a contractual date that falls on a non-business day.
3. Adjust it under `Following`, `Modified Following` and `Preceding`.
4. Report the weekday before and after each adjustment.

Part B — Coupon schedule Construct a semiannual schedule from the settlement date to 31 August 2031 using:

- TARGET calendar;
- `Modified Following`;
- `Actual/360` accrual fractions;
- backward date generation;
- no end-of-month rule.

Report the dates and the accrual fractions τ_i. Identify any irregular period and explain why it appears. Then generate the same schedule using forward generation and compare the results.

Part C — Equivalent rates Extract the 5Y LSEG discount factor. Calculate the equivalent:

- simply compounded zero rate;
- annually compounded zero rate;
- semiannually compounded zero rate;
- continuously compounded zero rate.

Reconstruct the discount factor from each rate and report the absolute errors.

Part D — Validation note Write a short paragraph (approximately 100 words) explaining why a rate cannot be interpreted or validated without its dates, day-count convention and compounding convention.

Deliverable Submit one executed notebook containing:

- clearly labelled input assumptions;
- the required tables;
- reproducibility checks using `assert` where appropriate;
- concise comments on the financial meaning of the results.

Expected working time: 45–60 minutes.

12 Linear-product laboratory: original student notebooks

The new laboratory contains eight notebooks and two reusable modules. Student TODO cells and their explicit NotImplementedError stops are preserved. The complete reference module is supplied below; no completed instructor notebooks were included in the archive, despite a source mention of that folder. No notebook was re-executed for this document. Financial foundations are integrated in Section 9; operational instructions and exercise prompts follow here.

12.1 00 library architecture

Fixed Income QFI 2026–27 · Lab

Before writing code, discuss the first column. Suggest a design choice before revealing the second.

Requirement	Design implication
Price several products with the same curve	Keep curves separate from pricing
Change curve construction without changing pricing	Agree on `discount(date)` as the interface
Change a contract without rewriting code	Pass contract data as arguments
Inspect how a price is obtained	Return a value and a cash-flow table
Use different fixed and floating conventions	Build each leg's schedule independently
Reuse a validated calculation	Move the function from notebook to a `.py` module
Detect mistakes	Check financial identities and reproducibility

12.1.1 Three possible organisations

Organisation	Advantage	Cost
Everything in notebooks	Immediate exploration	Duplicated code and hidden execution state
Functions in modules, examples in notebooks	Reuse with a small amount of structure	Imports and reloads
Custom classes for curves and products	Data and behaviour travel together	More concepts and design decisions

Our initial choice: functions and two small modules. We already use QuantLib objects; we do not need to design our own class hierarchy yet.

12.1.2 Responsibility of each file

File	Responsibility
`curves.py`	Provide discount factors at dates
`FI_library.py`	Build schedules and calculate prices
`data/curve_demo.csv`	Explicitly synthetic curve pillars
`build_*.ipynb`	Derive a calculation, complete its core, verify it
`test_library.ipynb`	Call the reusable functions in new examples

Discussion: If we replace interpolation by bootstrap, which files should change?

Discussion: Should a pricing function read Excel, draw a plot, or save a file? Which of those tasks belong in a notebook?

12.1.3 Our workflow

1. Specify an example and predict its behaviour.
2. Complete the central calculation in the notebook.
3. Inspect intermediate quantities and check financial identities.
4. Put the calculation inside a function with explicit inputs and outputs.
5. Move the validated function into the `.py` module using Spyder.
6. Import the module, restart the kernel and run all cells.

The supplied modules provide the reference implementation. Work on the student TODOs first. The source mentions completed instructor notebooks, but that folder was not supplied in this archive.

The financial scope, date distinctions, settlement rebasing and regular-schedule assumptions are integrated in Part I, Section 9.

12.2 01 python setup and syntax

Fixed Income QFI 2026–27

These notes collect the explanations discussed while reading the curve, coupon-bond, FRN and swap code. Prefer the simplest readable expression; add a conversion or abstraction only when it has a clear purpose.

12.2.1 1. Jupyter, Python and the kernel

A Jupyter notebook (`.ipynb`) is a document containing text, formulas and executable code cells. A Python **kernel** executes the cells and remembers variables, functions and imported modules. Restarting the kernel clears that memory, not the saved notebook. For a final check, restart and run all cells in order.

Spyder is another environment for editing and executing Python, including our `.py` modules. Notebook and Spyder consoles normally have separate Python sessions: changing a variable in one does not update the other.

12.2.2 2. Folders, working directory and module search

Concept	Meaning
Notebook folder	Where the `.ipynb` file is saved
Working directory	Starting point for relative file paths; inspect it with `Path.cwd()`
`sys.path`	List of folders Python searches when importing modules

The notebook folder and working directory often coincide, but not always.

```
ROOT = Path.cwd().resolve()
```

This stores the absolute working-directory path; assigning `ROOT` does not change the working directory. `ROOT.parent` is its parent folder. For a `Path` object, `/` joins path components:

```
ROOT / "data" / "curve_demo.csv"
```

This is an explicit path from `ROOT`, independent of later changes to the working directory. `.exists()` checks whether a path exists. Our setup searches the current folder and then its parent for `FI_library.py`; it does not search the entire computer. Renaming the main lab folder does not invalidate this search.

`sys.path` is a list in memory, not a system folder. Adding `str(ROOT)` with `sys.path.insert(0, str(ROOT))` tells Python where to find our module, without copying or moving it. Position zero gives the folder priority. The change applies only to the current session; rerun setup after restarting the kernel.

MATLAB comparison: working directory corresponds to *Current Folder*; adding a folder to `sys.path` plays the role of `addpath`.

12.2.3 3. Imports, reload and display

```
import FI_library as fi
from importlib import reload
reload(fi)
```

`fi` is a short name for the imported module. After editing and saving `FI_library.py` in Spyder, `reload(fi)` rereads the module for subsequent calls such as `fi.get_price_cb(...)`. Repeating `import` alone normally uses the cached module. Previously computed prices do not update themselves: rerun their calculation cells. A kernel restart followed by running all cells is the clean final check.

```
from IPython.display import display
display(table)
```

`display` shows a DataFrame as a formatted table. `print` produces a text representation. Jupyter also displays the last expression in a cell automatically; `display` lets us show several objects or an object in the middle of a cell. It does not change the data or perform pricing.

12.2.4 4. Integers, floats and NumPy arrays

In Python, `2` is an integer (`int`) and `2.0` is a floating-point number (`float`). In MATLAB both are normally `double`. Python handles many conversions automatically: `2 / 3` and `2 * 0.5` return floats.

Expression	Purpose
`np.array(values, dtype=float)`	Store all array elements as floating-point numbers
`float(value)`	Convert one scalar to a native Python float

An integer NumPy array retains its integer type when an element is assigned: assigning `1.5` to an element of `np.array([0, 1, 2])` stores `1`. Use floating arrays for fractional financial quantities. The final `float(...)` in `discount(date)` makes the scalar return type explicit; the interpolated value is already floating-point.

12.2.5 5. Copies and assignments

```
factors = np.array(discount_factors, dtype=float, copy=True)
table = schedule.copy()
```

These create independent data for this calculation. Changing the internal `factors` array does not overwrite the input discount-factor array. Changing the original times later does not change the curve's copied times. With `np.array`, copying is already the default; `copy=True` states the intention explicitly.

For our DataFrames, `.copy()` lets us add pricing columns without modifying the supplied schedule. By contrast, `table = schedule` gives two names to the same object. Pandas' default copy does not recursively duplicate arbitrary objects inside cells; we do not mutate the QuantLib date objects stored there.

Copies are not automatically read-only. To update the curve's input data coherently, construct a new curve with the new arrays.

12.2.6 6. Indexing, masks and table calculations

Python and NumPy indexing starts at zero. `factors[0]` is the first element; `times[-1]` is the last. Assignment such as `factors[0] = 1.0` overwrites that element. In pandas, use `.iloc` when you mean a position explicitly.

Expression	Meaning
`table["YearFrac"]`	Column as a pandas Series
`table.loc[mask, "YearFrac"]`	Values in the named column where the boolean mask is true
`table.iloc[0]`	First row, regardless of its index label
`table.iloc[[0, 2], 1]`	Second column of the first and third rows
`table.iloc[0:3]`	First three rows; position 3 is excluded

`.loc` uses labels or a boolean mask; `.iloc` uses positions. A Series mask is aligned by index labels, so construct it from the same table.

```
positive = pillars["Time"] > 0
pillars.loc[positive, "Time"]
```

This selects positive pillar times. It avoids dividing by zero when calculating zero rates.

Use pandas arithmetic directly when it is clear:

```
table["Coupon"] = notional * rate * table["YearFrac"]
```

No conversion is required for this multiplication by scalars. `.to_numpy(dtype=float)` extracts a floating NumPy array when an array is specifically needed. NumPy combines arrays by position; arithmetic between pandas Series aligns index labels. Do not convert every column merely for uniformity.

The final principal repayment can be added to a Series with:

```
cash_flow.iloc[-1] += notional
```

This explicitly selects the last position, rather than an index label.

12.2.7 7. Scalar functions and repeated calls

Choose the simplest way to use table columns

Task	Preferred approach	Reason
Apply the same arithmetic to a whole column	Direct pandas arithmetic	No explicit loop or conversion is needed
Read a few columns together, one row at a time	`zip(...)`	Assign the required values directly to named variables
Pass or process complete rows as dictionaries	`to_dict("records")`	Each row becomes a record with named fields

For the floating-rate calculation, we read three columns together:

```
rates = []
for start, end, tau in zip(
    table["Start Date"], table["End Date"], table["YearFrac"]
):
    if start <= settlement_date:
        rate = current_fixing
    else:
        rate = (forwarding(start) / forwarding(end) - 1.0) / tau
    rates.append(rate)
```

`zip` takes the first element of each column together, then the second, and so on. The loop assigns the three values to `start`, `end` and `tau`. These columns belong to the same table and have the same length. The example assumes the observed fixing has been supplied; the library also checks that it is available for a started period.

By contrast, `table.to_dict("records")` creates a list of dictionaries, one per row. Column names become dictionary keys; the DataFrame index is omitted:

```
for row in table.to_dict("records"):
    print(row["Start Date"], row["End Date"], row["YearFrac"])
```

This representation is useful when a receiving function or a data exchange format needs complete records with named fields, or when accessing many fields by name makes the code clearer. Creating these dictionaries is unnecessary for our three-column rate loop, so we use `zip` there. It does not replace direct column arithmetic for simple calculations such as `notional * rate * table["YearFrac"]`.

Evaluate a scalar function on several dates Our curve accepts one date at a time. To evaluate several dates:

```
np.array([discount(date) for date in dates], dtype=float)
```

The list comprehension means "for each date, calculate its discount factor". NumPy also supports vector arithmetic, for example `amounts * factors`. Iteration is needed here because the curve's date interface is scalar, not because Python lacks vector operations.

`make_discount_curve(...)` returns a function, not a price. It remembers its construction data: call it as `discount(payment_date)`. Later, use a QuantLib curve's `discount` method through the same interface; no second interpolation is needed.

12.2.8 Financial and curve conventions

The curve checks, date conventions, observed-fixing and settlement rules are integrated once in Part I, Section 9. Use that account when reading the unmodified source functions.

Exceptions and assertions. `raise ValueError(...)` stops a calculation with an explanatory message for an invalid input. `assert` checks a development expectation and raises `AssertionError` if it fails; Python optimisation can disable assertions, so essential input validation uses explicit exceptions. `np.testing.assert_allclose` checks numerical agreement within tolerances rather than exact floating-point equality. The supplied validation checks establish input consistency, rather than calibration to market prices.

12.2.9 11. Plots

Omit `figsize` to use Matplotlib's configured default size. `layout="constrained"` adjusts internal spacing for titles and labels; it does not automatically resize a static figure to the notebook window. `plt.show()` displays the figures. Jupyter usually displays them automatically at the end of the cell, but the explicit call makes the intention clear.

12.2.10 12. Our working rule

Keep technical setup, market inputs, contract inputs, and the central pricing calculation in separate cells. Write only the few central financial lines together; provide input, output and validation code. Validate in the notebook, then move the function to its module. Keep longer explanations in these notes, rather than inserting them into every pricing cell.

12.3 main irc

Inspect the synthetic pillars and the reference-date identity; plot discount factors and continuous spot rates, excluding zero time from the rate calculation.

12.3.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.3.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

```
display(pillars)
print('P(t,t) =', discount(reference_date))
print('P(t,19 Oct 2028) =', discount(ql.Date(19, 10, 2028)))
assert discount(reference_date) == 1.0
```

Implementation check. Replace the interpolator with a compatible QuantLib discount method without changing the pricers; interpret settlement rebasing using Section 9.4.

```
import matplotlib.pyplot as plt
positive = pillars['Time'] > 0
fig, axes = plt.subplots(1, 2, layout="constrained")
axes[0].plot(pillars['Time'], pillars['Discount Factor'], 'o-')
axes[0].set(xlabel='Years from reference date', ylabel='Discount factor')
axes[1].plot(pillars.loc[positive, 'Time'],
             -np.log(pillars.loc[positive, 'Discount Factor']) / pillars.loc[positive, 'Time'], 'o-')
axes[1].set(xlabel='Years from reference date', ylabel='Continuous spot rate')
for ax in axes: ax.grid(True)
plt.show()
# The spot rate at exactly zero is not computed as 0/0 or assigned a value of zero.
```

12.4 build get schedule dataframe

Recover the current regular coupon period and complete its contractual year fractions. The date and regular-period assumptions are in Section 9.1.

12.4.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.4.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

12.4.3 Contract inputs and schedule

```
maturity_date = ql.Date(20, 7, 2028)
tenor = ql.Period(6, ql.Months)
coupon_day_count = ql.Actual360()
calendar = ql.TARGET()

# Use an earlier temporary boundary to recover the current regular period.
# It is not the contractual effective date. Its artificial first stub is past.
schedule_start = settlement_date - ql.Period(2 * tenor.length(), tenor.units())
schedule = ql.Schedule(schedule_start, maturity_date, tenor, calendar,
                       ql.ModifiedFollowing, ql.ModifiedFollowing,
                       ql.DateGeneration.Backward, False)
dates = list(schedule)
periods = [(start, end) for start, end in zip(dates[:-1], dates[1:])
           if end > settlement_date]
starts = [start for start, end in periods]
ends = [end for start, end in periods]
```

Complete the coupon year fractions. Use the contract day count, not the curve day count.

```
# TODO: complete the central calculation described above.
raise NotImplementedError("Complete this cell before continuing.")
```

```
table = pd.DataFrame({'Start Date': starts, 'End Date': ends,
                      'Pay Date': ends, 'YearFrac': year_frac})
display(table)
assert table.iloc[0]['Start Date'] < settlement_date
library_table = fi.get_schedule_dataframe(maturity_date, settlement_date, tenor, coupon_day_count)
np.testing.assert_allclose(table['YearFrac'], library_table['YearFrac'])
assert list(table['Start Date']) == list(library_table['Start Date'])
assert list(table['Pay Date']) == list(library_table['Pay Date'])
```

The returned elapsed accrual supports Section 9.4. The comprehension calls a scalar QuantLib method for each date pair; `np.vectorize` would wrap the same calls rather than create native array operations.

12.5 build get price cb

Predict how a higher coupon affects a fixed-rate bond price, then complete the coupon and principal cash flows. Interpret dirty and clean settlement values using Section 9.4.

12.5.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.5.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

12.5.3 Contract inputs and schedule

```
notional = 100.0
rate = 0.03
coupon_day_count = ql.Actual360()
schedule = fi.get_schedule_dataframe(ql.Date(20, 7, 2028), settlement_date, ql.Period(6, ql.Months), coupon_day_count)
table = schedule.copy()
year_frac = table['YearFrac']
df_discounting = get_discount_factors(discount, table['Pay Date']) / discount(settlement_date)
display(table)
```

Complete only the cash flows and their present values. Where does the principal repayment enter?

```
# TODO: complete the central calculation described above.
raise NotImplementedError("Complete this cell before continuing.")
```

```
table['Coupon Rate'] = rate
table['Discount Factor'] = df_discounting
table['Cash Flow'] = cash_flow
table['Disc. Cash Flow'] = discounted_cash_flow
display(table)
accrued_interest = notional * rate * table.iloc[0]['Elapsed YearFrac']
print('Dirty value:', price)
print('Accrued interest:', accrued_interest)
print('Clean value:', price - accrued_interest)
# This accrued-interest formula follows this lab's simple coupon convention.
```

12.5.4 Validate, then move to a module

The reference function below contains the same calculation. In the classroom, transfer the validated function to `FI_library.py` using Spyder, reload the module, then compare.

```
library_price, library_table = fi.get_price_cb(schedule, discount, settlement_date, notional, rate)
np.testing.assert_allclose(price, library_price)
higher_price, _ = fi.get_price_cb(schedule, discount, settlement_date, notional, rate + 0.01)
assert higher_price > library_price
# Financial check: a zero-coupon bond has only the redemption value.
zero_coupon_price, _ = fi.get_price_cb(schedule, discount, settlement_date, notional, 0.0)
np.testing.assert_allclose(zero_coupon_price,
    notional * discount(schedule.iloc[-1]['Pay Date']) / discount(settlement_date))
```

12.6 build get price frn

Complete floating coupons using their own accrual fractions and the observed first fixing. Apply the spread and test the reset-date par identity in Section 9.5.

12.6.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.6.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

12.6.3 Contract inputs and schedule

```
notional = 100.0
spread = 10 / 10000  # 10 basis points
current_fixing = 0.025  # synthetic observed benchmark fixing, BEFORE spread
schedule = fi.get_schedule_dataframe(ql.Date(20, 7, 2028), settlement_date, ql.Period(3, ql.Months), ql.Actual360())
table = schedule.copy()
year_frac = table['YearFrac']
rates = []
for start, end, tau in zip(
    table['Start Date'], table['End Date'], table['YearFrac']
):
    if start <= settlement_date:
        rate = current_fixing
    else:
        rate = (forwarding(start) / forwarding(end) - 1.0) / tau
    rates.append(rate)
benchmark_rate = np.array(rates, dtype=float)
display(table)
```

Complete the coupon rate, cash flows and price. Does the first coupon include the spread?

```
# TODO: complete the central calculation described above.
raise NotImplementedError("Complete this cell before continuing.")
```

```
table['Benchmark Rate'] = benchmark_rate
table['Spread'] = spread
table['Coupon Rate'] = coupon_rate
table['Discount Factor'] = df_discounting
table['Cash Flow'] = cash_flow
table['Disc. Cash Flow'] = df_discounting * cash_flow
display(table)
print('FRN dirty value:', price)
library_price, _ = fi.get_price_frn(schedule, discount, settlement_date,
    notional, spread, current_fixing, forwarding=forwarding)
np.testing.assert_allclose(price, library_price)
```

Financial identity to explain before running. Prove the reset-date, zero-spread FRN par result using Section 9.5. Explain why an old fixing can break it.

```
reset_schedule = fi.get_schedule_dataframe(ql.Date(19, 10, 2028), settlement_date, ql.Period(3, ql.Months), ql.Actual360())
reset_price, _ = fi.get_price_frn(reset_schedule, discount, settlement_date, notional, forwarding=forwarding)
np.testing.assert_allclose(reset_price, notional, atol=1e-10)
print('Reset-date, zero-spread FRN:', reset_price)
```

12.7 build get price swap

Predict the payer value when its fixed rate exceeds par. Complete both leg values, the payer value and par rate; use separate schedules and no principal exchange as in Section 9.5.

12.7.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.7.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

12.7.3 Contract inputs and schedule

```
notional = 1_000_000.0
fixed_rate = 0.03
# Start at a future date: no historical fixing is needed.
effective_date = ql.Date(19, 4, 2027)
maturity_date = ql.Date(19, 4, 2029)
fixed = fi.get_schedule_dataframe(maturity_date, settlement_date,
    ql.Period(1, ql.Years), ql.Thirty360(ql.Thirty360.BondBasis),
    effective_date=effective_date)
floating = fi.get_schedule_dataframe(maturity_date, settlement_date,
    ql.Period(3, ql.Months), ql.Actual360(), effective_date=effective_date)
benchmark_rate = fi.get_floating_rates(floating, forwarding, settlement_date)
fixed_df = get_discount_factors(discount, fixed['Pay Date']) / discount(settlement_date)
floating_df = get_discount_factors(discount, floating['Pay Date']) / discount(settlement_date)
display(fixed)
display(floating)
```

Complete the leg values, payer value and par rate. Should a principal repayment be added?

```
# TODO: complete the central calculation described above.
raise NotImplementedError("Complete this cell before continuing.")
```

```
fixed['Coupon Rate'] = fixed_rate
fixed['Cash Flow'] = notional * fixed_rate * fixed['YearFrac']
fixed['Discount Factor'] = fixed_df
fixed['Disc. Cash Flow'] = fixed['Cash Flow'] * fixed_df
```

```
floating['Benchmark Rate'] = benchmark_rate
floating['Spread'] = 0.0
floating['Coupon Rate'] = floating['Benchmark Rate'] + floating['Spread']
floating['Cash Flow'] = notional * floating['Coupon Rate'] * floating['YearFrac']
floating['Discount Factor'] = floating_df
floating['Disc. Cash Flow'] = floating['Cash Flow'] * floating_df

print('Fixed leg: amounts paid by the payer swap holder')
display(fixed)
print('Floating leg: amounts received by the payer swap holder')
display(floating)
print('Fixed leg PV:', fixed_pv)
print('Floating leg PV:', floating_pv)
print('Payer swap value (floating minus fixed):', value)
print('Par fixed rate:', par_rate)
np.testing.assert_allclose(fixed['Disc. Cash Flow'].sum(), fixed_pv)
np.testing.assert_allclose(floating['Disc. Cash Flow'].sum(), floating_pv)
```

12.7.4 Validation

Compare the hand calculation and both cash-flow tables with the library; verify par value, payer/receiver sign and the floating-leg identity.

```
library_value, library_par, fixed_table, floating_table = fi.get_price_swap(
    fixed, floating, discount, settlement_date, notional, fixed_rate,
    forwarding=forwarding)
np.testing.assert_allclose([value, par_rate], [library_value, library_par])
np.testing.assert_allclose(fixed['Cash Flow'], fixed_table['Cash Flow'])
np.testing.assert_allclose(floating['Cash Flow'], floating_table['Cash Flow'])
par_value, _, _, _ = fi.get_price_swap(fixed, floating, discount,
    settlement_date, notional, par_rate, forwarding=forwarding)
np.testing.assert_allclose(par_value, 0.0, atol=1e-8)
receiver_value, _, _, _ = fi.get_price_swap(fixed, floating, discount,
    settlement_date, notional, fixed_rate, forwarding=forwarding, payer=False)
np.testing.assert_allclose(value, -receiver_value)
np.testing.assert_allclose(floating_pv,
    notional * (discount(floating.iloc[0]['Start Date'])
                - discount(floating.iloc[-1]['End Date'])) / discount(settlement_date))
```

12.8 test library

Restart and run the function-call tests after editing the modules; these cells use the reference library rather than reimplementing it.

12.8.1 Imports and library setup

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import QuantLib as ql
from IPython.display import display
from importlib import reload

ROOT = Path.cwd().resolve()
if not (ROOT / "FI_library.py").exists():
    ROOT = ROOT.parent
if not (ROOT / "FI_library.py").exists():
    raise FileNotFoundError("Open the notebook from the lab folder or its student/instructor subfolder.")
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from curves import make_discount_curve, get_discount_factors
import FI_library as fi
reload(fi)
```

12.8.2 Market inputs and curve setup

Synthetic data; the curve time convention is independent of the coupon day count.

```
reference_date = ql.Date(19, 10, 2026)
curve_day_count = ql.Actual365Fixed()
pillars = pd.read_csv(ROOT / "data" / "curve_demo.csv")

# The constructor requires the anchor (0, 1); add it if the file omits it.
if pillars.iloc[0]['Time'] > 0:
    anchor = pd.DataFrame({'Time': [0.0], 'Discount Factor': [1.0]})
    pillars = pd.concat([anchor, pillars], ignore_index=True)

discount = make_discount_curve(pillars['Time'], pillars['Discount Factor'],
                               reference_date, curve_day_count)
forwarding = discount  # same curve initially; separate roles in pricing
settlement_date = reference_date
```

12.8.3 Contract inputs and schedule

```
notional = 100.0
schedule = fi.get_schedule_dataframe(ql.Date(19, 10, 2028), settlement_date, ql.Period(6, ql.Months), ql.Actual360())
cb_price, cb_table = fi.get_price_cb(schedule, discount, settlement_date, notional, 0.03)
frn_price, frn_table = fi.get_price_frn(schedule, discount, settlement_date, notional, forwarding=forwarding)
swap_price, par_rate, fixed_table, floating_table = fi.get_price_swap(
    schedule, schedule, discount, settlement_date, notional, 0.03, forwarding=forwarding)
print('CB dirty value:', cb_price)
print('FRN dirty value:', frn_price)
print('Payer swap value:', swap_price, 'Par fixed rate:', par_rate)
np.testing.assert_allclose(frn_price, notional, atol=1e-10)
np.testing.assert_allclose(swap_price, frn_price - cb_price, atol=1e-10)
display(cb_table)
display(frn_table)

print('Swap fixed leg (paid)')
display(fixed_table)
print('Swap floating leg (received)')
display(floating_table)
```

12.8.4 Optional export

Keep file input/output in the notebook. Convert QuantLib dates into ISO strings before exporting so the table is portable. Rerunning this cell replaces only this example's output workbook.

```
# Uncomment when you want an Excel output.
# export_table = cb_table.copy()
# for column in ['Start Date', 'End Date', 'Pay Date']:
#     export_table[column] = export_table[column].map(lambda date: date.ISO())
# with pd.ExcelWriter(ROOT / 'pricing_example.xlsx', engine='openpyxl', mode='w') as writer:
#     pd.DataFrame({'Dirty Value': [cb_price]}).to_excel(writer, sheet_name='CB', index=False)
#     export_table.to_excel(writer, sheet_name='CB', index=False, startrow=3)
```

12.9 Original reference modules and synthetic input

The modules below are the supplied reference implementation, not generated solutions. Their source files and the CSV retain their original bytes and relative locations. Code comments and docstrings remain part of the original listing. Follow the input/output conventions in Section 9 when completing the student TODOs.

12.9.1 curves.py

```
"""Date-based discount curves for the Fixed Income lab.

Calendar dates are QuantLib Date objects (ql.Date), for example
ql.Date(19, 10, 2026): day, month, year. Strings, datetime.date,
pandas.Timestamp and numpy.datetime64 are not the calendar-date interface
of this library; convert them to ql.Date before calling these functions.

Curve pillar times are different: they are numerical year fractions,
not calendar dates. The curve's day count converts ql.Date into times.
```

```
A curve is a callable: discount(one_ql_date) returns one dimensionless
discount factor from the curve reference date to that date. Pricing can
use either this function or a compatible QuantLib curve's discount method.
File reading and plotting belong in the notebooks, not in this module.
"""
import numpy as np


def make_discount_curve(
    times,
    discount_factors,
    reference_date,
    day_count=None,
):
    """Construct a curve by linear interpolation of log discount factors.

    Parameters
    ----------
    times : one-dimensional array-like of real numbers
        Pillar times in years from reference_date, calculated with day_count.
        Accepts a list, tuple, NumPy array or pandas Series. Must be finite,
        start at 0, increase strictly and contain at least two entries.
    discount_factors : one-dimensional array-like of real numbers
        Dimensionless discount factors corresponding to times, in the same
        order and with the same length. Values must be finite and positive.
        The first value must equal 1 within absolute tolerance 1e-12.
    reference_date : ql.Date
        Origin of the curve: discount(reference_date) equals 1.
    day_count : QuantLib DayCounter, optional
        Convention used to compute times from calendar dates. Defaults to
        ql.Actual365Fixed(). It may differ from the coupon day count.

    Returns
    -------
    discount : callable
        Function discount(date), accepting ONE ql.Date and returning a Python
        float: the dimensionless discount factor from reference_date to date.
        Use get_discount_factors(discount, dates) for several ql.Date values.

    Notes
    -----
    The input arrays are copied. Later changes to the caller's arrays do not
    update the curve. Include the anchor (0, 1) before calling this function;
    it is not added here. Discount factors above one are allowed, so negative
    rates are not excluded. Requested dates must lie inside the pillar range;
    extrapolation is disabled. No conversion from other date formats is made.

    Raises
    ------
    ValueError
        If pillar dimensions, values, ordering or anchor are invalid, or if
        the returned curve is queried outside its range.
    """
    if day_count is None:
        import QuantLib as ql
        day_count = ql.Actual365Fixed()
    times = np.array(times, dtype=float, copy=True)
    factors = np.array(discount_factors, dtype=float, copy=True)
    if times.ndim != 1 or factors.ndim != 1 or len(times) != len(factors) or len(times) < 2:
        raise ValueError('Supply matching one-dimensional arrays with at least two pillars.')
    if not np.all(np.isfinite(times)) or not np.all(np.isfinite(factors)):
        raise ValueError('Curve pillars must be finite.')
    if times[0] != 0 or np.any(np.diff(times) <= 0):
        raise ValueError('Times must start at zero and be strictly increasing.')
    if not np.isclose(factors[0], 1.0, rtol=0, atol=1e-12) or np.any(factors <= 0):
        raise ValueError('Discount factors must be positive and P(t,t) must equal one.')
    factors[0] = 1.0
    log_factors = np.log(factors)

    def discount(date):
        """Return the discount factor for one calendar date.

        Parameters
        ----------
        date : ql.Date
            Requested date, including reference_date, within the curve's range.

        Returns
        -------
        float
            Dimensionless discount factor from reference_date to date. This is
            not a zero rate and is not rebased to a different settlement date.
        """
        time = day_count.yearFraction(reference_date, date)
        if time < times[0] or time > times[-1]:
            raise ValueError('Requested date is outside the curve range; extrapolation is disabled.')
        return float(np.exp(np.interp(time, times, log_factors)))

    return discount


def get_discount_factors(
    discount,
    dates,
):
    """Evaluate a date-based curve on a sequence of calendar dates.
```

```
    Parameters
    ----------
    discount : callable
        Function accepting one ql.Date and returning one numerical discount
        factor. For example, the result of make_discount_curve.
    dates : iterable of ql.Date
        List, tuple, one-dimensional NumPy object array, pandas Series, or
        another iterable containing ql.Date objects. These are calendar dates,
        not year fractions or date strings. Example:
        [ql.Date(19, 4, 2027), ql.Date(19, 10, 2027)].

    Returns
    -------
    numpy.ndarray
        One-dimensional floating-point array of dimensionless discount
        factors, with one entry per supplied date, in the same order.
        An empty iterable returns an empty array. Range restrictions are
        those of the supplied curve. Input dates are not modified.
    """
    return np.array([discount(date) for date in dates], dtype=float)
```

12.9.2 FI_library.py

```
"""Schedules and transparent pricing for the Fixed Income lab.

INPUT AND OUTPUT CONVENTIONS
All calendar dates, including dates inside DataFrames, are ql.Date objects.
Example: ql.Date(19, 10, 2026) means 19 October 2026 (day, month, year).
Convert strings, datetime.date, pandas.Timestamp or numpy.datetime64 before
calling the library. Numerical times and year fractions are not dates.

Rates, fixings and spreads are annual decimal rates: 0.03 means 3%, and
10 / 10000 means 10 basis points. Notionals and monetary outputs use the
same currency units. Numerical scalar outputs may be NumPy floating-point
scalars; they can be used directly in arithmetic, printing and pandas.

discount(date) is a callable from one ql.Date to a numerical discount
factor D(reference_date, date). forwarding(date) has the same date-based
interface, but supplies projection factors for floating-rate calculation.
These are separate roles, even when the same curve is passed to both.

SCHEDULE FORMAT
A schedule is a pandas DataFrame with one row per future coupon payment,
in chronological order. The schedule builder returns a 1-based row index.
Pricing uses row order; it does not require those particular index labels.
    'Start Date'       ql.Date: adjusted accrual-period start.
    'End Date'         ql.Date: adjusted accrual-period end.
    'Pay Date'         ql.Date: payment date, equal to End Date here.
    'YearFrac'         float: full contractual coupon year fraction.
    'Elapsed YearFrac' float: accrual from start to settlement, or zero
                           for a period starting on/after settlement.
Pricing requires the date/year-fraction columns it uses. Elapsed YearFrac
is supplied for accrued-interest calculations in the notebook.

PRICING TABLE FORMAT
Pricing returns a copy of the supplied schedule with additional columns:
    'Coupon Rate'      annual decimal rate, including any spread.
    'Discount Factor'  D(reference, payment) / D(reference, settlement).
    'Cash Flow'        currency amount due at payment.
    'Disc. Cash Flow'  Cash Flow times Discount Factor.
Floating tables also contain 'Benchmark Rate' and 'Spread', both annual
decimal rates. Date columns remain ql.Date objects; numeric columns contain
numerical values. Input schedules are not modified by the public pricers.

TEACHING SCOPE
Regular coupons; adjusted accrual dates; payment at accrual end; no fixing
or payment lag, ex-coupon period, amortisation, caps/floors or credit model.
Floating fixings are observed at accrual start. Values are currency amounts
at settlement, excluding payments on settlement. Bond values are dirty;
accrued interest and clean values are calculated separately in notebooks.
"""
import numpy as np
import pandas as pd
from curves import get_discount_factors


def get_schedule_dataframe(
    maturity_date,
    settlement_date,
    tenor,
    day_count=None,
    calendar=None,
    day_rolling=None,
    end_of_month=False,
    effective_date=None,
):
    """Build regular coupon periods and retain payments after settlement.

    Parameters
    ----------
    maturity_date : ql.Date
        Contractual maturity, before business-day adjustment.
    settlement_date : ql.Date
```

```
        Pricing settlement date; payments on this date are excluded.
    tenor : ql.Period
        Positive coupon interval, e.g. ql.Period(3, ql.Months) for quarterly
        coupons or ql.Period(1, ql.Years) for annual coupons.
    day_count : QuantLib DayCounter, optional
        Coupon year-fraction convention; defaults to ql.Actual360().
    calendar : QuantLib Calendar, optional
        Business-day calendar; defaults to ql.TARGET().
    day_rolling : QuantLib business-day convention constant, optional
        For example ql.ModifiedFollowing, which is the default. This is a
        QuantLib constant, not a string such as 'Modified Following'.
    end_of_month : bool, optional
        QuantLib end-of-month scheduling flag; defaults to False.
    effective_date : ql.Date or None, optional
        Actual contractual start, required for a forward-starting contract.
        If None, assume an existing regular contract and recover its current
        coupon period backwards from maturity. The temporary lookback used
        internally is not the contract's original effective date.

    Returns
    -------
    pandas.DataFrame
        Schedule in the module's documented format: three ql.Date columns
        and two numerical year-fraction columns, with a 1-based row index.
        Retains the full current period, even if its start precedes settlement,
        and later periods whose adjusted payment dates follow settlement.

    Raises
    ------
    ValueError
        If maturity does not follow settlement, tenor is nonpositive,
        effective date does not precede maturity, a relevant contractual stub
        is encountered, or no positive future coupon periods remain.

    Notes
    -----
    Only regular contractual periods are supported. Business-day adjustments
    and different month lengths can still produce unequal year fractions.
    """
    import QuantLib as ql
    day_count = ql.Actual360() if day_count is None else day_count
    calendar = ql.TARGET() if calendar is None else calendar
    day_rolling = ql.ModifiedFollowing if day_rolling is None else day_rolling
    if settlement_date >= maturity_date or tenor.length() <= 0:
        raise ValueError('Maturity must follow settlement and tenor must be positive.')
    # Two periods of lookback recover the current period; the temporary
    # starting boundary is not an issue date and its first period is past.
    schedule_start = (settlement_date - ql.Period(2 * tenor.length(), tenor.units())
                      if effective_date is None else effective_date)
    if schedule_start >= maturity_date:
        raise ValueError('Effective date must precede maturity.')
    schedule = ql.Schedule(schedule_start, maturity_date, tenor, calendar,
                           day_rolling, day_rolling, ql.DateGeneration.Backward,
                           end_of_month)
    dates = list(schedule)
    rows = []
    for i, (start, end) in enumerate(zip(dates[:-1], dates[1:]), start=1):
        if not schedule.isRegular(i) and (effective_date is not None or end > settlement_date):
            raise ValueError('Only regular coupon periods are supported; check contract dates.')
        if end > settlement_date:
            rows.append({'Start Date': start, 'End Date': end, 'Pay Date': end,
                         'YearFrac': day_count.yearFraction(start, end),
                         'Elapsed YearFrac': day_count.yearFraction(start, settlement_date)
                         if start < settlement_date else 0.0})
    table = pd.DataFrame(rows)
    if table.empty or (table['YearFrac'] <= 0).any():
        raise ValueError('No positive future coupon periods.')
    table.index = np.arange(1, len(table) + 1)
    return table


def _check_table(
    table,
    settlement_date,
):
    """Check the minimum schedule conditions used by the pricers.

    Parameters
    ----------
    table : pandas.DataFrame
        Schedule in the module's format; 'Pay Date' contains ql.Date objects
        and 'YearFrac' contains numerical year fractions.
    settlement_date : ql.Date
        Payments must occur strictly after this date.

    Returns
    -------
    None
        Raises ValueError for an empty table, nonfinite/nonpositive coupon
        year fractions, or a payment on/before settlement. This helper is not
        a complete contractual or date-type validator.
    """
    if table.empty:
        raise ValueError('Schedule must contain future payments.')
    tau = table['YearFrac']
    if not np.isfinite(tau).all() or (tau <= 0).any():
```

```
        raise ValueError('Coupon year fractions must be finite and positive.')
    if any(date <= settlement_date for date in table['Pay Date']):
        raise ValueError('Supply only payments strictly after settlement.')


def _discount_cash_flows(
    table,
    cash_flows,
    discount,
    settlement_date,
):
    """Add discounted cash flows to an internal working table.

    Parameters
    ----------
    table : pandas.DataFrame
        Working table owned by the calling pricer; Pay Date values are ql.Date.
        Unlike public pricers, this internal helper modifies its table.
    cash_flows : one-dimensional numerical array-like or pandas.Series
        Currency amounts, one per table row. A Series must have matching
        index labels; an array is assigned by position.
    discount : callable
        Accepts one ql.Date and returns a discount factor. Must cover both
        settlement and every payment date.
    settlement_date : ql.Date
        Date to which the cash flows are valued.

    Returns
    -------
    value : floating-point scalar
        Sum of discounted cash flows, in currency units at settlement.
    table : pandas.DataFrame
        The same working table, with Discount Factor, Cash Flow and
        Disc. Cash Flow columns added. Discount factors are rebased to
        settlement by dividing by discount(settlement_date).
    """
    factors = get_discount_factors(discount, table['Pay Date']) / discount(settlement_date)
    table['Discount Factor'] = factors
    table['Cash Flow'] = cash_flows
    table['Disc. Cash Flow'] = factors * table['Cash Flow']
    return table['Disc. Cash Flow'].sum(), table


def get_price_cb(
    schedule,
    discount,
    settlement_date,
    notional,
    rate,
):
    """Return a fixed-rate bond's dirty settlement value and cash flows.

    Parameters
    ----------
    schedule : pandas.DataFrame
        Regular coupon schedule in the module's format. Calendar-date
        columns contain ql.Date; YearFrac contains full coupon year fractions.
    discount : callable
        One ql.Date -> one dimensionless discount factor. Must cover
        settlement and the payment dates.
    settlement_date : ql.Date
        Payment/delivery date at which the bond value is expressed.
    notional : positive real scalar
        Principal in currency units, repaid with the last coupon.
    rate : real scalar
        Annual fixed coupon rate as a decimal, e.g. 0.03 for 3%.

    Returns
    -------
    dirty : floating-point scalar
        Dirty value in currency units at settlement. For a price per 100 of
        principal, compute 100 * dirty / notional. This is not a clean price.
    table : pandas.DataFrame
        Copy of schedule with the pricing columns documented at module level.
        The last Cash Flow includes both coupon and principal repayment.
        The input schedule is not modified.
    """
    _check_table(schedule, settlement_date)
    table = schedule.copy()
    table['Coupon Rate'] = rate
    cash_flows = notional * rate * table['YearFrac']
    cash_flows.iloc[-1] += notional
    return _discount_cash_flows(table, cash_flows, discount, settlement_date)


def get_floating_rates(
    schedule,
    forwarding,
    settlement_date,
    current_fixing=None,
):
    """Use observed benchmark fixings or project future benchmark rates.

    Parameters
    ----------
    schedule : pandas.DataFrame
```

```
        Schedule with Start Date, End Date and Pay Date as ql.Date objects
        and YearFrac as numerical contractual coupon year fractions.
    forwarding : callable
        One ql.Date -> one numerical projection factor. For future periods,
        the rate is (forwarding(start) / forwarding(end) - 1) / YearFrac.
    settlement_date : ql.Date
        Determines which reset periods have already started in this lab.
    current_fixing : real scalar or None, optional
        Observed annual benchmark rate BEFORE spread, in decimal form.
        Required if a retained period starts before settlement. If a period
        starts exactly on settlement, use this fixing when supplied; otherwise
        project it. This interface supports one current fixing, not a history
        indexed by fixing dates. Future-starting periods are projected.

    Returns
    -------
    numpy.ndarray
        One-dimensional floating-point array of annual benchmark rates,
        BEFORE spread, in schedule row order. Contains one rate per period.

    Raises
    ------
    ValueError
        If the schedule fails basic validation or a started coupon has no
        supplied observed fixing. Historical rates are not inferred from
        today's curve. No input table is modified.
    """
    _check_table(schedule, settlement_date)
    rates = []
    for start, end, tau in zip(
            schedule['Start Date'], schedule['End Date'], schedule['YearFrac']):
        if start < settlement_date:
            if current_fixing is None:
                raise ValueError('A started coupon period requires its observed benchmark fixing.')
            rate = current_fixing
        elif start == settlement_date and current_fixing is not None:
            rate = current_fixing
        else:
            rate = (forwarding(start) / forwarding(end) - 1.0) / tau
        rates.append(rate)
    return np.array(rates, dtype=float)


def get_price_frn(
    schedule,
    discount,
    settlement_date,
    notional,
    spread=0.0,
    current_fixing=None,
    forwarding=None,
):
    """Return an FRN's dirty settlement value and cash-flow table.

    Parameters
    ----------
    schedule : pandas.DataFrame
        Regular coupon schedule in the module's format; all calendar-date
        columns contain ql.Date objects.
    discount : callable
        One ql.Date -> one numerical discount factor, used to value payments.
    settlement_date : ql.Date
        Date at which the returned currency value is expressed.
    notional : positive real scalar
        Principal in currency units; repaid with the last coupon.
    spread : real scalar, optional
        Annual contractual spread in decimal form, added once to every
        benchmark fixing. Defaults to 0.0; 0.001 means 10 basis points.
    current_fixing : real scalar or None, optional
        Observed annual benchmark rate BEFORE spread; see get_floating_rates.
    forwarding : callable or None, optional
        One ql.Date -> one projection factor. Used only for future benchmark
        rates. If None, use discount (single curve). To make the two roles
        explicit, pass forwarding=forwarding even when it equals discount.

    Returns
    -------
    dirty : floating-point scalar
        Dirty settlement value in currency units, not a clean price.
    table : pandas.DataFrame
        Copy of schedule with pricing columns, Benchmark Rate and Spread.
        Coupon Rate equals Benchmark Rate plus Spread. The final Cash Flow
        includes principal repayment. Date columns remain ql.Date objects;
        the input schedule is not modified.
    """
    forwarding = discount if forwarding is None else forwarding
    rates = get_floating_rates(schedule, forwarding, settlement_date, current_fixing)
    table = schedule.copy()
    table['Benchmark Rate'] = rates
    table['Spread'] = spread
    table['Coupon Rate'] = rates + spread
    cash_flows = notional * table['Coupon Rate'] * table['YearFrac']
    cash_flows.iloc[-1] += notional
    return _discount_cash_flows(table, cash_flows, discount, settlement_date)
```

```
def get_price_swap(
    fixed_schedule,
    floating_schedule,
    discount,
    settlement_date,
    notional,
    fixed_rate,
    spread=0.0,
    current_fixing=None,
    forwarding=None,
    payer=True,
):
    """Return swap value, par fixed rate and detailed tables for both legs.

    Parameters
    ----------
    fixed_schedule, floating_schedule : pandas.DataFrame
        Separate regular schedules in the module's format. All calendar-date
        columns contain ql.Date objects. Legs may have different frequencies
        and day counts. Construct them for the same contract start/maturity.
    discount : callable
        One ql.Date -> one numerical discount factor; discounts both legs.
    settlement_date : ql.Date
        Date at which the returned swap value is expressed.
    notional : positive real scalar
        Common principal used to calculate interest, in currency units.
        There is no principal exchange in this swap.
    fixed_rate : real scalar
        Annual fixed-leg rate in decimal form.
    spread : real scalar, optional
        Annual decimal spread added to each floating benchmark rate;
        defaults to 0.0.
    current_fixing : real scalar or None, optional
        Observed floating benchmark fixing BEFORE spread, if required by a
        started period; see get_floating_rates.
    forwarding : callable or None, optional
        One ql.Date -> one projection factor for future floating rates.
        Defaults to discount when omitted.
    payer : bool, optional
        True (default): pay fixed, receive floating. False: receive fixed,
        pay floating. Controls the sign of value, not the leg-table amounts.

    Returns
    -------
    value : floating-point scalar
        Currency value at settlement to the selected side. For payer=True,
        equals floating-leg PV minus fixed-leg PV; receiver has opposite sign.
    par_rate : floating-point scalar
        Annual decimal fixed rate making the swap value zero for the supplied
        curves, schedules, floating spread and any known fixing.
    fixed_table, floating_table : pandas.DataFrame
        Copies of the input schedules with pricing columns. Floating table
        also contains Benchmark Rate and Spread. No principal repayment is
        included. Leg cash flows do not incorporate payer/receiver signs;
        combine the leg PVs using the direction above. Date columns remain
        ql.Date objects. Input schedules are not modified.
    """
    _check_table(fixed_schedule, settlement_date)
    forwarding = discount if forwarding is None else forwarding
    rates = get_floating_rates(floating_schedule, forwarding, settlement_date, current_fixing)
    fixed = fixed_schedule.copy()
    floating = floating_schedule.copy()
    fixed['Coupon Rate'] = fixed_rate
    floating['Benchmark Rate'] = rates
    floating['Spread'] = spread
    floating['Coupon Rate'] = rates + spread
    fixed_flows = notional * fixed_rate * fixed['YearFrac']
    floating_flows = notional * floating['Coupon Rate'] * floating['YearFrac']
    fixed_pv, fixed = _discount_cash_flows(fixed, fixed_flows, discount, settlement_date)
    float_pv, floating = _discount_cash_flows(floating, floating_flows, discount, settlement_date)
    annuity = (fixed['YearFrac'] * fixed['Discount Factor']).sum()
    if notional <= 0:
        raise ValueError('Notional must be positive.')
    par_rate = float_pv / (notional * annuity)
    value = float_pv - fixed_pv
    return (value if payer else -value), par_rate, fixed, floating
```

12.9.3 Synthetic curve pillars

```
Time,Discount Factor
0,1.0
0.25,0.9937297627068441
0.5,0.9874262008903658
1,0.9747571290946985
2,0.9493798172437226
3,0.9242314761937627
5,0.8753677073117426
10,0.7639243445836288
```

12.9.4 Supplied environment requirements

```
numpy
```

```
pandas
QuantLib
matplotlib
jupyter
openpyxl
```

13 Function-design discussion and API reference

13.1 From a checked notebook calculation to a reusable function

After completing and checking an example, identify the few lines that actually price the contract. Separate file reading, chosen inputs, table display and plotting from that calculation. Agree on explicit inputs and outputs before transferring it to the module; a function should not depend on variables left in notebook memory.

Complete the design table after each worked example:

Calculation	What must go in?	What should come out?
Regular schedule		
Coupon bond price		
FRN price		
Swap value		

Discuss whether a pricer should build its schedule or receive it; whether it should read a curve file or receive a date-to-factor function; and which arguments project rates versus discount payments. Identify the already known fixing and whether it includes spread. State the output units, pricing date and meaning of a positive swap value. Decide whether a scalar price is enough or a detailed cash-flow table is needed to inspect it. Return values from the function and display them in the notebook.

Then choose clear argument names, transfer the validated calculation to its module and save it. Reimport/reload and check the original example reproduces its result. Change one input, predict the effect and rerun. A kernel restart followed by execution from the top checks that no hidden notebook state was required.

13.2 Interfaces already supplied

All calendar dates in these interfaces are QuantLib dates, including date columns in DataFrames. Rates are annual decimals and monetary outputs use the same currency unit as the notional. The financial interpretation is in Section 9; this table is an implementation map to the original source listings.

Function	Caller supplies	Function returns
`make_discount_curve`	Numerical pillar times and factors, reference date, optional curve day count	Callable `discount(date)` returning one factor
`get_discount_factors`	Discount callable and sequence of dates	NumPy array in the same date order
`get_schedule_dataframe`	Maturity, settlement, tenor; optional day count, calendar, rolling, end-of-month and effective date	DataFrame with start, end, payment dates, full coupon fractions and elapsed fractions
`get_floating_rates`	Schedule, projection callable, settlement and observed current fixing when required	Annual benchmark rates before spread, in schedule order
`get_price_cb`	Schedule, discount callable, settlement, notional and annual fixed coupon rate	Dirty settlement currency value and detailed table including final principal
`get_price_frn`	Schedule, discount callable, settlement, notional; optional spread, current fixing, projection callable	Dirty settlement value and table including final principal
`get_price_swap`	Separate fixed/floating schedules, discount callable, settlement, notional, fixed rate; optional spread, fixing, projection and direction	Signed value, par fixed rate, fixed and floating tables; no principal exchange

The schedule builder's one-based row index is a display convention; pricing follows row order. Public pricers return copies of input schedules with calculation columns added. Those columns include Coupon Rate, Discount Factor, Cash Flow and Disc. Cash Flow; floating tables also carry Benchmark Rate and Spread. The internal working-table helper may

modify its own table. Accrued interest and clean prices are calculated separately in the notebooks. The original docstrings specify arguments, defaults, returned types and stated validation scope in full.

13.3 Proposed later responsibilities

The source legend's second page proposes additional teaching blocks: market-curve bootstrap, portfolio valuation, benchmark sensitivities, swap hedge/scenarios, Black/Bachelier caplet and cap pricing, G1 analytical bond pricing, G1 simulation and Monte Carlo valuation. Their inputs, outputs and financial distinctions are integrated once in Section 9.6. They are not implemented functions in the supplied modules and are not additional homework requirements. Optional Nelson–Siegel–Svensson fitting and separate G1 calibration remain proposals.

14 Practical support and source-data workflow

14.1 Classroom workflow

Use the conceptual date/convention chain in Section 9. For each topic, explain and annotate, run and read, attempt the exercise, discuss and validate. The indicative classroom plan allocates 35–45 minutes to opening/questionnaire/materials, 40–45 to dates/TARGET/day rolling, 35–40 to day count and exercises, 10 to a break, 35–45 to compounding and exercises, remaining time to coupon schedules, and 5 to discussion. Coupon schedules are useful but optional for that session; LSEG analysis follows the introduction of forward rates.

Exercise map. The supplied six-exercise sheet repeats the notebook tasks with a different numbering for schedules and equivalent rates. Each full request appears once in Section 11; the missing 1Y observation, numerical validation and curve-shape results are explained in Section 9.2. The equivalent-rate output for the five-year 0.85 factor remains beside its original code. No second copy of those requests is appended.

14.2 LSEG export workflow supplied for classroom verification

The source labels this a *provisional* workflow: command names and positions must be verified in the installed Workspace version.

1. Open Workspace and search for Curve Builder.
2. Select EUR, EURIBOR, 6M tenor, deposits and IRS versus EURIBOR 6M.
3. Set/verify 24 August 2026 to reproduce this dataset.
4. Inspect short, intermediate and long constituents, including deposits, EURIBOR futures and swaps.
5. Record price side, discounting tenor/collateral, interpolation, extrapolation, zero-rate day count and compounding.
6. Locate the zero-curve results; inspect tenor, maturity date, zero rate, discount factor and instrument identifier where available.
7. Use the available export/download/Excel command; verify its exact label.
8. Save with a transparent name, e.g. `lseg_euribor_6m_zero_curve_2026-08-24.xlsx`.
9. Retain the unmodified export and worksheet names under the lab's `data/raw` directory (the source course path is `01-dates-and-conventions/data/raw`).

The six supplied verification items are: exact application/view name, exact curve description, valuation-date control, results-table location/name, export command/workbook structure, and conventions/settings. The source deliberately postpones forward-curve analysis and export until forward rates have been introduced.

14.3 NumPy array syntax

The expressions below are reference snippets rather than one executable program. Choose compatible array shapes; individual examples need their own inputs.

```
a = np.array([1, 2, 3]) -- 1D array
b = np.array([[1, 2], [3, 4]]) -- 2D array (matrix)
```

```
np.zeros(5) → [0. 0. 0. 0. 0.]
np.ones((2,3)) → 2x3 matrix of ones
np.eye(3) → identity matrix
np.arange(0, 10, 2) → [0 2 4 6 8]
np.linspace(0, 1, 5) → [0. 0.25 0.5 0.75 1.]
```

```
a.shape -- (n,) or (m,n)
a.ndim -- number of dimensions
a.size -- total number of elements
a.dtype -- data type (int, float, etc.)
```

```
a[0] -- first element
a[-1] -- last element
a[1:4] -- slice
a[:3] -- first three
a[::2] -- every 2nd element
```

```
b[0,1] -- element (0,1)
b[:,1] -- 2nd column
b[1,:] -- 2nd row
b[0:2,0:2] -- top-left 2x2 block
```

```
a + b -- elementwise addition
a * b -- elementwise multiplication
a @ b -- matrix product
a.T -- transpose
np.dot(a,b) -- dot product
```

```
np.sqrt(a), np.exp(a), np.log(a)
np.sum(a), np.mean(a), np.max(a)
```

```
a.reshape(3,2)
np.hstack([a,b]) -- horizontal
np.vstack([a,b]) -- vertical
np.concatenate([a,b],axis=0)
```

```
mask = a > 2
a[mask] → filtered elements
a[a % 2 == 0] → even numbers
```

14.4 Pandas syntax

These source examples illustrate separate operations. Define the DataFrames used in each fragment and run only the operations appropriate to the current columns.

```
import pandas as pd
```

```
data = {"Time": [0.5, 1.0, 1.5, 2.0],
        "Rate": [0.02, 0.025, 0.027, 0.03]}
df = pd.DataFrame(data)
```

```
df = pd.read_excel("file.xlsx", sheet_name="Sheet1")
df = pd.read_csv("file.csv")
df.to_excel("output.xlsx", index=False)
df.to_csv("output.csv", index=False)  # no index column in file
```

```
# Columns
df["Rate"]            # Series
df[["Time","Rate"]]   # DataFrame

# Rows by label/position
df.loc[1]             # label
df.iloc[0]            # position

# Rows + columns
df.loc[0:2, ["Time","Rate"]]
df.iloc[0:2, 0:2]
```

```
df["Discount"] = 1 / (1 + df["Rate"] * df["Time"])  # add/modify
df = df.drop(columns=["Rate"])                      # drop column
df = df.drop(index=[0])                             # drop row
df = df.rename(columns={"Time":"Maturity"})         # rename
```

```
df["Forward"] = df["Rate"] + 0.001    # elementwise
df_sorted = df.sort_values("Rate", ascending=False)
df_filt = df[df["Rate"] > 0.025]
```

```
new_row = pd.DataFrame([[2.5, 0.031]], columns=["Time","Rate"])
df2 = pd.concat([df, new_row], ignore_index=True)
merged = pd.merge(df1, df2, on="Time", how="inner")  # joins: inner/left/right/outer
```

```
df.info()       # structure & dtypes
df.describe()   # stats
df.head(5)      # first rows
df.tail(3)      # last rows
```

```
# loc uses labels; iloc uses integer positions
# Use df.reset_index(drop=True) to reset to 0..n-1
# Set index: df.set_index("Time", inplace=True)
```

```
# CSV options
pd.read_csv("f.csv", usecols=["Time","Rate"])
# Excel writer overwrite (single sheet)
with pd.ExcelWriter("book.xlsx", engine="openpyxl", mode="a", if_sheet_exists="replace") as w:
    df.to_excel(w, sheet_name="Sheet1", index=False)
```

14.5 Jupyter: two modes and recovery

Escape enters Command Mode (manage cells); Enter enters Edit Mode (write text). In Command Mode, use the letter alone, not Ctrl plus the letter.

Keys	Action
Shift+Enter / Ctrl+Enter	Run and move to next cell / run and remain.
Alt+Enter / Ctrl+S	Run and insert below / save.
A / B	Insert above / below (Command Mode).
D,D / Z	Delete selected cells / undo cell deletion.
X / C / V / Shift+V	Cut / copy / paste below / paste above.
M / Y / R	Change cell to Markdown / Code / Raw.
Up/Down; Shift+Up/Down	Move; extend selection.
Shift+M / L	Merge selected cells / toggle line numbers.
Tab / Shift+Tab	Indent or completion / object or function information (Edit Mode).
Ctrl+/; Ctrl+] / Ctrl+[	Toggle comment; indent / dedent lines.
Ctrl+Z / Ctrl+Y / Ctrl+A	Undo / redo text edits / select cell text.
I,I / 0,0 / H	Interrupt / restart kernel / shortcuts help (Command Mode).

If a cell is stuck, Escape then I,I; if needed restart. Restarting clears variables, so rerun required cells from the top. Save before restarting or closing. Shortcuts can vary across Notebook/JupyterLab versions; consult Help or H for the installed version.

A Course roadmap

The supplied syllabus organises the course around the following topics:

- spot and forward contracts, including the taxonomy of rates;
- benchmark market rates: the LIBOR environment before and after the 2007 credit crisis and the reform from LIBOR to risk-free rates (RFRs);
- change of numeraire and change of measure;
- linear interest-rate derivatives: FRAs, futures, and swaps;
- bootstrapping and interpolation of the interest-rate curve;
- interest-rate options: bond options, caps and floors, and swaptions;
- market models, including Black and Bachelier pricing formulas;
- short-rate models, including Vasicek, CIR, and the one-factor Hull–White model;
- an overview of Heath–Jarrow–Morton models, an introduction to RFR modelling when time permits, and interest-rate-risk hedging.

Pricing and modelling are developed primarily in a single-curve framework. The distinction between discounting and forwarding curves and the main features of RFR-based instruments are introduced; multicurve and further RFR extensions are discussed as time permits.

A.1 Computational applications and homework

Guided Python examples and homework cover selected applications: curve bootstrapping and interpolation; coupon-bond and floating-rate-note pricing; transformation of cap and swaption volatility quotes into Black/Bachelier market prices; and Hull–White calibration, option pricing, and Monte Carlo simulation.

A.2 Assessment

The maximum grade is 30/30 cum laude. The written examination accounts for 75% and assesses theoretical foundations plus analytical and computational problem solving. A compulsory computational group project with an individual oral defence accounts for 25%. The defence checks understanding, individual contribution, and the ability to interpret and critically discuss the results. Detailed criteria and deadlines are published on Moodle. The supplied syllabus lists five sessions (December, January, February, June, and September) and permits up to three examination attempts during the academic year.

Outstanding project work may be taken into account in determining the final grade and, where appropriate, in the award of cum laude. Participation in the winter examination session is strongly encouraged.

A.3 References

- D. Brigo and F. Mercurio, *Interest Rate Models: Theory and Practice With Smile, Inflation and Credit*, Springer, 2006.
- M. Henrard, *Interest Rate Modelling in the Multi-Curve Framework: Foundations, Evolution, Transition, and Implementation*, 2nd ed., Springer, 2026.
- T. Björk, *Arbitrage Theory in Continuous Time*, Oxford University Press, 2009.
- J. Kienitz, *Interest Rates Explained*, vols. 1–2, Palgrave, 2013 and 2017.